

Noname manuscript No.
(will be inserted by the editor)

Empirical Studies of Parameter Efficient Methods for Large Language Models of Code and Knowledge Transfer to R

Amirreza Esmaeili · Iman Saberi · Fatemeh Fard

Received: date / Accepted: date

Abstract Parameter Efficient Fine-Tuning (PEFT) methods are proposed as an alternative fine-tuning approach for Large Language Models (LLM) to minimize high training costs. While prior research demonstrates the effectiveness of PEFT methods in knowledge transfer using smaller language models, their application to larger LLMs, particularly in low-resource and unseen programming languages such as R, remains under-explored. In this work, we evaluate PEFT methods, LoRA, Compacter, and IA³ on LLMs for code summarization and generation, with a particular emphasis on knowledge transfer to R as an unseen under-explored target language. Our experiments reveal that LoRA consistently outperforms Compacter and IA³ in all settings, while Compacter offers significant resource efficiency with minimal performance trade-offs. Additionally, we find that the number of trainable parameters has a greater influence on the functional accuracy of the generated code than PEFT architecture. Our study can direct future research in developing code intelligent tasks for unseen languages including R, as well as the choice of PEFT methods for knowledge transfer, especially when balancing the computational cost and performance.

This research is supported by a grant from Natural Sciences and Engineering Research Council of Canada RGPIN-2019-05175 and Mitacs Accelerate award, 2024.

Amirreza Esmaeili
University of British Columbia
3333 University Way, Kelowna, BC V1V 1V7, Canada
E-mail: a.esmaeili@ubc.ca

Iman Saberi
University of British Columbia
3333 University Way, Kelowna, BC V1V 1V7, Canada
E-mail: iman.saberi@ubc.ca

Fatemeh Fard
University of British Columbia
3333 University Way, Kelowna, BC V1V 1V7, Canada
E-mail: fatemeh.fard@ubc.ca

本文档由 funstory.ai 的开源 PDF 翻译库 BabelDOC v0.5.16 (http://yadt.io) 翻译，本仓库正在积极的建设当中，欢迎 star 和关注。

(由编辑插入)

Nonamemanuscript No.

大规模语言模型参数高效方法的实证研究：代码与知识迁移到R

Amirreza Esmaeili · Iman Saberi · Fatemeh Fard

收到：日期 / 接受：日期

摘要参数高效微调（PEFT）方法被提出作为大型语言模型（LLM）的一种替代微调策略，以降低高昂的训练成本。尽管先前的研究证明了PEFT方法在较小语言模型中的知识迁移效果，但其在更大规模LLM中的应用，尤其是在低资源和未见过的编程语言如R中的研究仍然有限。在本工作中，我们评估了LoRA、Compacter和IA³等PEFT方法在代码总结与生成任务中的表现，特别强调其在未见过的目标语言R中的知识迁移能力。实验结果显示，LoRA在所有设置中均优于Compacter和IA³，而Compacter则以较低的资源消耗实现了较小的性能折衷。此外，我们发现可训练参数的数量对生成代码的功能准确性影响大于PEFT架构本身。我们的研究可以指导未来在未见语言（包括R）中的代码智能任务开发，以及在平衡计算成本与性能时选择合适的PEFT方法。

本研究得到加拿大自然科学与工程研究委员会（Natural Sciences and Engineering Research Council of Canada）RGPIN-2019-05175项目和Mitacs Accelerate奖（2024）的资助。

艾米雷扎·埃斯迈伊利 加拿大不列颠哥伦比亚大学
加拿大卑诗省凯洛纳大学路3333号，邮编V1V1V7
电子邮箱：a.esmaeili@ubc.ca

伊曼·萨贝里 不列颠哥伦比亚大学 加拿大卑诗省凯洛纳市大学大道3333号 V1V1V7 E-mail: iman.saberi@ubc.ca

Fatemeh Fard 不列颠哥伦比亚大学 加拿大卑诗省凯洛纳 University Way 3333 邮政编码 V1V1V7
E-mail: fatemeh.fard@ubc.ca

Keywords Parameter Efficient Fine Tuning, Code Large Language Models, Knowledge Transfer, Unseen Languages, Low-Resource Language, R.

1 Introduction

Large Language Models (LLMs) such as T5 model [50], Llama2 [64], GPT-3 [7], CodeT5 [68] and CodeLlama [52] have achieved the state-of-the-arts performance in many natural language and software engineering tasks, including code summarization [68] and code generation [68, 52]. Considering the heavy computational cost of the models, Parameter Efficient Fine-Tuning (PEFT) methods are used to tune a small percentage of the models’ parameters to a downstream task. There are various studies [53, 54, 71, 36, 66, 67] that explore the application of PEFT methods such as Adapter [24], LoRA [25], Prompt Tuning [28] and IA³ [35] in software engineering. Some of these works [29, 36, 20] employ smaller models (less than 1B parameters) such as CodeBERT [18] and GraphCodeBERT [21], while others are specifically targeting larger models (more than 1B parameters) [71] in a diverse set of software engineering tasks such as code search, code summarization, clone detection, defect detection, and code generation.

Previous research has shown the advantage of using PEFT methods in knowledge transfer from natural language to code [20, 54] using smaller models CodeBERT [18] and GraphCodeBERT [21], and to low-resource languages, even outperforming full fine-tuning of the language models [54]. Other studies [10, 9, 53] cover knowledge transfer to the domain of *low-resource* and unseen languages, where training data is limited and the base model has not encountered the target language. However, there is little known about how PEFT methods perform when they are applied to LLMs, especially for knowledge transfer from natural language to code and also how they perform on R. Moreover, within all studies on PEFT methods in the software engineering domain to this date, the lack of an in-depth investigation of Compacter [39] is notable. Proposed by Mahabadi et al. [39], Compacter further improves the parameter efficiency of Adapters [24] by incorporating low-rank matrices and Kronecker multiplications.

Thus, we aim to assess the performance of the PEFT methods to determine their effectiveness in knowledge transfer from natural language to code, and whether they can effectively serve as substitutes for conventional full fine-tuning, specifically in unseen and low-resource scenarios. For the unseen and low-resource programming language, we choose R. First, given that R is rarely included in the training of common LLMs, it serves as an ideal target language to assess the performance of PEFT methods on low-resource and unseen programming languages. Second, R is a widely adopted programming language with a large community and ecosystem, especially known for its applications in statistical computing and data visualization. While not as popular as languages such as Python or Javascript, R is ranked 18th on the TIOBE index¹ in November 2024 and 21st on Stack Overflow’s 2024 Developer Survey². Choosing R as a target language can ultimately lead

¹ <https://www.tiobe.com/tiobe-index>
² <https://survey.stackoverflow.co/2024/technology>

关键词 参数高效微调、代码大型语言模型、知识迁移、未见语言、低资源语言、R

1 引言

大型语言模型（LLMs）如T5模型 [50], Llama2 [64], GPT-3 [7],CodeT5 [68] 和 CodeLlama [52] 在许多自然语言处理和软件工程任务中实现了最先进的性能，包括代码摘要 [68] 和代码生成 [68, 52]。鉴于模型计算成本较高，参数高效微调（PEFT）方法被用来对模型的少部分参数进行调优，以适应下游任务。已有多项研究 [53, 54, 71, 36, 66, 67] 探索了PEFT方法在软件工程中的应用，如Adapter [24]、LoRA [25]、Prompt Tuning [28] 和IA³ [35]。其中一些工作 [29, 36, 20]采用较小的模型（参数少于1B），如CodeBERT [18]和GraphCodeBERT [21]，而另一些则专注于较大的模型（参数超过1B），应用于多样化的软件工程任务，如代码搜索、代码摘要、克隆检测、缺陷检测和代码生成。

先前的研究表明，在知识迁移方面，使用PEFT方法具有优势，从自然语言到代码 [20, 54]，利用较小的模型Code- BERT [18]和GraphCodeBERT [21]，甚至在低资源语言中，表现优于对语言模型的完全微调 [54]。其他研究 [10, 9, 53] 涉及到向低资源和未见过的语言领域进行知识迁移，在这些领域中，训练数据有限，基础模型未曾遇到目标语言。然而，关于PEFT方法在应用到LLMs时的表现知之甚少，尤其是在从自然语言到代码的知识迁移方面，以及它们在R上的表现。此外，在迄今为止所有关于软件工程领域中PEFT方法的研究中，缺乏对Compacter [39] 的深入调查尤为引人注目。由Mahabadi等人提出的 [39], Compacter通过引入低秩矩阵和 Kronecker乘积，进一步提高了Adapter [24] 的参数效率。

因此，我们旨在评估PEFT方法的性能，以确定其在知识迁移从自然语言到代码方面的有效性，以及它们是否能有效替代传统的全参数微调，特别是在未见和低资源场景中。对于未见和低资源的编程语言，我们选择R。首先，鉴于R在常用大型语言模型的训练中很少被包含，它作为评估PEFT方法在低资源和未见编程语言上的表现的理想目标语言。其次，R是一种广泛采用的编程语言，拥有庞大的社区和生态系统，特别以其在统计计算和数据可视化方面的应用而闻名。虽然不如Python或JavaScript等语言流行，但在2024年11月的TIOBE指数¹ 中，R排名第18位，在 Stack Overflow的2024开发者调查²中排名第21位。选择R作为目标语言最终可以

¹ <https://www.tiobe.com/tiobe-indexhttps://survey.stackoverflow.co/2024/technology>

to further adoption and a better understanding of the models’ capabilities in this language.

We investigate T5 [50] and Llama2 [64], commonly employed open-source natural language LLMs (that we refer to as general-LLM), and code large language models (that we refer to as code-LLMs), CodeT5 [68] and CodeLlama [52], fine-tuned with three PEFT techniques: Compacter, LoRA, and IA³. Due to the large number of experiments and computational restrictions, we consider code summarization and code generation, which are two well-known tasks in software and have been studied extensively in recent years [68, 52, 21, 18].

Through extensive empirical experiments, we demonstrate that LoRA consistently outperforms both Compacter and IA³ for code summarization and code generation. This includes adapting from natural language to code on general-LLMs, from code to natural language on code-LLMs, and transferring knowledge to R as an unseen language. Furthermore, we show that while Compacter incurs a relatively small performance trade-off, it offers significant resource efficiency, making it an attractive choice in scenarios where minimizing computational requirements is a priority. Finally, our findings highlight that the number of trainable parameters has a greater impact on functional accuracy in code generation than the specific architecture of PEFT methods.

Our main contributions are an in-depth analysis of PEFT comparison using LLMs, domain adaptation from natural language and code-specific LLMs to code-related tasks, and adapting LLMs to unseen language, R. We make all the scripts available to support open science. The link is found in Section 8.

Employing PEFT methods as an alternative fine-tuning process in code-related tasks can notably decrease both training time and resource usage, ultimately reducing the cost and environmental side-effects of training large language models, making the LLMs more accessible in cases where computational resources are scarce. The focus on the low-resource language of R helps in research using LLMs to automate software engineering tasks for a wider community of developers. Our research provides interesting results in this direction and can guide future research for code intelligence for R, as well as using the studied PEFT methods, specifically LoRA and Compacter with LLMs.

Note. Please note that this work is the submission of our accepted Registered Report in MSR 2024 Registered Report Track, titled "Empirical Studies of Parameter Efficient Methods for Large Language Models of Code and Knowledge Transfer to R". The link to the registered report can be found here.

Deviations from Registered Report. There are small deviations from the registered report in this manuscript as follows. We discuss the details of the methodology and each process in Section 3.

- We adopted the SPP dataset for Python code generation instead of the CoNaLa dataset, which offers better compatibility with the dataset used for R code generation and benchmarks used to assess code generation quality, in addition to a larger filtered training data sample size, 28,182 compared to 2,379 samples, which is more beneficial for large LLMs.

以促进模型在该语言中的进一步应用和更好地理解其能力。

我们研究了T5 [50] 和Llama2 [64], 常用的开源自然语言大模型（我们称之为通用大模型），以及代码大模型（我们称之为代码大模型），包括通过三种PEFT技术：Compacter、LoRA和IA³进行微调的CodeT5 [68] 和CodeLlama [52], 。由于实验数量众多且受到计算限制，我们选择了代码摘要和代码生成这两个在软件领域广泛研究的经典任务，这也是近年来被广泛关注的研究方向 [68, 52, 21, 18]。

通过大量实证实验，我们证明了LoRA在代码总结和代码生成任务中始终优于Compacter和IA³。这包括在通用大模型（general-LLMs）上实现从自然语言到代码的迁移、在代码大模型（code-LLMs）上实现从代码到自然语言的迁移，以及将知识迁移到R作为未见过的语言。此外，我们显示，虽然Compacter会带来相对较小的性能折衷，但其在资源效率方面具有显著优势，在优先考虑最小化计算资源的场景中具有吸引力。最后，我们的研究结果强调，训练参数的数量对代码生成中的功能准确性影响大于PEFT方法的具体架构。

我们的主要贡献包括对使用大型语言模型（LLMs）进行参数高效方法（PEFT）比较的深入分析、从自然语言和特定于代码的LLMs到代码相关任务的领域适应，以及将LLMs适应于未见过的语言R。我们提供了所有脚本以支持开放科学，相关链接见第8节。

在代码相关任务中采用PEFT方法作为一种替代微调方式，能够显著降低训练时间和资源消耗，最终减少训练大型语言模型的成本和环境影响，使得在计算资源有限的情况下，LLMs的应用更加普及。对低资源语言R的关注，有助于利用LLMs自动化软件工程任务的研究，惠及更广泛的开发者群体。我们的研究在这一方向上取得了有趣的成果，并可为未来R的代码智能研究提供指导，同时也为使用所研究的PEFT方法，特别是LoRA和Compacter与LLMs结合的应用，提供了参考。

注意。 请注意，本工作是我们**在MSR 2024注册报告轨道中已接受的注册报告**的提交，题目为“面向代码的大型语言模型参数高效方法的实证研究及知识迁移到**R**”。注册报告的链接可在此处查看。

偏离已注册报告。 本文稿中存在以下与已注册报告的小偏差。我们在第3节中讨论了方法学和每个过程的详细内容。

我们采用了SPP数据集进行Python代码生成，而非CoNaLa数据集，原因在于SPP数据集与用于R代码生成的数据集具有更好的兼容性，以及用于评估代码生成质量的基准。此外，SPP数据集的过滤后训练样本规模更大，为28,182个样本，而CoNaLa为2,379个样本，这对于大型语言模型（LLMs）更为有利。

- We opted out of implementing a new R code generation benchmark, as a high-quality alternative came into existence that adequately aligns with the HumanEval Python benchmark, resulting in more consistent assessments.
- We initially proposed two scenarios in RQ3, the first one being excluding the target unseen language from the model during training, i.e., excluded during the pre-training and fine-tuning phase. We eliminated this scenario due to two reasons. We conducted initial experiments with some of the models and languages and we observed small differences between these models and the base models with no fine-tuning. More importantly, due to several configurations for the models and PEFT methods that we considered to ensure the reliability of the results, the number of experiments would increase significantly. Therefore, due to computational resources and our initial experiments, we eliminated this scenario.
- We utilized a zero-shot setting as the baseline for Llama models in place of full fine-tuning, as the substantial time and computational resources required for fine-tuning of such models were beyond our computational capacity.
- We included IA³ as the third PEFT method with limited experiments, which shed light on this novel fine-tuning technique and better elaborate the attributes of other PEFT methods.

2 Background

2.1 Pre-trained Language Models

Traditionally, researchers were required to collect large amounts of task-related annotated data to train a model. With the advent of the Transformer architecture [65] and pre-training [14, 44, 48], language models are introduced; models that are pre-trained on a vast amount of unlabelled data enabling the model to obtain commonsense information about a language or multiple languages. Then, these models are fine-tuned on supervised datasets for different downstream tasks. NLP researchers have proposed several pre-trained language models such as BERT [16], T5 [50], and GPT-3 [7], serving as a checkpoint of language knowledge ready for further training.

Early contributions have shown promising results by utilizing models pre-trained on code and natural language in the SE domain, and achieved state-of-the-art results for several tasks such as vulnerability detection [19], code classification [73], code search [32], code summarization [68] and code generation [69]. Since the introduction of CodeBERT [18], several lanauge models are introduced in the SE domain, GraphCodeBERT [21], CodeT5 [68], CoTexT [46], AlphaCode [33], CodeX [12] and CodeLlama [52].

2.2 Parameter Efficient Fine-Tuning

Recent studies have demonstrated the feasibility of modifying or adding a relatively small subset of parameters [24], in contrast with the standard full fine-

我们放弃了实施新的 R 代码生成基准测试，因为出现了一种高质量的替代方案，能够与 HumanEval Python 基准测试充分对齐，从而实现更一致的评估。

我们在RQ3中最初提出了两种情境，第一种是在训练过程中排除目标未见语言，即在预训练和微调阶段排除该语言。我们由于两个原因放弃了这一情境。我们对部分模型和语言进行了初步实验，观察到这些模型与未经过微调的基础模型之间差异很小。更重要的是，为了确保结果的可靠性，我们考虑了多种模型和PEFT方法的配置，实验次数将显著增加。因此，考虑到计算资源和我们的初步实验，我们排除了这一情境。

我们采用零样本设置作为Llamamodels的基线，取代完全微调，因为对这类模型进行微调所需的大量时间和计算资源超出了我们的计算能力。

我们将IA³ 作为第三种参数高效微调方法纳入研究，进行了有限的实验，旨在阐明这一新颖的微调技术并更好地阐述其特性。其他PEFT方法。

s

2 背景

2.1 预训练语言模型

传统上，研究人员需要收集大量与任务相关的标注数据以训练模型。随着Transformer架构[65] 和预训练 [14, 44, 48], 语言模型的出现，模型在大量未标注数据上进行预训练，使其能够获得关于一种或多种语言的常识信息。随后，这些模型在监督数据集上进行微调，以适应不同的下游任务。自然语言处理领域的研究人员提出了多种预训练语言模型，如BERT [16]、T5 [50], 和GPT-3 [7],，作为语言知识的检查点，准备进行进一步的训练。

早期的研究表明，在软件工程领域利用在代码和自然语言上预训练的模型取得了令人鼓舞的成果，并在漏洞检测 [19]、代码分类 [73]、代码搜索 [32]、代码摘要 [68] 和代码生成 [69]等多个任务中达到了最新的技术水平。自从引入CodeBERT [18], 以来，软件工程领域相继推出了多种语言模型，包括GraphCodeBERT [21]、CodeT5 [68]、CoTexT [46]、AlphaCode [33]、CodeX [12]和CodeLlama [52]。

2.2 参数高效微调

近期研究已证明，相较于标准的全参数微调，修改或添加一小部分参数 [24], 的可行性。

tuning approach, which involves updating all parameters of a language model. Parameter-efficient fine-tuning significantly reduces memory and storage requirements during model training. State-of-the-art PEFT methods have demonstrated the capability to reach the performance achieved by fully fine-tuning a model by modifying a small percentage of the model's parameters. Additionally, PEFT methods can achieve better accuracy than fully fine-tuning a model in low-resource settings [53, 22], where labelled data for fine-tuning is scarce.

PEFT encompasses several methods, such as inserting small trainable feed-forward layers into a fixed pre-trained Transformer-based model [24], only updating the bias parameters while freezing the rest of a model's parameters [6], decomposing weight matrices into approximate low-rank matrices [25], introducing Kronecker multiplication with weight matrices and utilizing low-rank matrices [39], concatenating embedding weight matrices to fixed models [31, 28], and adding efficient element-wise multiplication to the feed-forward and attention layers of models to rescale weight matrices [35]. In this work, we will study LoRA [25], Compacter [39] and IA³ [35]. Though LoRA was previously experimented with within SE, the value of Compacter and IA³ is not well-known for SE tasks.

LoRA. Proposed by Hu et al. [25], LoRA draws inspiration from IntrinsicSAID [1] and introduces a more straightforward method for executing low-rank fine-tuning. In LoRA, the parameter update process for a weight matrix δW involves decomposing it into a product of two low-rank matrices W_A and W_B :

$$\delta W = W_A W_B \quad (1)$$

Where $W_A \in \mathbb{R}^{in \times r}$ and $W_B \in \mathbb{R}^{r \times out}$.

Notably, LoRA maintains the pre-trained model parameters in a frozen state, allowing only the W_A and W_B matrices to be trainable. The scaling factor is typically constant and often set to $1/r$. Post-training, these decomposed matrices— W_A and W_B —can be seamlessly integrated back into the original matrix W by a straightforward addition of the matrix product $W_A W_B$.

This method showcases notable efficacy in achieving improved performance while employing a simplified approach to low-rank fine-tuning, showcasing its applicability across extensive model architectures.

Compacter. Mahabadi et al. [39] proposed Compacter, which adopts a unique approach, leveraging the Kronecker product, low-rank matrices, and parameter sharing across layers to create adapter weights. Within each adapter, every W parameter is constructed as a sum of n Kronecker products of a set of smaller dimension matrices A_i and B_i :

$$\hat{W} = \sum_{i=0}^n A_i \otimes B_i \quad (2)$$

Where $\hat{W} \in \mathbb{R}^{k \times d}$, $A_i \in \mathbb{R}^{n \times n}$, $B_i \in \mathbb{R}^{\frac{k}{d} \times \frac{n}{d}}$.

This results in forming a linear layer referred to as the parametrized hyper-complex multiplication (PHM) layer [75]. Expanding on this, Compacter refines its methodology by parametrizing B_i similar to LoRA, with the expression $B_i =$

调优方法, 涉及更新语言模型的所有参数。参数高效微调在模型训练过程中显著降低了内存和存储需求。最先进的PEFT方法已证明能够通过调整模型少部分参数, 达到完全微调模型的性能。此外, 在标注数据稀缺的低资源环境中, PEFT方法还能实现比完全微调更优的准确率 [53, 22], 。

PEFT 包含多种方法, 例如在固定的预训练Transformer模型中插入小型可训练的前馈层 [24], 仅更新偏置参数, 同时冻结模型的其他参数 [6], 将权重矩阵分解为近似的低秩矩阵 [25], 引入Kro- necker乘法与权重矩阵结合并利用低秩矩阵 [39], 将嵌入权重矩阵拼接到固定模型中 [31, 28], 以及在模型的前馈和注意力层中添加高效的逐元素乘法以重新缩放权重矩阵 [35]。在本工作中, 我们将研究LoRA [25], 、Compacter [39] 和IA³ [35]。虽然LoRA之前在SE中进行了实验, 但Compacter和IA³ 在SE任务中的价值尚不明确。

LoRA. 由Hu等人提出。[25],LoRA借鉴了IntrinsicSAID[1] 的思想, 提出了一种更为简便的低秩微调方法。在LoRA中, 对权重矩阵 δW 的参数更新过程涉及将其分解为两个低秩矩阵的乘积 W_A 和 W_B :

$$\delta W = W_A W_B \quad (1)$$

其中 $W_A \in \mathbb{R}^{in \times r}$ 和 $W_B \in \mathbb{R}^{r \times out}$ 。

值得注意的是, LoRA 维持预训练模型参数的冻结状态, 仅允许 W_A 和 W_B 矩阵可训练。缩放因子通常为常数, 常设为 $1/r$ 。训练后, 这些分解的矩阵—— W_A 和 W_B ——可以通过简单地将矩阵乘积 $W_A W_B$ 与原始矩阵 W 相加, 无缝地整合回原始矩阵。

该方法在实现性能提升方面表现出显著的效果, 同时采用简化的低秩微调方法, 展现了其在广泛模型架构中的适用性。

Compacter. Mahabadi 等人 [39] 提出了 Compacter 方法, 该方法采用一种独特的策略, 利用克罗内克积、低秩矩阵以及跨层参数共享来生成适配器权重。在每个适配器中, 所有 W 参数都被构建为一组较小维度矩阵 A_i 和 B_i 的 n 个克罗内克积之和:

$$\hat{W} = \sum_{i=0}^n A_i \otimes B_i \quad (2)$$

其中 $\hat{W} \in \mathbb{R}^{k \times d}$ 、 $A_i \in \mathbb{R}^{n \times n}$ 、 $B_i \in \mathbb{R}^{\frac{k}{d} \times \frac{n}{d}}$ 。

这导致形成一个被称为参数化超复数乘法 (PHM) 层的线性层 [75]。在此基础上, Compacter通过参数化 B_i 类似于LoRA, 进一步优化其方法, 其表达式为 $B_i =$

$B_i^{down} B_i^{up}$. Here, all matrices maintain a maximum rank of r . Additionally, A_i matrices are shared across all adapter layers to enhance parameter efficiency.

IA³. In a recipe dubbed as *T-Few*, Liu et al. [35] proposed IA³, which introduces three trainable vectors to the keys and values of the attention layers and the feed-forward layers of a transformer-based language model. These vectors rescale the parameters of their respective parameter modules during the fine-tuning phase by operating on an element-wise level. Formally, the attention layers of an IA³-infused model are transformed as:

$$\text{softmax}(\frac{Q(l_k \odot K^T)}{\sqrt{d_k}})(l_v \odot V) \quad (3)$$

and the feed-forward modules transformed as:

$$W_2(l_{ff} \odot \gamma(W_1 x)), \quad (4)$$

where γ is the non-linearity of the feed-forward modules. The simplicity of the element-wise multiplication operations employed by IA³ introduces little computational overhead. Despite its straightforward operations, IA³ achieves significant performance, surpassing the human baseline on the RAFT benchmark [4]. This benchmark, which focuses on real-world and economically valuable problems, marked IA³ as the first method to exceed the human baseline at the time of its publication.

3 Study Design

In this section, we detail the experimental setup and study design, aiming to address the following research questions (RQ) in our investigation:

RQ1: What is the effectiveness of applying PEFT methods for the selected tasks in code-LLMs?

In this RQ, we compare the performance of applying LoRA, Compacter and IA³ on our selected LLMs for multiple programming languages, compared to their respective baselines. The results of this experiment will shed light on the capabilities of each PEFT method, while also providing insight on the adaptability of each LLM with these methods. As there are new architectures for each PEFT approach, and there are various architectures for LLMs, it is important to understand which methods could be applied to which models and their results.

RQ2: How do PEFT methods perform when used for transferring knowledge from natural language to code with LLMs?

Previous studies [20, 54] show that a specific PEFT method, called adapters, can help in transferring knowledge from natural language models to code-related tasks. They focused on the adaptability of smaller models, i.e., RoBERTa [37] in comparison to pre-trained programming language models, such as CodeBERT [18], and demonstrate that RoBERTa plus adapters can achieve on-par results compared to CodeBERT for code summarization. Given the resource-intensive nature

$B_i^{down} B_i^{up}$. 在此，所有矩阵的最大秩均为 r 。此外， A_i 矩阵在所有适配器层中共享，以提高参数效率。

IA³. 在一项名为 *T-Few* 的方案中，刘等人 [35] 提出了 IA³，该方法在基于变换器的语言模型的注意力层和前馈层引入了三个可训练向量。这些向量在微调阶段通过逐元素操作对各自参数模块的参数进行重新缩放。形式上，含有 IA³ 的模型的注意力层被转化为：

$$\text{softmax}(\frac{Q(l_k \odot K^T)}{\sqrt{d_k}})(l_v \odot V) \quad (3)$$

以及前馈模块被转换为：

$$W_2(l_{ff} \odot \gamma(W_1 x)), \quad (4)$$

其中 γ 是前馈模块的非线性部分。IA³ 所采用的逐元素乘法操作具有较低的计算开销。尽管操作简单，IA³ 仍实现了显著的性能提升，在 RAFT 基准测试 [4] 上超越了人类基线。该基准测试关注实际应用和经济价值问题，标志着 IA³ 在发表时成为首个超越人类基线的方法。

3 研究设计

在本节中，我们将详细介绍实验设置和研究设计，旨在回答我们研究中的以下问题（RQ）：

RQ1：在代码大模型中应用PEFT方法于所选任务的效果如何？

在本研究问题中，我们比较了在所选大型语言模型（LLMs）上应用LoRA、Compacter和IA³在多种编程语言中的性能，与各自的基线进行对比。该实验的结果将揭示每种PEFT方法的能力，同时也提供关于各个LLM在这些方法下的适应性见解。由于每种PEFT方法都存在新的架构，并且LLMs具有多样的架构，因此理解哪些方法可以应用于哪些模型及其效果具有重要意义。

RQ2：在使用大规模语言模型（LLMs）将知识从自然语言转移到代码的过程中，参数高效微调（PEFT）方法的表现如何？

先前的研究 [20, 54] 表明，一种名为适配器（adapters）的特定参数高效微调（PEFT）方法，有助于将知识从自然语言模型转移到与代码相关的任务中。研究重点在于较小模型的适应性，即以 RoBERTa [37] 为例，与预训练的编程语言模型（如 CodeBERT [18]）进行比较，结果显示，RoBERTa 加适配器在代码总结任务中可以达到与 CodeBERT 相当的效果。鉴于资源密集型的性质

of pretraining large language models on new domains (i.e., code), we aim to investigate whether the PEFT methods can achieve similar results when applied to LLMs that are not pre-trained on code. If successful, the results would hold promise for significantly mitigating the computational resources required for training code-LLM.

RQ3: How do PEFT-tuned LLMs perform in transferring their learned knowledge to an unseen programming language?

Prior research [49] suggests that language models may encounter performance limitations when confronted with languages that were not part of their training data. Consequently, we focus on evaluating LoRA's, Compacter's and IA³'s capabilities for leveraging the knowledge of a code-LLM for adapting to a programming language that is not seen during their pre-training phase. This experiment offers valuable insights into the extent of PEFT methods' abilities for leveraging the learned knowledge of programming languages and adapting them to new ones.

3.1 Approach

In this study, we employ two base code-LLM, CodeT5 [68] and CodeLlama [52], and their counterparts, i.e., T5 and Llama2, when LoRA, Compacter and IA³ are added to them.

To answer RQ1, we fine-tune the code-LLMs on the datasets of the two tasks, i.e., code summarization and code generation, separately. The details of the datasets are in Section 3.3. For each of the tasks, we only consider the mono-lingual data and report the results for each programming language separately. Models fine-tuned with PEFT methods will be compared to their respective baselines. To this end, we employ a fully fine-tuned CodeT5 as the baseline of the experiments on this model. However, for the CodeLlama, we use a pre-trained checkpoint of the model as the baseline since this model is nearly an order of magnitude larger than the T5 family of models, and fully fine-tuning is not feasible given our resource limitation.

To answer RQ2, we use T5 [50] and Llama2 [64] models and add LoRA, Compacter and IA³ to these models, as an alternative to fine-tuning them for code summarization and code generation tasks. Similar to RQ1, we conduct experiments for each programming language separately. Additionally, following RQ1, we use a fully fine-tuned T5 and a pre-trained checkpoint of Llama2 as baselines to ensure fairness and reduce computational costs.

For RQ3, we choose R as the target programming language for both code summarization and code generation tasks, as it has a wide community, but is an under-explored language for code intelligence. R is considered a low-resource programming language (i.e., it has limited training data available) and thus is chosen here [9]. Moreover, R is not used in the training dataset of CodeT5 [68] and CodeLlama [52], serving as a suitable unseen programming language. This is confirmed as the training dataset of CodeT5 is publicly available. Additionally, R is not among the list of programming languages used in the training of CodeLlama, as stated by its authors. To answer this RQ, we fine-tune T5, CodeT5, Llama2 and CodeLlama

在新领域（即代码）上进行大规模语言模型的预训练的基础上，我们旨在探讨 PEFT方法在未在代码上预训练的LLMs中应用时是否能够取得类似的效果。如果成功，相关结果将有望显著减轻训练代码LLMs所需的计算资源。

RQ3：经过PEFT调优的大型语言模型在将其学习到的知识迁移到未见过的编程语言中的表现如何？

先前的研究 [49] 表明，当面对未包含在训练数据中的语言时，语言模型可能会遇到性能限制。因此，我们重点评估LoRA、Compacter和IA³在利用代码-LLM的知识以适应未在预训练阶段出现的编程语言方面的能力。此实验为PEFT方法在利用已学知识以适应新编程语言方面的能力提供了宝贵的见解。

3.1 方法

在本研究中，我们采用了两种基础代码大模型（CodeT5 [68] 和 CodeLlama [52],）及其对应模型，即 T5 和 Llama2, 当加入 LoRA、Compacter 和 IA³ 时。

为回答RQ1，我们在两个任务的数据集上分别对code-LLMs进行微调，即代码总结和代码生成。数据集的详细信息见第3.3节。对于每个任务，我们仅考虑单一语言数据，并分别报告每种编程语言的结果。采用PEFT方法微调的模型将与其各自的基线模型进行比较。为此，我们使用完全微调的CodeT5作为本次实验的基线模型。然而，对于CodeLlama，我们采用预训练的模型检查点作为基线，因为该模型的规模几乎比T5系列模型大一个数量级，考虑到我们的资源限制，完全微调不可行。

为回答RQ2，我们使用T5 [50] 和Llama2 [64]模型，并在这些模型上加入LoRA、Compacter和IA³，作为对其进行微调以执行代码摘要和代码生成任务的替代方案。与RQ1类似，我们对每种编程语言单独进行实验。此外，遵循RQ1的做法，我们采用完全微调的T5和Llama2的预训练检查点作为基线，以确保公平性并降低计算成本。

对于RQ3，我们选择R作为代码总结和代码生成任务的目标编程语言，因为它拥有广泛的社区，但在代码智能领域仍属于研究较少的语言。R被认为是一种低资源的编程语言（即可用的训练数据有限），因此在此被选用 [9]。此外，R未在CodeT5 [68] 和CodeLlama[52],的训练数据集中使用，作为一种未见过的编程语言，具有良好的代表性。这一点得到了CodeT5训练数据集公开可用的确认。此外，正如其作者所述，R也不在CodeLlama训练所用的编程语言列表中。为回答此RQ，我们对T5、CodeT5、Llama2和CodeLlama进行了微调。

using the PEFT methods on code summarization and code generation for R. Following the previous RQs, we use fully fine-tuned T5 and CodeT5 and pre-trained checkpoints of Llama2 and CodeLlama as baselines. This experiment gives insight into how capable each model and PEFT method is when transferring the learned knowledge from other languages used in pre-training to R.

3.2 Downstream Tasks

To evaluate the models, we choose the two code summarization and code generation tasks. These tasks are studied in several works [68, 69, 52, 54, 33] and are two of the tasks on CodeXGLUE benchmark³. We choose these tasks as many of these are code-to-text and text-to-code tasks, showing the capabilities of the LLMs for both code and natural language comprehension. Additionally, in RQ2, we intend to investigate the knowledge transferability of the LLMs to code intelligence tasks. Therefore, choosing code-to-code tasks might not be the best option for T5 and Llama2 models, as there is no natural language involved in those tasks. Exploring the full spectrum of this knowledge transferability for various code intelligence tasks is beyond the scope of this study. The last reason for choosing these tasks is the scarcity of the available data for R, which prohibits exploring more tasks.

Code Summarization. The primary objective of a code summarization task is to distill the functionality and purpose of a code snippet or function into a concise and understandable summary. Code summarization enables streamlining documentation generation processes and automating commit generation. We train the models on the code summarization datasets. Then we test the model by feeding the code snippets as input sequences and evaluate the description comments as target sequences (*Code* $\rightarrow$ *Comment*). We employ this task to gauge the models' capabilities to analyze the structure, logic, and functionality of the code snippets [3].

Code Generation. Code generation task aims to transform natural language descriptions or instructions into executable code or programming language constructs. This task enables the automation of code writing based on human-provided specifications or requirements. By converting natural language descriptions into functional code snippets, code generation contributes to automating software development processes and aids in rapid prototyping or code scaffolding. To train a model for code generation, we provide natural language descriptions, along with additional desired context such as a specific function header, as input sequences and expect the corresponding code snippets as target sequences (*Comment* $\rightarrow$ *Code*). This setup allows us to assess the models' proficiency in understanding and translating human-readable descriptions into executable code segments.

³ <https://microsoft.github.io/CodeXGLUE/>

在R的代码总结和代码生成任务中采用PEFT方法。遵循前述研究问题，我们使用完全微调的T5和CodeT5，以及Llama2和CodeLlama的预训练检查点作为基线。本实验旨在揭示各模型及PEFT方法在将预训练中其他语言所学知识迁移到R时的能力。

3.2 下游任务

为了评估模型，我们选择了两个代码总结和代码生成任务。这些任务在多个研究中有所涉及 [68, 69, 52, 54, 33]，并且是CodeXGLUE基准测试³中的两个任务。我们选择这些任务，是因为其中许多是代码到文本和文本到代码的任务，展示了大规模语言模型在代码和自然语言理解方面的能力。此外，在RQ2中，我们打算研究大规模语言模型在代码智能任务中的知识迁移能力。因此，选择代码到代码的任务可能并不最适合T5和Llama2模型，因为这些任务不涉及自然语言。探索这种知识迁移在各种代码智能任务中的完整范围超出了本研究的范围。选择这些任务的最后一个原因是R的数据有限，无法探索更多的任务。

代码总结。代码总结任务的主要目标是将代码片段或函数的功能和目的提炼成简洁且易于理解的摘要。代码总结有助于简化文档生成流程和自动化提交信息的生成。我们在代码总结数据集上训练模型，然后通过输入代码片段作为序列进行测试，并将描述性注释作为目标序列进行评估（代码 $\rightarrow$ 注释）。我们采用此任务来评估模型分析代码片段的结构、逻辑和功能的能力[3]。

代码生成。代码生成任务旨在将自然语言描述或指令转换为可执行的代码或编程语言结构。该任务实现了基于人类提供的规范或需求自动化编写代码的可能性。通过将自然语言描述转化为功能性代码片段，代码生成有助于自动化软件开发流程，并支持快速原型设计或代码脚手架的构建。为了训练代码生成模型，我们提供自然语言描述以及额外的上下文信息，如特定的函数头，作为输入序列，并期望得到相应的代码片段作为目标序列（*Comment* $\rightarrow$ *Code*）。这种设置使我们能够评估模型理解和转换人类可读描述为可执行代码段的能力。

<https://microsoft.github.io/CodeXGLUE/>

3.3 Datasets and Benchmarks

CodeSearchNet Dataset. CodeSearchNet [27], curated by Microsoft Research, is a dataset of 2,326,976 functions paired with corresponding explanation comments in six programming languages, namely, Go, Java, JavaScript, PHP, Python, and Ruby. CodeSearchNet is derived from a diverse set of real-world code repositories, including GitHub. We have opted for the utilization of CodeSearchNet in RQ1 and RQ2 because it has been a subject of recent scholarly investigations in the realm of code summarization [18, 21, 68, 53]. Throughout this paper, we refer to the programming languages present in CodeSearchNet as the *six seen languages*.

SPP Dataset. The SPP dataset [74] consists of approximately 450,000 synthetic Python programming problems, each containing a task description in the form of a function header and a comment, one to three use-cases, a code solution, and one to three test cases, generated using the CodeGeeX-13B model. For our experiments, we utilized a subset of this data in RQ1 and RQ2: a verified split containing 30,000 samples that have been de-duplicated and verified by a Python interpreter. This ensures the side effect of low-quality data is eliminated from the experiments given the unverified subset may contain syntactically incorrect code and multiple function definitions per sample, degrading the data quality.

R Code Summarization Dataset. We use the $R_{combined}$ subset of the R dataset curated by Zhao and Fard [76], which contains 32,671, and 4,078, and 4,084 code and comment pairs for train, evaluation, and test, respectively.

This dataset is collected by scraping active and public GitHub repositories containing R files, which have an open-source license. The dataset is further filtered to eliminate repositories that are personal or do not follow proper R packaging rules. The collection is parsed using the Tree-sitter⁴ parser and R documentation tools to transpile it into a dataset containing R code snippets and natural language descriptions [76]. This dataset has the same structure as CodeSearchNet and assures consistency among datasets when comparing the results of code summarization. We utilize this dataset for the R code summarization tasks in RQ3.

MultiPL-T. MultiPL-T is an approach for curating semi-synthetic datasets for low-resource languages proposed by Cassano et al. [9]. This approach translates high-quality code snippets of high-resource languages such as Python into Julia, Lua, OCaml, R, and Racket using a variation of the StarCoderBase-15B model [30]. The source code snippets consist of test-validated function bodies and descriptive document strings, making the target datasets ideal for code generation tasks. The R split of the MultiPL-T dataset contains 37,592 pairs of natural language descriptions and R functions. We utilize this dataset for R code generation in RQ3 by splitting it into 35,338, 376, and 1,860 samples for train, validation and testing, respectively.

HumanEval. Proposed by OpenAI, HumanEval [12] is a Python benchmark consisting of 164 hand-written instructions and on average 7.7 unit tests for each instruction for code generation evaluation. HumanEval directly assesses the functionality correctness of generated code by testing each sample against a suite of

⁴ <https://tree-sitter.github.io/tree-sitter>

3.3 数据集与基准测试

CodeSearchNet 数据集。CodeSearchNet 由微软研究院整理，是一个包含 2,326,976 个函数及其对应注释说明的多语言代码数据集，涵盖六种编程语言，即 Go、Java、JavaScript、PHP、Python 和 Ruby。CodeSearchNet 来源于多样化的真实代码仓库，包括 GitHub。我们在 RQ1 和 RQ2 中选择使用 CodeSearchNet，是因为它在代码摘要领域的学术研究中具有一定的代表性和应用价值 [18, 21, 68, 53]。在本文中，我们将 CodeSearchNet 中的编程语言统称为六种常见语言。

SPPDataset. SPP 数据集 [74] 由大约 45 万道合成的 Python 编程题组成，每题包括任务描述（以函数头和注释的形式）、一到三个用例、代码解决方案以及一到三个测试用例，均由 CodeGeeX-13B 模型生成。在我们的实验中，RQ1 和 RQ2 使用了该数据的一个子集：一个经过验证的划分，包含 30,000 个样本，已通过去重和 Python 解释器验证。这确保了实验中低质量数据的副作用被消除，因为未验证的子集可能包含语法错误的代码和每个样本多个函数定义，从而降低数据质量。

R 代码总结数据集。我们使用由赵和法尔整理的 R 数据集的 $R_{combined}$ 子集，该子集包含用于训练、评估和测试的代码与注释对，分别为 32,671、4,078 和 4,084 对。

该数据集通过抓取包含 R 文件的活跃且公开的 GitHub 仓库收集而来，这些仓库均具有开源许可证。数据集经过筛选，排除个人仓库或不遵循规范 R 打包规则的仓库。采用 Tree-sitter⁴ 解析器和 R 文档工具对其进行解析，将其转化为包含 R 代码片段和自然语言描述的数据集 [76]。该数据集的结构与 CodeSearchNet 相同，确保在代码摘要结果的比较中数据集的一致性。我们在 RQ3 的 R 代码摘要任务中使用该数据集。

MultiPL-T. MultiPL-T 是由 Cassano 等人提出的一种用于低资源语言半合成数据集构建的方法 [9]。该方法利用 StarCoderBase-15B 模型的变体，将高资源语言（如 Python）的高质量代码片段翻译成 Julia、Lua、OCaml、R 和 Racket [30]。源代码片段包括经过测试验证的函数体和描述性文档字符串，使得目标数据集非常适合用于代码生成任务。MultiPL-T 数据集的 R 份包含 37,592 对自然语言描述与 R 函数。我们在 RQ3 中利用该数据集进行 R 代码生成，将其划分为 35,338、376 和 1,860 个样本，分别用于训练、验证和测试。

HumanEval. 由 OpenAI 提出，HumanEval [12] 是一个 Python 基准测试，包括 164 个手工编写的指令，平均每个指令配备 7.7 个单元测试，用于代码生成评估。HumanEval 通过对每个样本进行一系列测试，直接评估生成代码的功能正确性。

⁴ <https://tree-sitter.github.io/tree-sitter>

unit tests hand-crafted for each instruction within the benchmark. This method of code generation evaluation mitigates reported performance inaccuracy caused by functionally correct but textually dissimilar output samples. We will use this benchmark to evaluate our trained model for code generation in Python.

MultiPL-E. As a translation system for code generation test-driven benchmarks, MultiPL-E [8] is designed to translate sample benchmark functions and their associated unit tests into new arbitrary programming languages, similar to MultiPL-T. The authors have utilized MultiPL-E to translate the HumanEval [12] and MBPP [5] benchmarks into 18 new languages, including R. As the R counterpart of HumanEval for Python code generation, we employ the R translation of the HumanEval benchmark created using MultiPL-E for this study. This benchmark has a structure similar to the HumanEval benchmark, containing 161 problems. This benchmark is used for R code generation evaluation in this study.

3.4 Evaluation Metrics

For evaluating code summarization, we follow previous research by adopting the smoothed BLEU-4 [43] as our evaluation metric [2, 69, 60]. Aligning with previous methodologies, we maintain consistency by averaging the BLEU scores to derive a comprehensive assessment of the generated summaries in comparison to the reference texts.

BLEU. The BLEU score [43] is a metric originally proposed for evaluating the quality of translations at the document level. This metric relies on precision and evaluates the geometric precision of n-grams between the generated text and the ground truth text. For a translation T , a reference R and a range from 1 to N of n-grams, the BLEU score is calculated as follows:

$$BLEU(N, T, R) = \left(\prod_{n=1}^N \frac{m_n}{l_n} \right)^{\frac{1}{N}} \times BP(T, R) \quad (5)$$

where m_n is the number of matching n-grams between a translation and its reference, and $BP(T, R)$ associates a penalty to the score when the length of the translation is less than the reference using the following formula:

$$BP(T, R) = \min(1, \exp(1 - \frac{\text{len}(T)}{\text{len}(R)})) \quad (6)$$

As noted, the BLEU score is used for evaluating a prediction across the entire reference corpus. For our use case (i.e., evaluating a sentence level output against a ground truth sentence), a smoothed variant of the BLEU score is required, which is obtained by replacing m_n in formula 5 with a small positive number ϵ if there are no matching n-grams (i.e. $m_n = 0$). For evaluating code summarization, we follow prior research [2, 69, 60] by adopting the smoothed BLEU score with $N = 4$ (henceforth referenced as BLEU-4) as our evaluation metric.

CodeBLEU. The BLEU score, designed to evaluate text similarity between generated and reference text, possesses limitations when applied to assess code generation tasks due to its emphasis on linguistic resemblance rather than functional

为基准中的每个指令手工编写的单元测试。此代码生成评估方法减轻了由功能正确但文本上不同的输出样本引起的性能不准确问题。我们将使用该基准评估我们在Python中的代码生成训练模型。

MultiPL-E。作为一种面向代码生成测试驱动基准的翻译系统，MultiPL-E [8] 旨在将示例基准函数及其相关的单元测试翻译成新的任意编程语言，类似于 MultiPL-T。作者已利用MultiPL-E将HumanEval [12] 和MBPP [5] 基准翻译成包括R在内的18种新语言。作为Python代码生成的HumanEval在R中的对应版本，我们在本研究中采用了由MultiPL-E创建的HumanEval基准的R翻译版本。该基准结构类似于HumanEval基准，包含161个问题。本基准用于本研究中的R代码生成评估。

3.4 评估指标

在评估代码摘要时，我们遵循先前的研究，采用平滑BLEU-4 [43] 作为我们的评估指标 [2, 69, 60]。为了与以往的方法保持一致，我们通过平均BLEU分数来获得对生成摘要与参考文本之间的综合评估。

BLEU. BLEU得分 [43] 是一种最初提出用于评估文档级翻译质量的指标。该指标依赖于精确率，评估生成文本与参考文本之间n-gram的几何精确度。对于一个翻译 T 、一个参考 R 以及从1到 N 的n-gram范围，BLEU得分的计算公式如下：

$$BLEU(N, T, R) = \left(\prod_{n=1}^N \frac{m_n}{l_n} \right)^{\frac{1}{N}} \times BP(T, R) \quad (5)$$

其中 m_n 表示翻译与参考之间匹配的 n-gram 数量， $BP(T, R)$ 在翻译长度小于参考长度时对得分施加惩罚，使用以下公式：

$$BP(T, R) = \min(1, \exp(1 - \frac{\text{len}(T)}{\text{len}(R)})) \quad (6)$$

如前所述，BLEU得分用于评估整个参考语料库的预测结果。对于我们的使用场景（即将句子级输出与真实句子进行比较），需要采用平滑版本的BLEU得分，该版本通过在公式5中的 m_n 用一个小的正数 ϵ 替代（当没有匹配的n-gram时，即 $m_n = 0$ ）。在代码总结评估中，我们遵循先前的研究 [2, 69, 60]，采用带有 $N = 4$ （以下简称BLEU-4）作为我们的评估指标的平滑BLEU得分。

CodeBLEU。BLEU分数旨在评估生成文本与参考文本之间的相似度，但在应用于代码生成任务时存在局限性，因为其强调语言上的相似性而非功能性。

equivalence. Consequently, accurately functional yet linguistically dissimilar code may receive suboptimal BLEU scores, rendering it an inadequate metric for code evaluation. For code generation, we employ CodeBLEU [51] and EM@k.

CodeBLEU is proposed to validate the syntactic and semantic data flow accuracy inherent in code generation models. CodeBLEU stands as a composite measure that injects code syntax via abstract syntax trees (AST) and code semantics via data flow. Compared to the BLEU score, This metric serves as a more comprehensive assessment tool, facilitating the quantification and analysis of the multifaceted aspects associated with the correctness of generated code.

$$CodeBLEU = \alpha \cdot BLEU + \beta \cdot BLEU_{weight} + \gamma \cdot Match_{ast} + \delta \cdot Match_{df}$$

Where $BLEU$, $BLEU_{weight}$, $Match_{ast}$, and $Match_{df}$ are calculated based on standard BLEU, weighted n-gram match, syntactic AST match, and semantic dataflow match, respectively. α , β , γ and δ determine the the weighted combination of four parts.

EM@k. Exact Match pertains to a strict quantification textual prediction, where if a prediction for a sample exactly matches the ground truth (or one of several ground truths) associated with that sample, $EM = 1$, otherwise $EM = 0$. In the scope of this metric, even a single character discrepancy between the prediction and the ground truth will result in $EM = 0$. Formally, the average Exact Match score of $k \in \{1, N\}$ candidate samples generated by the model for a reference ground truth r is computed as follows:

$$EM@k = \frac{\sum_{i=1}^k EM(s_i, r)}{k} \quad (7)$$

In this study, we employ EM@1 as part of our evaluations for code generation experiments.

Pass@k. Despite alleviating some of the BLEU score's issues when applied for code generation and its widespread use in the academic scene, CodeBLEU still possesses caveats [12]. For instance, the CodeBLEU score for a dissimilar (both textual and AST) generated code compared to the ground truth will degrade, even if the dissimilar generated code does satisfy the requested functionality. To address these issues, OpenAI has proposed HumanEval [12], a Python benchmark dataset, as explained above. The evaluation metric associated with HumanEval is $Pass@k$, where if among the $n > k$ generated sample codes c samples pass all unit tests corresponding to an instruction, the evaluation metric is obtained as follows:

$$Pass@k := E[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}] \quad (8)$$

In all experiments, we use greedy decoding for generation, thus eliminating randomness from generations. Consequently, we chose $k = 1$ with one generation per sample to calculate Pass@k and EM@k, as subsequent generations with the same input produce the same results.

等价性。因此，功能准确但在语言上不相似的代码可能获得次优的BLEU分数，导致其成为不充分的代码评估指标。在代码生成方面，我们采用CodeBLEU [51] 和EM@k。

CodeBLEU 被提出用于验证代码生成模型中固有的句法和语义数据流的准确性。CodeBLEU 作为一种复合指标，通过抽象语法树（AST）引入代码句法，通过数据流引入代码语义。与 BLEU 分数相比，该指标作为一种更全面的评估工具，有助于量化和分析与生成代码正确性相关的多方面因素。

$$CodeBLEU = \alpha \cdot BLEU + \beta \cdot BLEU_{weight} + \gamma \cdot Match_{ast} + \delta \cdot Match_{df}$$

$BLEU$ 、 $BLEU_{weight}$ 、 $Match_{ast}$ 、和 $Match_{df}$ 分别基于标准BLEU、加权n-gram匹配、句法AST匹配和语义数据流匹配进行计算。 α 、 β 、 γ 和 δ 决定这四部分的加权组合。

EM@k. 精确匹配（Exact Match）指的是一种严格的文本预测量化方法，当模型对某个样本的预测与该样本的真实值（或多个真实值之一）完全一致时， $EM = 1$ ，否则 $EM = 0$ 。在此指标范围内，即使预测与真实值之间仅有一个字符的差异，也会导致 $EM = 0$ 。正式地，模型为参考真实值 r 生成的 $k \in \{1, N\}$ 个候选样本的平均精确匹配得分计算公式如下：

$$EM@k = \frac{\sum_{i=1}^k EM(s_i, r)}{k} \quad (7)$$

在本研究中，我们将 EM@1 作为代码生成实验评估的一部分。

Pass@k. 尽管在应用于代码生成时缓解了一些BLEU分数的问题，并在学术界得到广泛使用，CodeBLEU仍存在一些局限性 [12]。例如，对于与真实代码在文本和抽象语法树（AST）方面均不相似的生成代码，即使该生成代码满足所请求的功能，其CodeBLEU分数也会降低。为了解决这些问题，OpenAI提出了HumanEval [12]，这是一个如上所述的Python基准数据集。与HumanEval相关的评估指标为 $Pass@k$ ，即在 $n > k$ 个生成样本代码 c 中，若有样本通过了所有与指令对应的单元测试，则该评估指标的值如下计算：

$$Pass@k := E[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}] \quad (8)$$

在所有实验中，我们采用贪婪解码进行生成，从而消除生成过程中的随机性。因此，我们选择 $k = 1$ 每个样本生成一次，以计算Pass@k和EM@k，因为后续在相同输入下的生成会产生相同的结果。

Textual and Functionality Similarity in Code Generation. Code snippets generated by a model can be assessed based on textual similarity and functional correctness. A snippet with high textual similarity to the ground truth may still fail functionally, as even a single incorrect operator can compromise its functionality despite its resemblance to the original code. The BLEU-4 score measures textual similarity between a sample and the ground truth, making it suitable for code summarization. However, it falls short in evaluating functionality for code generation. CodeBLEU addresses this limitation by incorporating the abstract syntax tree and data flow, offering a more in-depth functional assessment.

Taking these characteristics into account, the only metric that ensures the code is executable and correct is the Pass@K. The HumanEval benchmark addresses these limitations by evaluating correctness through a predefined set of test cases focused solely on functionality, rather than textual similarity to the ground truth. Since the problems in this benchmark are uniquely crafted, a high Pass@k HumanEval score suggests that the model may perform well on out-of-distribution problems. For the code generation experiments in this study, we use all the mentioned metrics when available, considering both textual similarity and functionality correctness.

3.5 Base Models

This section covers the base LLMs used in this study. We refer to the code-specific models as *code-LLM* and general-purpose natural language models as *general-LLM*.

CodeT5. CodeT5 [68] is a code-LLM based on T5 [50], pre-trained on CodeSearchNet [27], and additional C and C# data. By considering the type of identifiers in code and utilizing an encoder-decoder architecture, CodeT5 achieves state-of-the-art performance in a wide range of code comprehension and generation tasks in the CodeXGLUE benchmark [38].

CodeLlama. CodeLlama [52] is another state-of-the-art decoder-only code-LLM that comes in different variants and parameter budgets. CodeLlama is based on Llama2 [64] with further training on an extensive corpus consisting of 500 billion tokens (1 trillion tokens for CodeLlama 70B) in the case of the foundation model. CodeLlama is trained on a proprietary dataset, encompassing programming languages such as Java, Javascript, PHP, Python, C++, and C#, among others.

T5. T5 [50] was proposed as a Text-to-Text Transformer, a unified model used for Transfer learning. The authors introduced a unified framework that converted all text-based problems into a text-to-text format. They obtained state-of-the-art results for many benchmarks. This model is the base of CodeT5 and is used for RQ2 as a general-LLM.

Llama2. Llama2 [64] is the second version of Llama [63], a series of foundational large language models proposed in 7 to 65 billion parameter budgets, which can outperform larger models such as GPT-3 [7] with 175 billion parameters and keep up with much larger models such as PaLM [13] with 540 billion parameters. Llama's architecture is inspired by various advancements from different mod-

代码生成中的文本和功能相似性。由模型生成的代码片段可以基于文本相似性和功能正确性进行评估。与真实代码高度文本相似的片段可能仍然在功能上失败，因为即使只有一个操作符错误也可能破坏其功能，尽管它与原始代码相似。BLEU-4分数衡量样本与真实代码之间的文本相似性，适用于代码摘要。然而，它在评估代码生成的功能方面存在不足。CodeBLEU通过结合抽象语法树和数据流，提供了更深入的功能评估，弥补了这一局限。

考虑到这些特性，唯一能够确保代码可执行且正确的指标是 Pass@K。HumanEval 基准通过评估一组预定义的测试用例来解决这些限制，测试仅关注功能性，而非与真实答案的文本相似度。由于该基准中的问题是经过特殊设计的，高的 Pass@k HumanEval 分数表明模型在分布外问题上可能表现良好。在本研究的代码生成实验中，我们在可用时使用所有提及的指标，兼顾文本相似性和功能正确性。

3.5 基础模型

本节介绍本研究中使用的基础大型语言模型（LLMs）。我们将专门针对代码的模型称为*code-LLM*，而通用自然语言模型则称为*general-LLM*。

CodeT5. CodeT5 [68] 是一种基于T5 [50], 预训练的代码大模型（Code-LLM），在CodeSearchNet [27], 及额外的C和C#数据上进行训练。通过考虑代码中标识符的类型并采用编码器-解码器架构，CodeT5在CodeXGLUE基准测试中的代码理解与生成任务中实现了最先进的性能。

CodeLlama. CodeLlama [52] 是另一种最先进的仅解码器代码大模型，具有不同的变体和参数预算。CodeLlama基于Llama2 [64]，在一个包含5000亿标记（CodeLlama 70B为1万亿标记）的大型语料库上进行了进一步训练，属于基础模型。CodeLlama在一个专有数据集上进行训练，涵盖了Java、Javascript、PHP、Python、C++和C#等多种编程语言。

T5. T5 [50] 被提出作为一种文本到文本的转换器（Text-to-Text Transformer），是一种用于迁移学习的统一模型。作者引入了一个统一框架，将所有基于文本的问题转换为文本到文本的格式。在多个基准测试中取得了最先进的结果。该模型是 CodeT5 的基础，并作为通用大型语言模型（general-LLM）用于 RQ2。

Llama2 [64] 是Llama [63], 系列基础大型语言模型的第二个版本，参数规模在7亿到65亿之间，能够超越参数规模更大的模型，如具有1750亿参数的GPT-3 [7]，并且与参数规模更大的模型如具有5400亿参数的PaLM [13] 保持竞争。Llama的架构借鉴了来自不同模型的多项创新。

els such as SwiGLUE [58] and Rotary Embeddings [62]. In contrast with the GPT series, the open-source nature of Llama models allows independent users and researchers to use and experiment with them more easily. Compared to Llama, Llama2 was trained on 40% more training data and allows for a larger context length. Choosing Llama2 over Llama provides us with a fairer comparison in RQ2, given that CodeLlama is based on Llama2.

We employ T5 and the base version of CodeT5 with 220 million parameters and the 7 billion parameters Llama2 and CodeLlama-base in all of our experiments. It is worth mentioning that our experiments are mainly focused on the comparison of the performance of PEFT methods within the same base models, not across different based models with varied parameter budgets, hence, the difference in the number of parameters among the models is not a threat to the validity of our experiments. Our implementation of the base models involves initializing model implementations similar to HuggingFace’s for T5 and Llama2 by publicly available CodeT5 and CodeLlama checkpoints.

To reduce the effect of different prompting techniques on the performance of the instruction-tuned models for CodeLlama, we use the base version of Llama2 and CodeLlama models. Additionally, we sample models in zero-shot settings without any prompt engineering. These settings eliminate unnecessary confounding variables, ensuring that the comparison of the results is fair among all PEFT methods.

3.6 PEFT Methods

In our experiments, we employ LoRA, Compacter and IA³ in combination with the fixed-parameter models, CodeT5 and CodeLlama (and T5 and Llama2 for RQ2), across all experiments. We choose to incorporate LoRA in our experiments due to its stronger performance compared to other PEFT methods, as shown by previous researchers in various settings [11, 78, 70]. Given this, LoRA is a suitable representative of state-of-the-art PEFT methods for studying knowledge transfer. Compacter and IA³ on the other hand are relatively unexplored PEFT methods, specifically for software engineering tasks. By comparing them to LoRA, our study can provide valuable insight into their performance, specifically when utilized for transferring knowledge to unseen languages (RQ3). We only select these three PEFT techniques due to restrictions of computational resources of fine-tuning LLMs, and since previous studies have shown these are SOTA approaches in terms of their achieved performance in the NLP field. As these three PEFT methods are under-explored in SE, we study them in our experiments.

As implementation of the PEFT methods is not within our scope, we rely on third-party implementations in this study. For LoRA and IA³, we use HuggingFace’s *peft* library [40], and for Compacter we use the *AdapterHub* [47] platform to facilitate composing all model configurations. Given the large number of experiments resulting from various combinations of models, programming languages, target tasks and PEFT methods, we refrain from experimenting with multiple hyperparameters available for each PEFT method and instead rely on the hyperparameters

例如, SwiGLUE [58] 和 Rotary Embeddings [62]。与 GPT 系列不同, Llama 模型的开源特性使得独立用户和研究人员更容易使用和实验。与 Llama 相比, Llama2 在训练数据上增加了 40%, 并支持更长的上下文长度。选择 Llama2 而非 Llama, 有助于在 RQ2 中进行更公平的比较, 因为 CodeLlama 是基于 Llama2 构建的。

在所有实验中, 我们采用了T5和CodeT5的基础版本(参数量为2.2亿)以及参数量为70亿的Llama2和CodeLlama-base。值得一提的是, 我们的实验主要集中在相同基础模型内PEFT方法性能的比较, 而非在不同基础模型和不同参数预算之间进行比较, 因此模型参数数量的差异并不影响实验的有效性。我们对基础模型的实现包括使用类似HuggingFace的实现方式, 通过公开的CodeT5和CodeLlama检查点初始化模型。

为了减少不同提示技术对指令调优模型在CodeLlama上的性能影响, 我们采用了Llama2和CodeLlama的基础版本模型。此外, 我们在零样本设置中采样模型, 未进行任何提示工程。这些设置消除了不必要的混杂变量, 确保所有PEFT方法的结果比较公平。

3.6 PEFT 方法

在我们的实验中, 我们采用了LoRA、Compacter和IA³, 结合固定参数模型 CodeT5和CodeLlama (以及RQ2中的T5和Llama2), 贯穿所有实验。我们选择在实验中引入LoRA, 是因为其相较于其他PEFT方法表现更优, 正如之前研究者在各种设置中所展示的 [11, 78, 70]。鉴于此, LoRA是研究知识迁移的最先进PEFT方法的合适代表。另一方面, Compacter和IA³ 是相对较少探索的PEFT方法, 特别是在软件工程任务中。通过将它们与LoRA进行比较, 我们的研究可以为它们的性能提供宝贵的见解, 特别是在用于迁移知识到未见过的语言时 (RQ3)。我们仅选择这三种PEFT技术, 原因是微调大型语言模型的计算资源有限, 并且已有研究表明这些方法在自然语言处理领域的性能达到了最先进水平。由于这三种PEFT方法在软件工程中的研究较少, 我们在实验中对其进行了探索。

由于PEFT方法的实现不在我们的研究范围内, 本研究依赖第三方实现。对于LoRA和IA³, 我们使用HuggingFace的*peft*库 [40], ; 对于Compacter, 我们使用*AdapterHub*平台 [47], 以便于组合所有模型配置。鉴于由模型、编程语言、目标任务和PEFT方法的各种组合所产生的大量实验, 我们避免对每种PEFT方法的多组超参数进行多次试验, 而是依赖于预设的超参数。

proposed by the authors of each PEFT method and third-party libraries as optimal defaults.

Moreover, we use the 16-bit *brain floating point* data type to greatly reduce runtime and memory usage during fine-tuning, without impacting performance significantly. While LoRA and Compacter’s implementations support this data type, IA³’s implementation lacks support. As a fallback, we use traditional 32-bit floating point data types for IA³. This does not impact the results obtained for IA³, however, this is not the case for its runtime and memory usage. Thus, we only report the performance metrics of IA³ and eliminate this PEFT when discussing the computational resource efficiency of the PEFT methods in the discussion section (Section 5).

3.7 Statistical Test.

The statistical tests are used to determine if the differences in performance between PEFT-infused models and their baselines are significant. To assess the significance of the results of this study’s experiments, we report the statistical tests using ANOVA and pair-wise t-tests [23] for the results obtained from each PEFT and its respective baseline. We employ bootstrap sampling with replacement, 1000 repetitions and sample sizes of 100 and ensure the normality of distributions by the Shapiro–Wilk normality test [56]. We limit the reports on the p-values of the BLEU-4. Specifically, we omit the statistical test reports on the HumanEval benchmarks for Python and R code generation tasks for the Pass@1 metric, as we use deterministic generation for these benchmarks and eliminate randomness.

3.8 Experimental Setup

We conduct the experiments using one Nvidia 40GB A100 GPU to fine-tune and test all model configurations in this study. We use HuggingFace’s *peft* library [40] and *AdapterHub* [47] platform to facilitate composing all model configurations. Throughout the experimentation process, the Llama and T5 family of models undergo fine-tuning with learning rates of $2e-4$ and $3e-3$, respectively, for a total of five epochs for smaller datasets (i.e., SPP and R datasets) and one epoch for CodeSearchNet, with early stopping mechanisms in place to ensure optimal performance. We use 500 warmup steps and a batch size of 16 for the larger models (i.e., Llama2 and CodeLlama) and 100 warmup steps and a batch size of 64 for the smaller models (i.e., T5 and CodeT5). Additionally, we utilize 4-bit float quantized base models to facilitate fine-tuning.

Our approach aligns with default implementation, following the established guidelines for setting hyperparameters specific to these PEFT methods. Following previous studies [71], for LoRA and IA³, we add matrices to the attention layers and maintain a dimension rank of 8 and $\alpha = 16$ and for Compacter, we use a *phm* dimension and rank of 4 and 1, respectively. For other hyperparameters, we rely on libraries’ defaults.

由各个PEFT方法的作者和第三方库提出的作为最佳默认值。

此外，我们使用16位*brain floating point*数据类型，在微调过程中大大减少了运行时间和内存使用，而不会显著影响性能。虽然LoRA和Compacter的实现支持该数据类型，但IA³的实现不支持。作为备选方案，我们对IA³采用传统的32位浮点数据类型。这不会影响IA³的结果，但在运行时间和内存使用方面则不同。因此，我们仅报告IA³的性能指标，并在讨论部分（第5节）讨论PEFT方法的计算资源效率时，排除该PEFT。

3.7 统计检验

统计检验用于确定引入PEFT的模型与其基线模型之间性能差异是否具有统计学意义。为了评估本研究实验结果的显著性，我们采用ANOVA和成对t检验 [23] 对每个PEFT及其对应基线模型的结果进行统计分析。我们采用自助抽样法（bootstrap sampling）进行重复抽样，重复1000次，样本量为100，并通过Shapiro–Wilk正态性检验 [56]确保分布的正态性。我们仅报告BLEU-4的p值，特别是，对于Python和R代码生成任务的HumanEval基准测试中的Pass@1指标，我们省略了统计检验报告，因为这些基准测试采用确定性生成，消除了随机性。

3.8 实验设置

我们使用一台Nvidia 40GB A100 GPU进行本研究中所有模型配置的微调 and 测试。我们采用HuggingFace的*peft*库 [40]和*AdapterHub* [47] 平台，以便组合所有模型配置。在整个实验过程中，Llama和T5系列模型分别以学习率 $2e-4$ 和 $3e-3$ 进行微调，对于较小的数据集（即SPP和R数据集）进行五个epoch的训练，对于CodeSearchNet数据集则进行一个epoch，且采用提前停止机制以确保最佳性能。我们为较大模型（即Llama2和CodeLlama）设置500个预热步骤和批次大小16，为较小模型（即T5和CodeT5）设置100个预热步骤和批次大小64。此外，我们还利用4-bit浮点量化基础模型以促进微调。

我们的方法与默认实现保持一致，遵循针对这些PEFT方法设定超参数的既定指南。根据之前的研究 [71], 关于LoRA和IA³，我们在注意力层中添加矩阵，并保持维度秩为8和 $\alpha = 16$ ，对于Compacter，我们使用*phm*维度和秩分别为4和1。对于其他超参数，我们依赖于库的默认设置。

PEFT	Parameters trained %	Seen							Unseen R
		Python	Go	Java	Javascript	Ruby	PHP	Average	
CodeT5									
Full	100%	19.65	19.10	19.92	15.48	14.73	25.14	19.00	6.72
Compacter	0.051%	19.30	18.36	19.20	15.16	15.05	24.54	18.60	5.30
LoRA	0.595%	19.55	18.94	19.47	15.48	14.76	24.80	18.83	7.63
IA ³	0.037%	19.06	18.22	18.88	14.94	14.34	24.32	18.29	5.04
T5									
Full	100%	18.48	18.13	17.57	13.80	12.58	22.66	17.20	13.30
Compacter	0.051%	16.61	15.41	15.50	11.58	11.81	20.73	15.27	3.35
LoRA	0.595%	18.10	17.25	16.46	12.75	12.70	21.82	16.51	4.65
IA ³	0.037%	16.68	15.47	15.40	11.29	11.14	20.75	15.12	3.19
CodeLlama									
Zero-Shot	0.000%	0.79	1.01	1.00	0.82	0.97	0.58	0.86	1.10
Compacter	0.012%	20.21	19.86	19.89	15.28	15.70	25.09	19.34	5.36
LoRA	0.093%	20.86	20.97	20.77	16.45	16.47	26.06	20.26	10.62
IA ³	0.006%	19.91	19.65	19.26	14.16	13.43	24.45	18.48	5.17
Llama2									
Zero-Shot	0.000%	0.91	1.06	0.93	0.76	0.87	0.69	0.87	1.04
Compacter	0.012%	18.75	18.53	18.71	14.08	14.51	23.18	17.96	5.19
LoRA	0.093%	19.06	19.15	18.72	14.86	15.21	24.10	18.52	9.52
IA ³	0.006%	18.03	18.30	17.87	13.56	11.26	22.63	16.94	4.80

Table 1 The BLEU-4 scores and percentage of trained parameters of the models fine-tuned with Compacter, LoRA and IA³ for code summarization per programming language and their average across the six seen programming languages. T5 and CodeT5 are fully fine-tuned and Llama2 and CodeLlama are sampled in a zero-shot setting as respective baselines. "*Seen*" denotes languages included in the pre-training phase of corresponding base models. "*Unseen*" denotes languages excluded from the pre-training phase of corresponding base models. A higher score is better. The darker the cell color, the better the score; heatmap is presented per each LLM and programming language.

For datasets that do not come in pre-defined train/validation/test splits (i.e., Python and R code generation datasets), we randomly select 4.95% and 1% for testing and validation, respectively. These proportions are necessary for effective training since the original datasets are relatively small (less than 40,000 samples), especially for the larger Llama family of models.

4 Results

This section explains the results. For better interpretation and comparison, we discuss the results of RQ1 and RQ2 separated by the tasks and model architectures together. We will first describe the obtained results for code summarization for T5 family of models and then for the Llama family of models in section 4.1. A similar process is followed to explain code generation results for both RQ1 and RQ2 in section 4.2. Then, we discuss the RQ3 results for knowledge transfer to unseen language, R, for both code summarization and code generation, in section 4.3.

For all experiments in which we report BLEU scores, including code summarization and code generation, we applied statistical tests to determine whether the obtained results were significant. For all cases, we found that the performance differences in terms of the BLEU-4 scores between each PEFT method and its respective baselines are significant, with $P < 0.001$. As the significant differences are a common observation, we report the p-value here and in the following, we discuss

PEFT	参数 训练过的 %	Seen							未见 R
		Python	Go	Java	JavaScript	Ruby	PHP	平均	
CodeT5									
Full	100%	19.65	19.10	19.92	15.48	14.73	25.14	19.00	6.72
紧凑型模型	0.051%	19.30	18.36	19.20	15.16	15.05	24.54	18.60	5.30
LoRA	0.595%	19.55	18.94	19.47	15.48	14.76	24.80	18.83	7.63
IA ³	0.037%	19.06	18.22	18.88	14.94	14.34	24.32	18.29	5.04
T5									
Full	100%	18.48	18.13	17.57	13.80	12.58	22.66	17.20	13.30
紧凑型模型	0.051%	16.61	15.41	15.50	11.58	11.81	20.73	15.27	3.35
LoRA	0.595%	18.10	17.25	16.46	12.75	12.70	21.82	16.51	4.65
IA ³	0.037%	16.68	15.47	15.40	11.29	11.14	20.75	15.12	3.19
CodeLlama									
零样本	0.000%	0.79	1.01	1.00	0.82	0.97	0.58	0.86	1.10
紧凑型模型	0.012%	20.21	19.86	19.89	15.28	15.70	25.09	19.34	5.36
LoRA	0.093%	20.86	20.97	20.77	16.45	16.47	26.06	20.26	10.62
IA ³	0.006%	19.91	19.65	19.26	14.16	13.43	24.45	18.48	5.17
Llama2									
零样本	0.000%	0.91	1.06	0.93	0.76	0.87	0.69	0.87	1.04
紧凑型模型	0.012%	18.75	18.53	18.71	14.08	14.51	23.18	17.96	5.19
LoRA	0.093%	19.06	19.15	18.72	14.86	15.21	24.10	18.52	9.52
IA ³	0.006%	18.03	18.30	17.87	13.56	11.26	22.63	16.94	4.80

表 1使用 Pacter、LoRA 和 IA³ 微调的模型在代码总结任务中的 BLEU-4 分数及训练参数百分比，按编程语言分类及六个已见编程语言的平均值。T5 和 CodeT5 完全微调，Llama2 和 CodeLlama 在零样本设置下作为各自的基线。"已见" 表示包含在对应基础模型预训练阶段的语言。"未见" 表示未包含在对应基础模型预训练阶段的语言。分数越高越好。单元格颜色越深，表示得分越好；热图按每个大模型和编程语言展示。

对于没有预定义训练/验证/测试划分的数据集（即Python和R代码生成数据集），我们随机选择4.95%用于测试，1%用于验证。这些比例对于有效训练是必要的，因为原始数据集相对较小（少于40,000个样本），尤其是对于较大的Llama系列模型。

4 结果

本节阐述了结果。为了更好地解释和比较，我们将根据任务和模型架构分别讨论 RQ1和RQ2的结果。首先在第4.1节中描述T5系列模型在代码总结任务中的所得结果，然后介绍Llama系列模型的相关结果。第4.2节采用类似的方法，解释RQ1和RQ2在代码生成任务中的结果。随后，在第4.3节中，我们讨论RQ3关于知识迁移到未见过的语言R的结果，涵盖代码总结和代码生成两个方面。

在所有报告BLEU分数的实验中，包括代码摘要和代码生成，我们都进行了统计检验以确定所得结果的显著性。在所有情况下，我们发现各个参数高效微调（PEFT）方法与其对应基线之间在BLEU-4分数方面的性能差异具有统计学显著性， $P < 0.001$ 。由于显著差异是常见的观察结果，本文在此报告p值，并在后续讨论中继续提及。

the results for each RQ. Please note that as the Pass@1 results are deterministic, there is no need to apply statistical tests and comparing the Pass@1 scores suffice.

4.1 Code Summarization

T5 Family of Models. Table 1 illustrates the performance of each PEFT technique for the code summarization task. Among the PEFT techniques applied to CodeT5, an encoder-decoder code-LLM, LoRA achieves the highest average BLEU-4 score among the three PEFT techniques across all six programming languages, with an average score of 18.83, followed by Compacter with 18.60 and IA³ with 18.29. Notably, the fully fine-tuned CodeT5 outperforms all three PEFT techniques in this setting; Compared to full fine-tuning, LoRA, Compacter and IA³ degrade summarization performance by 0.89%, 2.11% and 3.74%, respectively. The observed order of performance obtained by the three PEFT techniques is directly correlated with the percentage of trained parameters per each PEFT technique, where LoRA with the largest trainable parameter budget (0.595% of total parameters) performs best, Compacter with the second largest parameter budget (0.051% of total parameters) stands in the middle performance-wise, and IA³ with the smallest trainable parameter budget (0.037% of total parameters) perform worst.

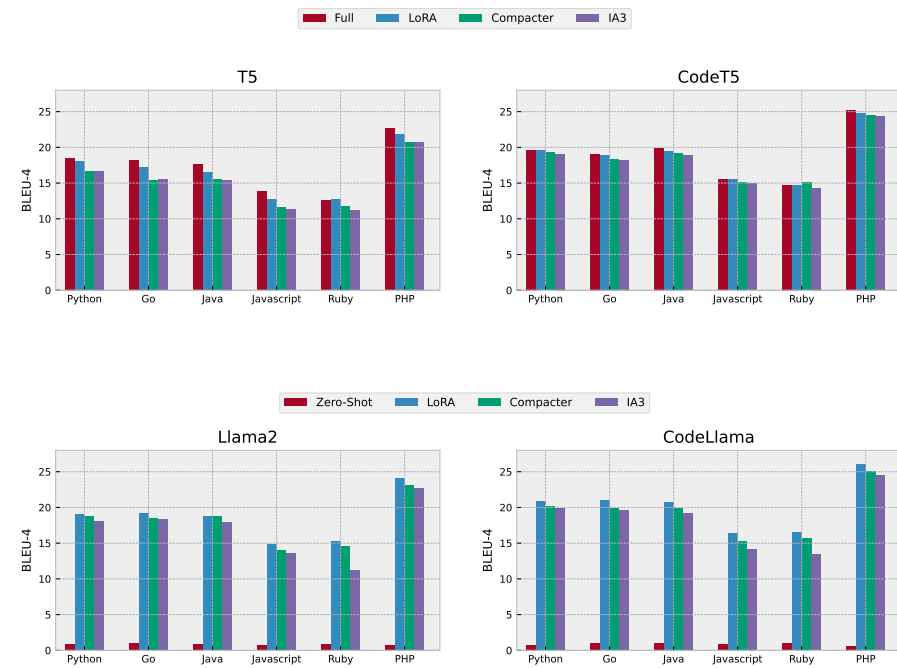


Fig. 1 BLEU-4 scores of LoRA, Compacter and IA³ per programming language for code summarization on T5 (top left), CodeT5 (top right), Llama2 (bottom left) and CodeLlama (bottom right) and their respective baselines.

每个研究问题 (RQ) 的结果。请注意, 由于Pass@1结果是确定性的, 无需进行统计检验, 仅比较Pass@1分数即可。

4.1 代码总结

T5模型家族表1展示了每种PEFT技术在代码总结任务中的性能。在应用于CodeT5 (一种编码器-解码器代码大模型) 的PEFT技术中, LoRA在所有六种编程语言中实现了最高的平均BLEU-4分数, 平均分为18.83, 其次是Compacter (18.60) 和IA³ (18.29)。值得注意的是, 完全微调的CodeT5在此设置中优于所有三种PEFT技术; 与完全微调相比, LoRA、Compacter和IA³ 的总结性能分别下降了0.89%、2.11%和3.74%。这三种PEFT技术的性能顺序与每种技术所训练参数的比例直接相关, 其中参数可训练比例最大的LoRA (占总参数的0.595%) 表现最佳, 参数比例第二的Compacter (占总参数的0.051%) 居中, 而参数比例最小的IA³ (占总参数的0.037%) 表现最差。

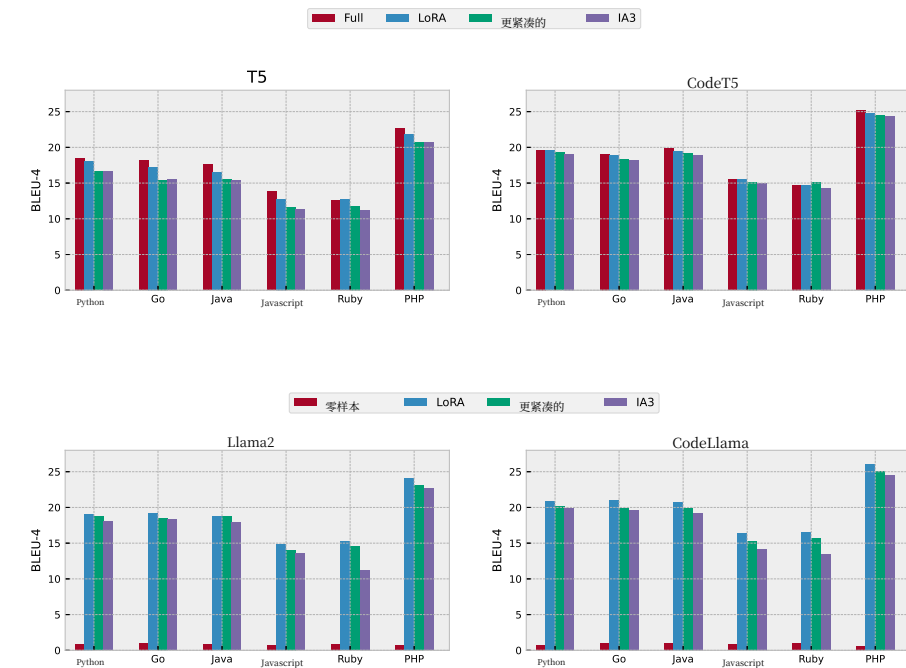


图 1 LoRA、Compacter 和 IA³ 在 T5 (左上)、CodeT5 (右上)、Llama2 (左下) 和 CodeLlama (右下) 上进行代码总结任务的 BLEU-4 分数及其各自的基线。

The performance trend of all three PEFT techniques remains consistent across all programming languages for code summarization on CodeT5, with an exception for Ruby, where Compacter outperforms other fine-tuning techniques with a BLEU-4 score of 15.05 and Javascript, where LoRA performs on par with full fine-tuning, obtaining a BLEU-4 score of 15.48 (see top right plot of Fig. 1). For the T5 model, LoRA with an average BLEU-4 score of 16.51 outperforms Compacter with an average score of 15.27 and IA³ with an average score of 15.12 on T5, resulting in a similar PEFT technique effectiveness ranking when compared to CodeT5, while full fine-tuning T5 yields the highest BLEU-4 score of 17.20, outperforming LoRA, Compacter and IA³ for code summarization by 4.18%, 12.64% and 13.76%, respectively.

While among all the PEFT techniques applied to T5, LoRA is dominant for all six programming languages, Compacter and IA³ do not perform consistently relative to each other (Fig. 1, top left). Specifically, Compacter is more advantageous in Java, Javascript and Ruby, while for other programming languages, IA³ slightly outperforms Compacter. Overall, however, both Compacter and IA³ perform very closely in terms of BLEU-4 score, with no clear top performer on the T5 model.

For code summarization, all three PEFT techniques prove more beneficial on CodeT5 than T5 when compared to their fully fine-tuned counterparts. On CodeT5, the most effective PEFT technique (i.e. LoRA) results in a 0.89% performance drop when compared to full fine-tuning, while on T5, this performance gap increases to 4.01%. This difference in effectiveness can be rooted in the fact that PEFT techniques utilize a limited budget of trainable parameters during fine-tuning. Due to this limitation, they are more suitable for leveraging present knowledge in the base model and adapting it for better usage of that knowledge than teaching the model new information.

Llama Family of Models. Table 1 presents the results for each PEFT technique applied to CodeLlama and Llama2 in the code summarization task. For CodeLlama, LoRA achieves the highest average BLEU-4 score across all six programming languages, with an average score of 20.26, followed by Compacter at 19.34 and IA³ at 18.48. This mirrors the pattern seen in the T5 family models, where LoRA, with its relatively larger percentage of trainable parameters (0.093%), outperforms the other PEFT techniques. In comparison to zero-shot performance, where CodeLlama scores an average of just 0.86, all PEFT techniques significantly boost the BLEU-4 scores, emphasizing the importance of fine-tuning. In terms of individual language performance on CodeLlama, LoRA demonstrates superiority across the board, performing best in all languages, particularly in Javascript, where LoRA surpasses Compacter and IA³ by 7.66% and 16.17%, respectively (Fig. 1, bottom right).

For Llama2, the overall ranking of PEFT techniques remains consistent with CodeLlama. LoRA achieves the highest average score of 18.52, followed by Compacter at 17.96 and IA³ at 16.94. As with CodeLlama, zero-shot results for Llama2 are substantially lower, with an average score of just 0.87. However, despite the highly comparable performance of CodeLlama and Llama2 in the zero-shot setting in terms of BLEU-4, the performance gains for PEFT techniques on Llama2 are slightly lower than those observed for CodeLlama. This suggests that CodeL-

在CodeT5上进行代码总结的所有三种PEFT技术在所有编程语言中的性能趋势保持一致，除了Ruby，在该语言中，Compacter以BLEU-4得分15.05优于其他微调技术；在JavaScript中，LoRA的表现与全量微调相当，获得BLEU-4得分15.48（见图1右上角的图）。对于T5模型，LoRA的平均BLEU-4得分为16.51，优于Compacter的平均得分15.27和IA³的平均得分15.12，在T5上的表现与CodeT5的PEFT技术效果排名类似，而全量微调T5的BLEU-4最高为17.20，比LoRA、Compacter和IA³分别高出4.18%、12.64%和13.76%，在代码总结任务中表现最佳。

在应用于T5的所有PEFT技术中，LoRA在六种编程语言中均占主导地位，Compacter和IA³的表现则不一致（图1，左上角）。具体而言，Compacter在Java、Javascript和Ruby中具有更明显的优势，而在其他编程语言中，IA³略微优于Compacter。然而，总体而言，Compacter和IA³在BLEU-4分数方面表现非常接近，T5模型中没有明显的最佳表现者。

在代码总结任务中，所有三种PEFT技术在CodeT5上相较于完全微调的对应模型表现出更大的优势。在CodeT5上，最有效的PEFT技术（即LoRA）相比完全微调的性能下降仅为0.89%，而在T5上，这一性能差距则扩大到4.01%。这种效果差异的根本原因在于PEFT技术在微调过程中使用的可训练参数预算有限。由于这一限制，它们更适合利用基础模型中已有的知识并对其进行适应性调整，以实现对该知识的更好利用，而非向模型灌输新信息。

Llama 系列模型。 表 1 展示了在代码总结任务中应用于 CodeLlama 和 Llama2 的每种 PEFT 技术的结果。对于 CodeLlama，LoRA 在所有六种编程语言中的平均 BLEU-4 分数最高，平均得分为 20.26，其次是 Compacter，得分为 19.34，以及 IA³，得分为 18.48。这与 T5 系列模型中观察到的模式类似，LoRA 由于其相对较大的可训练参数比例（0.093%），优于其他 PEFT 技术。与零-shot 性能相比，CodeLlama 的平均得分仅为 0.86，所有 PEFT 技术都显著提升了 BLEU-4 分数，强调了微调的重要性。在 CodeLlama 的单个语言性能方面，LoRA 在所有语言中表现出优越性，尤其是在 Javascript 中，LoRA 比 Compacter 和 IA³ 分别高出 7.66% 和 16.17%（图 1，右下角）。

对于Llama2，PEFT技术的整体排名与CodeLlama保持一致。LoRA以18.52的平均分获得最高，其次是Compacter（17.96）和IA³（16.94）。与CodeLlama一样，Llama2的零样本结果明显较低，平均分仅为0.87。然而，尽管在BLEU-4指标上，CodeLlama和Llama2在零样本设置中的表现高度相似，PEFT技术在Llama2上的性能提升略低于在CodeLlama上的表现。这表明CodeL-

PEFT	Parameters trained %	CodeBLEU	BLEU-4	EM@1	Pass@1
<i>CodeT5</i>					
Full	100%	25.60	41.69	14.56	6.10
Compacter	0.051%	6.05	29.23	10.79	2.44
LoRA	0.595%	22.32	42.01	14.90	5.49
IA ³	0.037%	8.83	28.20	8.97	1.83
<i>T5</i>					
Full	100%	22.59	42.74	0.00	0.00
Compacter	0.051%	4.19	15.93	0.00	0.00
LoRA	0.595%	12.76	34.01	0.00	0.00
IA ³	0.037%	3.69	15.36	0.00	0.00
<i>CodeLlama</i>					
Zero-Shot	0.000%	26.37	5.28	0.00	4.88
Compacter	0.012%	26.17	43.31	14.63	21.95
LoRA	0.093%	31.10	47.67	16.72	26.83
IA ³	0.006%	26.25	43.07	14.97	7.93
<i>Llama2</i>					
Zero-Shot	0.000%	23.67	4.30	0.00	2.44
Compacter	0.012%	26.02	42.25	14.23	12.80
LoRA	0.093%	29.07	46.11	15.71	15.24
IA ³	0.006%	21.86	39.95	12.20	9.15

Table 2 The CodeBLEU, BLEU-4, EM@1 and Pass@1 scores and percentage of trained parameters of the models fine-tuned with Compacter, LoRA and IA³ for code generation in Python. T5 and CodeT5 are fully fine-tuned and Llama2 and CodeLlama are sampled in a zero-shot setting as respective baselines. Higher is better. The darker the color, the better the score; heatmap is presented per each model and metric.

lama, specifically designed for coding tasks, benefits more from these fine-tuning techniques than the general-purpose Llama2.

LoRA maintains its dominance when applied to Llama2 where it consistently performs better than Compacter and IA³. Unlike when applied to CodeLlama however, Compacter better closes the performance gap with LoRA on Llama2, specifically for Python and Java, where it lags behind only 1.65% and 0.05%, respectively (Fig. 1, bottom left).

IA³, despite its small parameter budget (0.006%), manages to keep pace in some cases but generally lags behind LoRA and Compacter across most languages. While IA³ performs close to Compacter on the T5 and CodeT5 models, the performance gap is slightly larger on the Llama family of models, which might be related to its small parameter budget.

Overall, LoRA has the best score across a broad range of programming languages, which might be due to its larger trainable parameter allocation compared to other PEFT techniques. In Section 5, we provide an analysis in a specific setting to compare the performance of LoRA and Compacter with respect to their parameter budget.

PEFT	参数训练的%	CodeBLEU	BLEU-4	EM@1	Pass@1
<i>CodeT5</i>					
Full	100%	25.60	41.69	14.56	6.10
更紧凑的	0.051%	6.05	29.23	10.79	2.44
LoRA	0.595%	22.32	42.01	14.90	5.49
IA ³	0.037%	8.83	28.20	8.97	1.83
<i>T5</i>					
Full	100%	22.59	42.74	0.00	0.00
更紧凑的	0.051%	4.19	15.93	0.00	0.00
LoRA	0.595%	12.76	34.01	0.00	0.00
IA ³	0.037%	3.69	15.36	0.00	0.00
<i>CodeLlama</i>					
零样本	0.000%	26.37	5.28	0.00	4.88
更紧凑的	0.012%	26.17	43.31	14.63	21.95
LoRA	0.093%	31.10	47.67	16.72	26.83
IA ³	0.006%	26.25	43.07	14.97	7.93
<i>Llama2</i>					
零样本	0.000%	23.67	4.30	0.00	2.44
更紧凑的	0.012%	26.02	42.25	14.23	12.80
LoRA	0.093%	29.07	46.11	15.71	15.24
IA ³	0.006%	21.86	39.95	12.20	9.15

表 2使用 Compacter、LoRA 和 IA³ 微调的模型在 Python 代码生成任务中的 CodeBLEU、BLEU-4、EM@1 和 Pass@1 分数及训练参数比例。T5 和 CodeT5 进行了完全微调，Llama2 和 CodeLlama 在零样本设置下采样，作为各自的基线。分数越高越好。颜色越深，表示得分越好；热图显示每个模型和指标的情况。

lama，专为编码任务设计，在这些微调技术方面比通用的Llama2受益更大。

LoRA在应用于Llama2时仍然保持其优势，表现始终优于Compacter和IA³。然而，当应用于CodeLlama时，Compacter在Llama2上的性能差距被更好地缩小，特别是在Python和Java方面，其性能仅落后1.65%和0.05%，如图1左下角所示。

尽管IA³，的参数预算较小（0.006%），在某些情况下仍能保持一定的性能，但在大多数语言中通常落后于LoRA和Compacter。虽然IA³在T5和CodeT5模型上与Compacter的表现接近，但在Llama系列模型上的性能差距略大，这可能与其较小的参数预算有关。

总体而言，LoRA在广泛的编程语言中都取得了最佳成绩，这可能归因于其相较于其他PEFT技术具有更大的可训练参数分配。在第5节中，我们在特定设置下对比了LoRA和Compacter在参数预算方面的性能表现。

For the code summarization in the six seen languages, LoRA performs best, followed by Compacter which slightly outperforms IA³ on CodeT5 and T5. A similar performance trend appears on CodeLlama and Llama2, though the gap between Compacter and IA³ is larger for the Llama models. All three PEFT methods demonstrate better effectiveness on code-LLMs, but the performance gap between the two categories of models suggests that all three methods are effective in code summarization knowledge transfer on general-LLMs for the seen languages.

4.2 Code Generation

The results of all models for code generation are shown in Table 2.

In addition to BLEU-4 which was also used for evaluating code summarization, we utilize CodeBLEU, EM@1 and Pass@1 for code generation. However, we consider Pass@1 as the main performance metric for code generation given that HumanEval evaluation problems are designed to represent real-world scenarios not included in the training datasets. Moreover, we refrain from discussing EM@1 in detail, as some EM@1 scores are zero, but most EM@1 results are generally tightly coupled with BLEU-4 as the main textual similarity metric.

T5 Family of Models. The gap in performance between LoRA, Compacter and IA³ increases for Python code generation in contrast to code summarization, as illustrated in Table 2.

On CodeT5, besides fully fine-tuning, which proves most effect with a Pass@1 score of 6.10, LoRA is the best-performing PEFT followed by Compacter and IA³ with Pass@1 scores of 5.49, 2.44 and 1.83, respectively.

Interestingly, when strictly looking at the BLEU-4 and EM@1 scores, LoRA appears to outperform all other fine-tuning techniques with a BLEU-4 score of 42.01, followed by fully fine-tuning with a score of 41.69, Compacter with a score of 29.23 and IA³ with a score of 28.20. The EM@1 scores follow an identical trend. While the higher BLEU-4 and EM@1 scores of LoRA compared to the fully fine-tuned CodeT5 suggests that LoRA performs better in generating textually similar to ground truth samples, it does not mean it generalizes better to out-of-distribution code generation problems, judged by higher Pass@1 score for the fully fine-tuned model. LoRA is the next high Pass@1, with a score of 5.49 compared to 6.10 for the fully fine-tuned CodeT5.

Moreover, it can be observed that despite relatively close BLEU-4 scores of Compacter and IA³ (29.23 and 28.20, respectively), their CodeBLEU scores are more apart, with scores of 6.05 and 8.83, respectively. This discrepancy in difference and trend can be attributed to the fact that the BLEU-4 score is strictly concerned with textual similarity. On the other hand, besides textual similarity, CodeBLEU additionally accounts for the type of tokens and the similarity in the abstract tree and the data flow. This means that between Compacter and IA³, while the former generates more textually similar to ground truth code snippets, the latter generations are closer to the ground truth in terms of functionality. Consequently, the same

在六种已见语言的代码总结任务中, LoRA表现最佳, 其次是Compacter, 后者略优于IA³在CodeT5和T5上的表现。在CodeLlama和Llama2上也出现了类似的性能趋势, 尽管Compacter与IA³之间的差距在Llama模型中更大。所有三种PEFT方法在代码大模型 (code-LLMs) 上都显示出更好的效果, 但两类模型之间的性能差异表明, 这三种方法在已见语言的通用大模型 (general-LLMs) 中的代码总结知识迁移方面均具有有效性。

4.2 代码生成

所有模型在代码生成任务中的结果如表2所示。

除了用于评估代码摘要的BLEU-4外, 我们还采用了CodeBLEU、EM@1和Pass@1进行代码生成。然而, 考虑到HumanEval评估问题旨在反映训练数据集中未包含的实际场景, 我们将Pass@1视为代码生成的主要性能指标。此外, 我们避免详细讨论EM@1, 因为一些EM@1得分为零, 但大多数EM@1结果通常与BLEU-4紧密相关, 作为主要的文本相似性指标。

T5模型家族。在Python代码生成方面, LoRA、Compacter和IA³之间的性能差距比代码总结任务中更为显著, 如表2所示。

在CodeT5上, 除了完全微调 (其Pass@1得分为6.10) 表现最佳外, LoRA是性能最优的参数高效微调方法, 其次是Compacter和IA³, Pass@1得分分别为5.49、2.44和1.83。

有趣的是, 当严格观察 BLEU-4 和 EM@1 分数时, LoRA 似乎优于所有其他微调技术, BLEU-4 分数为 42.01, 其次是完全微调, 得分为 41.69, Compacter 的得分为 29.23, IA³ 的得分为 28.20。EM@1 分数也呈现出相同的趋势。虽然 LoRA 相较于完全微调的 CodeT5 具有更高的 BLEU-4 和 EM@1 分数, 表明 LoRA 在生成与真实样本文本相似的文本方面表现更佳, 但这并不意味着它在外部分布代码生成问题上的泛化能力更强, 从 Pass@1 分数来看, 完全微调模型的表现更优。LoRA 的 Pass@1 分数为 5.49, 紧随其后的是完全微调的 CodeT5, 得分为 6.10。

此外, 可以观察到, 尽管Compacter和IA³的BLEU-4分数相对接近, 分别为29.23和28.20, 但它们的CodeBLEU分数差异更大, 分别为6.05和8.83。这种差异和趋势的差异可以归因于BLEU-4分数严格关注文本相似性。另一方面, 除了文本相似性之外, CodeBLEU还考虑了标记类型、抽象树的相似性以及数据流的相似性。这意味着在Compacter和IA³之间, 前者在生成与真实代码片段文本更相似方面表现更好, 而后者在功能上更接近真实代码。因此, 二者在性能表现上存在不同的侧重点。

is true for LoRA and fully fine-tuning with CodeBLEU scores of 22.32 and 25.60 respectively, where LoRA's generations are more similar textually and less similar functionally to the ground truth compared to fully fine-tuning.

T5's performance in code generation is notably poor, as none of the code snippets produced by any of the fine-tuning methods are capable of successfully solving any HumanEval problems, all with Pass@1 equal to zero. Nevertheless, besides fully fine-tuning with a BLEU-4 score of 42.74 and a CodeBLEU score of 22.59, LoRA is the best-performing fine-tuning technique with a BLEU-4 score of 34.01 and a CodeBLEU score of 12.76. Among Compacter and IA³, the former's generations are slightly more aligned with the ground truth in terms of textual similarity, as observed by a slightly better BLEU-4 score, while the latter's generations are more functionally similar to the ground truth, as indicated by higher CodeBLEU.

The zero Pass@1 scores obtained by all fine-tuned T5 models suggest that even though these models learn to generate code snippets from their training dataset's distribution, none of the fine-tuning techniques are capable of adapting and generalizing to real-world code generation problems. LoRA on CodeT5 is, however, capable of generalizing to real-world problems to some extent, reaching close to fully fine-tuned CodeT5 on the HumanEval benchmark.

Overall, for CodeT5, all fine-tuning techniques obtain very low Pass@1 scores; indicating that T5 family are not capable of generating executable code.

Llama Family of Models. Aligned with previous observations, CodeLlama is the best-performing model for Python code generation when fine-tuned using LoRA. LoRA yields the highest HumanEval Pass@1 score of 26.83, followed by 21.95 of Compacter and 7.93 of IA³, which are significant performance boosts compared to the baseline zero-shot setting with a score of 4.88.

Considering other metrics, LoRA achieves the highest BLEU-4 score of 47.67, CodeBLEU score of 31.10 and EM@1 score of 16.72. Similar to T5 family models, Compacter and IA³ are ordered differently when ranked by BLEU-4 CodeBLEU, even though the discrepancy is not as large. Compacter slightly outperforms IA³ in textual similarity (i.e., BLEU-4), while IA³ slightly outperforms Compacter in functionality similarity (i.e., CodeBLEU). In comparison, the differences in textual and functional similarities of CodeLlama's zero-shot setting generations are more noticeable, with a BLEU-4 score of 5.28 and a CodeBLEU score of 26.37.

The general decoder-only Llama2's code generation results are similar to CodeLlama in terms of comparison between the PEFT metrics. On Llama2, LoRA gains the highest Pass@1 of 15.24, followed by Compacter with 12.80 and IA³ with 9.15. The zero-shot scores of the Llama2 model obtain a relatively low Pass@1 score of 2.44. The BLEU-4, CodeBLEU and EM@1 scores of PEFT methods are ranked in the same order, LoRA being the highest, followed by Compacter and IA³ for all three metrics. Similar to CodeLlama, Llama2 obtains a much higher CodeBLEU score compared to its BLEU-4 score in the zero-shot setting.

For both Llama2 and CodeLlama in the zero-shot setting, we observe a discrepancy between the BLEU-4 and CodeBLEU scores. This is due to the fact that before any fine-tuning, both models will not stop their generation after completing the target function's body, repeating the function definition until they hit the maximum token ceiling. Unlike BLEU-4 which penalizes predictions that are longer

对于 LoRA 和完全微调而言, CodeBLEU 分别为 22.32 和 25.60, LoRA 的生成在文本上更为相似, 而在功能上与真实值相比则较少相似。

T5在代码生成方面的表现明显较差, 任何微调方法生成的代码片段都无法成功解决任何HumanEval问题, 所有的Pass@1均为零。然而, 除了完全微调的 BLEU-4得分为42.74、CodeBLEU得分为22.59之外, LoRA是表现最佳的微调技术, 其BLEU-4得分为34.01, CodeBLEU得分为12.76。在Compacter和IA³之间, 前者的生成结果在文本相似性方面略微更接近真实值, 表现为略高的BLEU-4得分, 而后的生成结果在功能相似性方面更接近真实值, 表现为更高的CodeBLEU。

所有微调T5模型获得的零Pass@1分数表明, 尽管这些模型能够从其训练数据集的分布中学习生成代码片段, 但没有一种微调技术能够适应并推广到实际的代码生成问题。然而, 基于CodeT5的LoRA在一定程度上能够推广到实际问题, 在 HumanEval基准测试中接近完全微调的CodeT5。

总体而言, 对于CodeT5, 所有微调技术的Pass@1分数都非常低; 表明T5系列模型无法生成可执行代码。

Llama 系列模型与之前的观察一致, CodeLlama 在使用 LoRA 微调后在 Python 代码生成方面表现最佳。LoRA 实现了最高的 HumanEval Pass@1 分数 26.83, 其次是 Compacter 的21.95 和 IA³, 的7.93, 与基线零样本设置的4.88相比, 性能提升显著。

考虑其他指标后, LoRA在BLEU-4得分为47.67、CodeBLEU得分为31.10和 EM@1得分为16.72方面表现最佳。与T5系列模型类似, Compacter和IA³ 在按 BLEU-4和CodeBLEU排名时顺序不同, 尽管差异不如前者明显。Compacter在文本相似性(即BLEU-4)方面略优于IA³, 而IA³ 在功能相似性(即CodeBLEU)方面略优于Compacter。相比之下, CodeLlama在零样本设置生成的文本和功能相似性差异更为明显, BLEU-4得分为5.28, CodeBLEU得分为26.37。

仅使用解码器的Llama2在代码生成结果方面与CodeLlama在PEFT指标的比较中类似。在Llama2上, LoRA的Pass@1最高, 为15.24, 其次是Compacter, 为12.80, 以及IA³, 为9.15。Llama2模型的零-shot得分相对较低, Pass@1为2.44。PEFT方法的BLEU-4、CodeBLEU和EM@1得分排名相同, LoRA最高, 其次是Compacter和IA³, 在这三个指标中均如此。与CodeLlama类似, Llama2在零-shot设置中获得的CodeBLEU分数远高于其BLEU-4分数。

在零样本设置下, Llama2 和 CodeLlama 的 BLEU-4 和 CodeBLEU 分数之间存在差异。这是因为在任何微调之前, 两个模型在完成目标函数体后不会停止生成, 而是会重复函数定义, 直到达到最大令牌数限制。与 BLEU-4 不同, 后者会惩罚较长的预测结果

than the ground truth, CodeBLEU ignores the prediction length. This means that a significantly lengthy prediction can obtain a high CodeBLEU score along with a low BLEU-4 score, given it contains a large number of similar-to-reference code constructs.

Comparing the similar BLEU-4 and CodeBLEU scores of Llama2 and CodeLlama suggests all PEFT techniques are capable of transferring most of the information contained in the training dataset to adapt both models for its distribution adequately. On the other hand, when dealing with out-of-distribution real-world HumanEval problems, CodeLlama still outperforms Llama2 by a large margin as obvious by Pass@1 scores.

For code generation in the six seen languages, LoRA significantly outperforms Compacter and IA³ on the T5 and Llama family of models. While Compacter and IA³ produce similar results in some cases, the former generally achieves higher functional accuracy (i.e., the Pass@1 score). All three PEFT methods consistently show better effectiveness when applied to code-LLMs, with an exception for IA³, which performs slightly better in terms of functional accuracy (i.e., the Pass@1 score) when applied to Llama2 compared to CodeLlama.

4.3 Knowledge Transfer to Unseen Language R

R serves as an unseen programming language for evaluating each PEFT technique's ability to transfer knowledge to unseen domains. Following the previous structure, we discuss R code summarization and R code generation separately, with results provided in Table 1 and Table 3, respectively.

4.3.1 R Code Summarization

T5 Family of Models. As shown in Table 1, for CodeT5, LoRA stands out as the top-performing technique, achieving a BLEU-4 score of 7.63, which surpasses even the fully fine-tuned model's score of 6.72. Compacter and IA³, though also effective, result in lower BLEU-4 scores of 5.30 and 5.04, respectively, demonstrating a noticeable drop in performance compared to LoRA and full fine-tuning. For the T5 model, however, a different trend is observed. Fully fine-tuning the T5 model results in the highest BLEU-4 score of 13.30, significantly outperforming all PEFT techniques. Among the PEFT techniques, LoRA again shows the best performance with a BLEU-4 score of 4.65, followed by Compacter at 3.35 and IA³ at 3.19.

Notably, fully fine-tuning the model results in inconsistent performance between CodeT5 and T5. The fully fine-tuned CodeT5 performance is within the performance range of PEFT techniques on this model. Specifically, LoRA outperforms

比起真实值, CodeBLEU 忽略了预测长度。这意味着, 预测长度显著较长的预测可以获得较高的 CodeBLEU 分数, 同时 BLEU-4 分数较低, 前提是其包含大量与参考代码结构相似的代码构造。

比较Llama2与CodeLlama的BLEU-4和CodeBLEU得分相似性表明, 所有的参数高效微调 (PEFT) 技术都能够将训练数据集中包含的大部分信息转移, 从而充分适应两种模型的分布。另一方面, 在处理分布外的真实世界HumanEval问题时, CodeLlama仍然显著优于Llama2, 尤其在Pass@1评分方面表现突出。

在六种已见语言的代码生成任务中, LoRA在T5和Llama系列模型上显著优于Compacter和IA³。虽然Compacter和IA³在某些情况下产生类似的结果, 但前者通常在功能准确性 (即Pass@1分数) 方面表现更优。

所有三种PEFT方法在应用于代码大规模语言模型 (code-LLMs) 时均表现出更优的效果, 唯有IA³, 在功能准确性 (即Pass@1分数) 方面略优于CodeLlama时, 应用于Llama2的表现稍好一些。

4.3 未见语言 R 的知识迁移

R 作为一种未被广泛使用的编程语言, 用于评估每种 PEFT 方法将知识迁移到未见领域的能力。遵循前述结构, 我们分别讨论 R 代码总结和 R 代码生成, 相关结果分别列于表 1 和表 3。

4.3.1 R 代码总结

T5模型家族如表1所示, 对于CodeT5, LoRA表现出色, 成为性能最优的技术, 达到7.63的BLEU-4分数, 超过了全模型微调的6.72分。Compacter和IA³, 虽然也有效, 但分别获得了较低的BLEU-4分数, 分别为5.30和5.04, 显示出与LoRA和全微调相比性能明显下降。然而, 对于T5模型, 则观察到不同的趋势。对T5模型进行全微调后, 获得了最高的BLEU-4分数13.30, 显著优于所有PEFT技术。在PEFT技术中, LoRA再次表现最佳, BLEU-4分数为4.65, 其次是Compacter, 为3.35, IA³为3.19。

值得注意的是, 完全微调模型导致CodeT5与T5之间的性能不一致。完全微调的CodeT5性能位于该模型上PEFT技术的性能范围内。具体而言, LoRA的表现优于

the fully fine-tuned CodeT5, despite updating a fraction of the model’s parameters. In Contrast, the fully fine-tuned T5 model significantly outperforms the best-performing PEFT technique on this model, even surpassing the fully fine-tuned CodeT5.

Without considering the fully fine-tuned models, however, the performance ranking on the T5 family of models is consistent with the performance obtained for the six seen programming languages, with LoRA performing the best, followed by Compacter and IA³.

Llama Family of Models. When shifting focus to CodeLlama, the performance of PEFT techniques improves significantly compared to their zero-shot baseline. Without any fine-tuning, CodeLlama achieves a BLEU-4 score of 1.10, but after applying LoRA, this score jumps to 10.62, a substantial improvement that highlights LoRA’s effectiveness for this model. Compacter and IA³ also boost performance, with BLEU-4 scores of 5.36 and 5.17, respectively. However, LoRA clearly dominates in this setting, showing a much higher capability in enhancing CodeLlama’s R summarization abilities compared to the other PEFT techniques. Llama2 follows the same trend as CodeLlama, where the zero-shot BLEU-4 score is relatively low at 1.04. Once again, LoRA emerges as the best-performing fine-tuning method with a BLEU-4 score of 9.52, significantly improving the model’s ability to summarize R code. Compacter and IA³, with scores of 5.19 and 4.80, respectively, also show improvement but fall short of LoRA’s performance.

On the Llama family of models, the performance ranking of the PEFT techniques remains the same, where LoRA showcases the highest performance, followed by Compacter and IA³. However, the performance gap between LoRA and the other two PEFT techniques on the Llama family of models increases when compared to the T5 family of models.

Across all four models, LoRA consistently performs the best among the PEFT techniques for R code summarization. While full fine-tuning remains the most effective approach for T5, LoRA demonstrates the ability to achieve comparable results with fewer parameter updates. On CodeT5, LoRA even manages to outperform full fine-tuning, showcasing its potential to not only match but sometimes exceed traditional methods. Compacter and IA³, while still beneficial, generally lag behind LoRA in terms of performance, particularly on the Llama family of models.

Compared to code summarization in the six seen languages, LoRA outperforms Compacter and IA³ in the unseen setting, R code summarization, with a larger gap on both the T5 and Llama family of models. Although Compacter performs better than IA³ in this setting, the performance gap between them is smaller compared to their gap with LoRA. On the Llama family of models, all three PEFT methods achieve similar performance between the code-LLM and general-LLM variants. This performance difference increases for all three methods on the T5 family of models.

尽管只更新了模型参数的一小部分，完全微调的CodeT5仍然表现出色。相比之下，完全微调的T5模型在该模型上显著优于表现最好的参数高效微调（PEFT）技术，甚至超越了完全微调的CodeT5。

不考虑完全微调的模型时，T5系列模型的性能排名与在六种已见编程语言上的表现一致，LoRA表现最佳，其次是Compacter和IA³。

Llama 系列模型。**Llama Family of Models.** 当转向 CodeLlama 时，PEFT 技术的性能相比其零样本基线显著提升。在未进行任何微调的情况下，CodeLlama 的 BLEU-4 分数为 1.10，但应用 LoRA 后，该分数跃升至 10.62，显示出 LoRA 在该模型中的有效性。Compacter 和 IA³ 也提升了性能，BLEU-4 分别为 5.36 和 5.17。然而，在此设置中，LoRA 明显占优，展现出在增强 CodeLlama 的 R 代码总结能力方面远超其他 PEFT 技术的能力。Llama2 也遵循相同的趋势，其零样本 BLEU-4 分数较低，为 1.04。再次，LoRA 作为表现最佳的微调方法，BLEU-4 分数为 9.52，显著提升了模型的 R 代码总结能力。Compacter 和 IA³，的分数分别为 5.19 和 4.80，也有所提升，但仍不及 LoRA 的表现。

在Llama系列模型上，PEFT技术的性能排名保持不变，其中LoRA表现最佳，其次是Compacter和IA³。然而，与T5系列模型相比，LoRA与其他两种PEFT技术在Llama系列模型上的性能差距有所扩大。

在所有四个模型中，LoRA在R代码总结任务中始终是PEFT技术中表现最优的方法。虽然完全微调仍然是T5最有效的策略，但LoRA展现出在参数更新较少的情况下实现可比结果的能力。在CodeT5上，LoRA甚至优于完全微调，展示了其不仅可以匹配传统方法，有时还可以超越的潜力。Compacter和IA³，虽然仍具有一定优势，但在性能方面通常落后于LoRA，尤其是在Llama系列模型上。

与六种已见语言中的代码总结相比，在未见环境下的R代码总结中，LoRA优于Compacter和IA³，在T5和Llama系列模型上差距更大。虽然Compacter在此环境下的表现优于IA³，但它们之间的性能差距比与LoRA之间的差距要小。在Llama family模型中，所有三种PEFT方法在代码-LLM和通用-LLM变体之间的性能表现相似。对于T5系列模型，这三种方法的性能差异都在增加。

PEFT	Parameters trained %	BLEU-4	EM@1	Pass@1
CodeT5				
Full	100%	14.20	0.27	2.48
Compacter	0.051%	8.70	0.00	0.62
LoRA	0.595%	13.15	0.27	1.86
IA ³	0.037%	9.02	0.00	0.62
T5				
Full	100%	12.74	0.00	0.00
Compacter	0.051%	5.12	0.00	0.00
LoRA	0.595%	7.44	0.00	0.00
IA ³	0.037%	5.45	0.00	0.00
CodeLlama				
Zero-Shot	0.000%	8.22	0.00	11.18
Compacter	0.012%	24.81	0.91	18.01
LoRA	0.093%	26.30	0.81	20.50
IA ³	0.006%	24.42	0.91	17.39
Llama2				
Zero-Shot	0.000%	6.12	0.00	1.24
Compacter	0.012%	19.94	0.59	8.70
LoRA	0.093%	21.44	0.75	10.56
IA ³	0.006%	19.60	0.54	7.45

Table 3 The CodeBLEU, BLEU-4, EM@1 and Pass@1 scores and percentage of trained parameters of the models fine-tuned with Compacter, LoRA and IA³ for code generation in R. T5 and CodeT5 are fully fine-tuned and Llama2 and CodeLlama are sampled in a zero-shot setting as respective baselines. Higher is better. The darker the color, the better the score; heatmap is presented per each LLM and metric.

4.3.2 R Code Generation

For evaluating code generation on R, we omit the CodeBLEU metric, as we could not access a readily available R tree-sitter parser to calculate the CodeBLEU score, and a custom implementation is out of the scope of this study. The results are summarized in Table 3.

T5 Family of Models. Consistent with previous observations, fully fine-tuning the CodeT5 model achieves the highest Pass@1 and BLEU-4 score of 2.48 and 14.20, respectively. Closely following the fully fine-tuned CodeT5 model, LoRA achieves a Pass@1 score of 1.86 and a BLEU-4 score of 13.15. IA³ and Compacter rank lower than LoRA in terms of performance, with BLEU-4 scores of 9.02 and 8.70, respectively. Both IA³ and Compacter obtain the same Pass@1 score of 0.62. The EM@1 scores of all fine-tuning methods are arranged according to a similar pattern to the Pass@1 scores. Though LoRA achieves close scores to fully fine-tuned CodeT5’s BLEU-4 and Pass@1 scores, the low Pass@1 for all fine-tuning methods shows that the generated code is mostly non-executable for R.

Similar results are seen for the T5 model, where full fine-tuning has the highest BLEU-4, followed by LoRA. Compacter and IA³ have lower scores. Similar to code generation on Python, T5 fails to pass any HumanEval problems using any fine-tuning technique, with all Pass@1 scores equal to zero. This suggests that T5 is not

PEFT	参数训练过的%	BLEU-4	EM@1	Pass@1
CodeT5				
Full	100%	14.20	0.27	2.48
更紧凑的模型	0.051%	8.70	0.00	0.62
LoRA	0.595%	13.15	0.27	1.86
IA ³	0.037%	9.02	0.00	0.62
T5				
Full	100%	12.74	0.00	0.00
更紧凑的模型	0.051%	5.12	0.00	0.00
LoRA	0.595%	7.44	0.00	0.00
IA ³	0.037%	5.45	0.00	0.00
CodeLlama				
零样本	0.000%	8.22	0.00	11.18
更紧凑的模型	0.012%	24.81	0.91	18.01
LoRA	0.093%	26.30	0.81	20.50
IA ³	0.006%	24.42	0.91	17.39
Llama2				
零样本	0.000%	6.12	0.00	1.24
Compacter	0.012%	19.94	0.59	8.70
LoRA	0.093%	21.44	0.75	10.56
IA ³	0.006%	19.60	0.54	7.45

表 3 使用 Compacter、LoRA 和 IA³ 微调的模型在 R 语言代码生成任务中的 CodeBLEU、BLEU-4、EM@1 和 Pass@1 分数及训练参数比例。T5 和 CodeT5 采用完全微调，Llama2 和 CodeLlama 在零样本设置下采样，作为各自的基线。分数越高越好。颜色越深，表示得分越优；热图按每个大模型（LLM）和指标展示。

4.3.2 R 代码生成

在评估R语言的代码生成时，我们省略了CodeBLEU指标，因为无法获取现成的R树拆解析器以计算CodeBLEU分数，而自定义实现超出了本研究的范围。结果总结见表3。

T5 模型家族。 与之前的观察结果一致，完全微调的 CodeT5 模型在 Pass@1 和 BLEU-4 分数上分别达到了 2.48 和 14.20 的最高值。紧随完全微调的 CodeT5 模型之后，LoRA 实现了 1.86 的 Pass@1 分数和 13.15 的 BLEU-4 分数。IA³ 和 Compacter 在性能方面低于 LoRA，分别获得了 9.02 和 8.70 的 BLEU-4 分数。IA³ 和 Compacter 的 Pass@1 分数均为 0.62。所有微调方法的 EM@1 分数的排列方式与 Pass@1 分数类似。虽然 LoRA 在 BLEU-4 和 Pass@1 分数上接近完全微调的 CodeT5，但所有微调方法的低 Pass@1 表明生成的代码大多不可在 R 中执行。

在T5模型中也观察到类似的结果，其中完全微调的BLEU-4得分最高，其次是LoRA。Compacter和IA³ 的得分较低。与Python代码生成类似，T5在使用任何微调技术时都未能通过任何HumanEval问题，所有Pass@1得分均为零。这表明T5并不

capable of learning any meaningful code generation abilities for R when tasked with generating out-of-distribution unseen code snippets.

Overall, for R code generation, all fine-tuning techniques obtain very low Pass@1 scores, indicating that T5 family models are not capable of generating executable code for R. In terms of generation textual similarity, LoRA is the best-performing fine-tuning technique, with IA³ and Compacter following closely. However, the performance gap between LoRA and fully fine-tuned models is significantly larger for T5 compared to CodeT5, indicating that T5 benefits more from fully fine-tuning for R code generation.

Llama Family of Models. For R code generation using CodeLlama, LoRA obtains the highest BLEU-4 of 26.30, followed by Compacter (24.81) and IA³ (24.42). The zero-shot setup, which uses no fine-tuning, lags behind significantly, yielding a BLEU-4 score of 8.22. These results indicate that while fine-tuning boosts the model's ability to generate textually similar code, there is still room for improvement in overall performance compared to Python with BLEU-4 scores of above 40 for the PEFT techniques.

When evaluating CodeLlama on the HumanEval benchmark, the Pass@1 scores mirror the BLEU-4 results, with LoRA achieving the highest score at 20.50. Compacter follows with a score of 18.01, and IA³ scores slightly lower at 17.39. The zero-shot setup records a Pass@1 score of 11.18, demonstrating that even without fine-tuning, CodeLlama can generate some executable R code. However, fine-tuning considerably improves its ability to solve R code tasks, particularly with LoRA. Compared to Python using CodeLlama for code generation, the obtained Pass@1 scores are lower for R.

Shifting the focus to the Llama2 model, performance is lower across both BLEU-4 and Pass@1 metrics compared to CodeLlama. LoRA remains the top-performing technique with a BLEU-4 score of 21.44, followed by Compacter at 19.94 and IA³ at 19.60. The zero-shot BLEU-4 score is lower than CodeLlama's, standing at 6.12, which suggests that Llama2 is less capable of producing R code that aligns textually with the reference solutions when compared to its counterpart, CodeLlama.

For the HumanEval benchmark, Llama2's performance is lower than CodeLlama's. LoRA achieves the highest Pass@1 score of 10.56, followed by Compacter at 8.70, and IA³ at 7.45. The zero-shot setting performs poorly, recording a Pass@1 score of 1.24. These results highlight that Llama2 struggles more than CodeLlama to generate executable R code, even when fine-tuning techniques are applied. Similar results were observed for Python as discussed in Table 2.

Comparing the fine-tuning techniques on both Llama models, LoRA consistently delivers the highest BLEU-4 and Pass@1 scores, indicating that it is the most effective method for improving both text generation quality and executable code. Compacter and IA³ follow closely in terms of performance, with Compacter slightly outperforming IA³ in most cases. However, even with these fine-tuning techniques, the overall capability of both CodeLlama and Llama2 to generate executable R code remains limited.

Overall, for R code generation, the T5 family of models performs poorly in comparison to Llama models. CodeLlama achieves the best results among all models. While fine-tuning improves the models' ability to generate textually similar code

能够在被要求生成分布外未见代码片段时，学习任何有意义的 R 代码生成能力。

总体而言，在R代码生成方面，所有微调技术的Pass@1得分都非常低，表明T5系列模型无法生成可执行的R代码。在生成文本相似性方面，LoRA表现最佳，IA³和Compacter紧随其后。然而，LoRA与完全微调模型之间的性能差距在T5中明显大于在CodeT5中，表明T5在R代码生成方面更依赖于完全微调。

Llama 系列模型。对于使用 CodeLlama 进行 R 代码生成，LoRA 的 BLEU-4 达到 26.30，次之是 Compacter (24.81) 和 IA³ (24.42)。零样本设置，即不进行微调，显著落后，BLEU-4 得分为 8.22。这些结果表明，虽然微调提升了模型生成文本相似代码的能力，但与 Python 的 BLEU-4 分数（超过 40）相比，整体性能仍有提升空间，尤其是在 PEFT 技术方面。

在HumanEval基准测试中评估CodeLlama时，Pass@1分数与BLEU-4结果一致，LoRA达到最高分20.50。Pacter紧随其后，得分为18.01，而IA³的得分略低，为17.39。零样本设置下的Pass@1分数为11.18，表明即使未进行微调，CodeLlama也能生成一些可执行的R代码。然而，微调显著提升了其解决R代码任务的能力，尤其是使用LoRA时。与使用CodeLlama进行Python代码生成相比，R的Pass@1得分较低。

将焦点转向Llama2模型，在BLEU-4和Pass@1指标上均低于CodeLlama。LoRA仍然是表现最佳的方法，BLEU-4得分为21.44，其次是Compacter，得分为19.94，以及IA³，得分为19.60。零样本BLEU-4得分低于CodeLlama，达6.12，这表明与其对应的CodeLlama相比，Llama2在生成与参考解答文本一致的R代码方面能力较弱。

在HumanEval基准测试中，Llama2的表现低于CodeLlama。LoRA实现了最高的Pass@1分数，为10.56，其次是Compacter，为8.70，以及IA³，为7.45。零样本设置表现较差，Pass@1分数为1.24。这些结果表明，即使采用微调技术，Llama2在生成可执行的R代码方面仍然比CodeLlama更为困难。类似的结果也在Python中观察到，如表2所述。

在对Llama模型的微调技术进行比较时，LoRA始终在BLEU-4和Pass@1评分中表现最佳，表明其在提升文本生成质量和可执行代码方面是最有效的方法。Compacter和IA³在性能上紧随其后，Compacter在大多数情况下略优于IA³。然而，即使采用这些微调技术，CodeLlama和Llama2在生成可执行R代码方面的整体能力仍然有限。

总体而言，在R代码生成方面，T5系列模型的表现不及Llama模型。CodeLlama在所有模型中表现最佳。虽然微调提升了模型生成文本相似代码的能力

and solve some test cases, none of the methods yield consistently strong results, except for the CodeLlama model. LoRA provides the best balance between textual similarity and code execution, but the gap between the models’ BLEU-4 and HumanEval Pass@1 scores suggests that generating functional R code remains a challenge for the Llama family.

LoRA exhibits the best R code generation performance followed closely by Compacter and IA³ on the T5 and Llama family of models. Unlike Python code generation, the gap between the three is relatively small in the unseen code generation setting. All three methods perform significantly on the code-specific models, with T5 failing to pass any of the HumanEval samples and Llama2 performing half as strongly as CodeLlama when assessed on HumanEval.

5 Discussions

In this section, we discuss our findings. Due to time and computational resource restraints, and based on the obtained results, we only include code-LLMs, the code generation task and the Python and R languages (as *seen* and *unseen* languages) in the additional experiments. Moreover, we omit IA³ from all of the experiments conducted in the discussions to reduce time and computational resource requirements and avoid unreliable reports due to its inconsistent implementation, as explained in Section 3.

5.1 Computational Resource Efficiency

While the performance of each PEFT technique is important, the resource usage is of significance too. We evaluate the peak memory usage and runtime of Compacter and LoRA in a controlled scenario where the training steps and the batch size are equal for all models to ensure fairness. In this setting, all models are trained on the Python code generation task for 1000 steps with a train batch size of 8.

Table 4 illustrates the peak memory usage and runtime for training. For all models, Compacter utilizes a smaller GPU memory compared to LoRA. Specifically, LoRA employs 27.9% and 19.15% more peak GPU memory on the T5 and Llama family of models, respectively. Moreover, Compacter completes the training phase in a slightly shorter period on all Llama family of models and CodeT5, when compared to LoRA. Only on T5, LoRA demonstrates a faster training phase.

Besides resource requirements, performance is important in determining resource efficiency and guiding end-users in decision-making regarding performance-resource trade-offs. Given that Compacter’s lower resource requirements can be attributed to its smaller trainable parameter budget the efficiency of both PEFT methods can be assessed by a comparative analysis between their trainable parameter budget and relative performance.

并解决一些测试用例，除CodeLlama模型外，其他方法都未能持续获得优异的结果。LoRA在文本相似性和代码执行之间提供了最佳平衡，但模型的BLEU-4和HumanEval Pass@1分数之间的差距表明，生成功能性R代码仍然是Llama系列模型面临的挑战。

LoRA在R代码生成性能方面表现最佳，其次是Compacter和IA³，在T5和Llama系列模型中表现突出。与Python代码生成不同，在未见代码生成场景中，这三者之间的差距相对较小。

在代码特定模型上，三种方法均表现出显著的效果，其中T5未能通过任何HumanEval样本，而Llama2在HumanEval上的表现仅为CodeLlama的一半。

5 讨论

在本节中，我们讨论了我们的研究发现。由于时间和计算资源的限制，并基于所获得的结果，在附加实验中我们仅包括了代码-LLMs、代码生成任务以及Python和R语言（作为已见和未见语言）。此外，为了减少时间和计算资源的需求，并避免由于其实现不一致而导致的不可靠报告，我们在所有讨论中都省略了IA³，如第3节所述。

5.1 计算资源效率

虽然每种PEFT技术的性能表现很重要，但资源使用情况也具有重要意义。我们在一个受控场景中评估了Compacter和LoRA的峰值内存使用和运行时间，在该场景下所有模型的训练步骤和批次大小相等，以确保公平。在此设置中，所有模型在Python代码生成任务上训练1000步，训练批次大小为8。

表4展示了训练的峰值内存使用和运行时间。对于所有模型，Compacter的GPU内存占用均小于LoRA。具体而言，LoRA在T5和Llama系列模型上分别使用了比Compacter多27.9%和19.15%的峰值GPU内存。此外，在所有Llama系列模型和CodeT5上，Compacter完成训练阶段所用时间略短于LoRA。仅在T5模型上，LoRA表现出更快的训练速度。

除了资源需求之外，性能在确定资源效率和指导终端用户关于性能-资源权衡的决策中也具有重要意义。鉴于Compacter较低的资源需求可以归因于其较小的可训练参数预算，两个PEFT方法的效率可以通过其可训练参数预算与相对性能之间的比较分析进行评估。

PEFT	Total Param	Updated Param	Peak Memory (GB)	Runtime (Minutes)
CodeT5				
Compacter	222,903,552	0.051%	2.97	5.32
LoRA	222,903,552	0.595%	3.80	5.53
T5				
Compacter	222,903,552	0.051%	2.97	10.43
LoRA	222,903,552	0.595%	3.80	8.45
CodeLlama				
Compacter	6,738,415,616	0.012%	13.99	1.50
LoRA	6,738,415,616	0.093%	16.67	1.56
Llama2				
Compacter	6,738,415,616	0.012%	13.99	1.47
LoRA	6,738,415,616	0.093%	16.67	1.57

Table 4 The peak GPU memory and runtime of CodeT5, T5, CodeLlama and Llama2 fine-tuned with Compacter and LoRA, and their respective baselines during the training phase of the limited scenario. In this limited scenario, all models undergo 1000 training steps with a batch size of 8. The peak GPU memory usage and the runtime are reported in gigabytes and minutes, respectively.

Figures 2 and 3 depict Pass@1 (code generation) and BLEU-4 (code generation and code summarization) scores of LoRA and Compacter in Python and R for the Llama and the T5 family of models, respectively. For R code generation and Python code summarization on Llama2 and CodeLlama, Compacter offers a relative performance close to LoRA’s despite utilizing 7.75 times less trainable parameters. However, this smaller trainable parameter budget may affect Compacter’s capabilities for Python code generation and R code summarization, as LoRA pulls ahead on these tasks.

On the smaller T5 family of models, LoRA better illustrates its advantages. Besides Python code summarization where Compacter comes close to LoRA in terms of BLEU-4 score, LoRA outperforms Compacter dominantly for code generation in R and Python.

The larger resource requirements of LoRA become less effective on larger models. This is due to the fact that the trainable parameters introduced to the models constitute a larger portion of the total parameter count on smaller models (assuming similar hyperparameters for PEFT methods). For instance, while LoRA utilizes a 7.75 times larger parameter budget on the Llama family of models compared to Compacter, this number increases to 11.66 on the T5 family of models. The magnitude of this proportion directly affects the effectiveness of LoRA compared to Compacter.

Overall and across all models, Compacter undergoes training using less GPU memory and for most models completes the training phase faster when compared to LoRA. Even though its lower resource requirements come at the cost of worse performance (e.g., for code generation in R, Compacter achieved 18 Pass@1 compared to 20.5 Pass@1 by LoRA), Compacter can be a viable option depending on

PEFT	参数总数	更新的参数	峰值内存 (GB)	运行时 (分钟)
CodeT5				
更紧凑的模型	222,903,552	0.051%	2.97	5.32
LoRA	222,903,552	0.595%	3.80	5.53
T5				
更紧凑的模型	222,903,552	0.051%	2.97	10.43
LoRA	222,903,552	0.595%	3.80	8.45
CodeLlama				
更紧凑的模型	6,738,415,616	0.012%	13.99	1.50
LoRA	6,738,415,616	0.093%	16.67	1.56
Llama2				
更紧凑的模型	6,738,415,616	0.012%	13.99	1.47
LoRA	6,738,415,616	0.093%	16.67	1.57

表4在有限场景下，使用Compacter和LoRA微调的CodeT5、T5、CodeLlama和Llama2的峰值GPU内存和运行时间，以及它们各自的基线模型在训练阶段的表现。在此有限场景中，所有模型均进行1000个训练步骤，批次大小为8。峰值GPU内存使用量以GB为单位，运行时间以分钟为单位进行报告。

图2和图3分别展示了LoRA和Compacter在Python和R中针对Llama及T5系列模型的Pass@1（代码生成）和BLEU-4（代码生成与代码摘要）得分。对于Llama2和CodeLlama上的R代码生成以及Python代码摘要，Compacter的性能表现接近LoRA，尽管其可训练参数仅为LoRA的7.75倍。然而，这一较小的可训练参数预算可能会影响Compacter在Python代码生成和R代码摘要任务中的能力，因为LoRA在这些任务中表现优越。

在较小的T5模型系列中，LoRA更好地展现了其优势。除了Python代码总结中，Compacter在BLEU-4分数方面接近LoRA外，LoRA在R和Python的代码生成任务中显著优于Compacter。

LoRA的较大资源需求在更大模型上变得不那么有效。这是因为引入到模型中的可训练参数在较小模型中占总参数数目的比例更大（假设PEFT方法的超参数相似）。例如，虽然LoRA在Llama系列模型上使用的参数预算比Compacter多7.75倍，但在T5系列模型上这一数字增加到11.66。这一比例的大小直接影响LoRA相较于Compacter的效果。

总体而言，在所有模型中，Compacter在训练过程中使用的GPU内存更少，并且在大多数模型中相较于LoRA，完成训练阶段的时间更短。尽管其较低的资源需求以性能略逊为代价（例如，在R语言的代码生成任务中，Compacter的Pass@1为18，而LoRA为20.5），但根据具体应用场景，Compacter仍然可以是一种可行的选择。

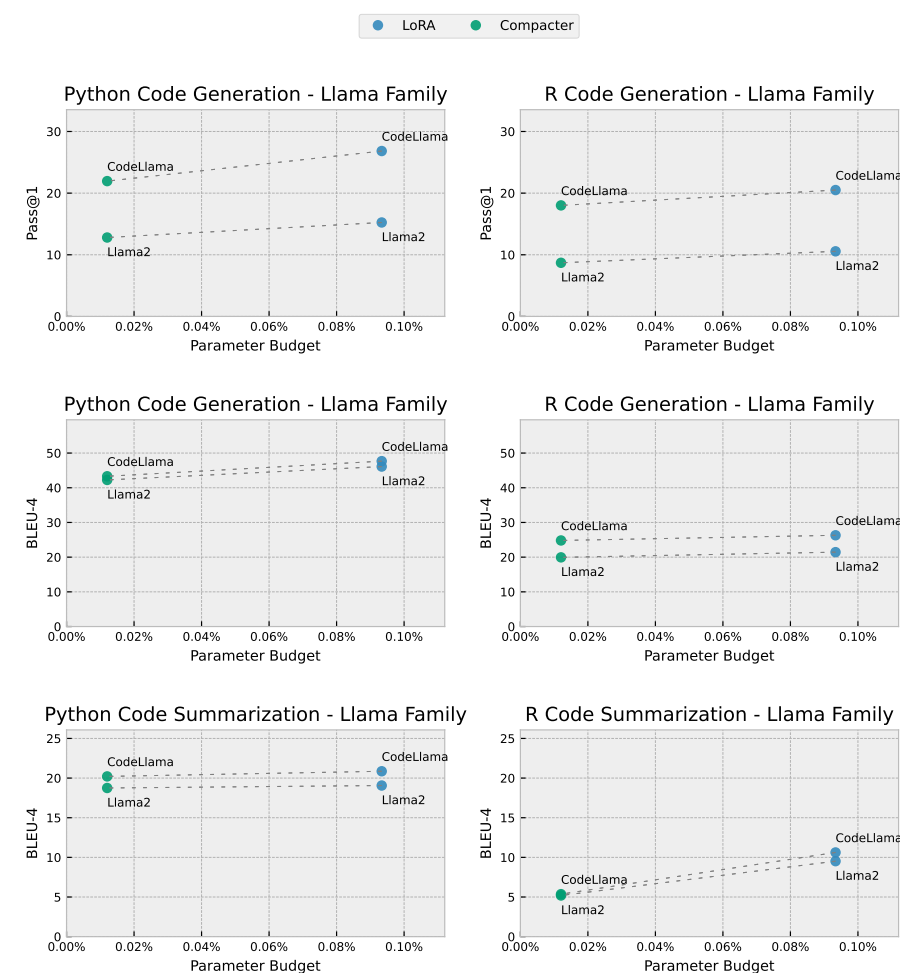


Fig. 2 Performance versus parameter budget of Compacter and LoRA for Python and R code generation and summarization tasks using the Llama family of models. BLEU-4 represents performance. The parameter budget is the percentage of trained parameters compared to the number of total parameters.

the base models and target tasks when available training resources become a limitation.

5.2 Effect of Trainable Parameter Budget

As shown previously, LoRA dominantly outperforms Compacter for Python and R. However, using the hyperparameters proposed by their authors, LoRA's increased performance comes at the cost of a larger trainable parameter budget and more computational resources. To compare their performance fairly, we modified the

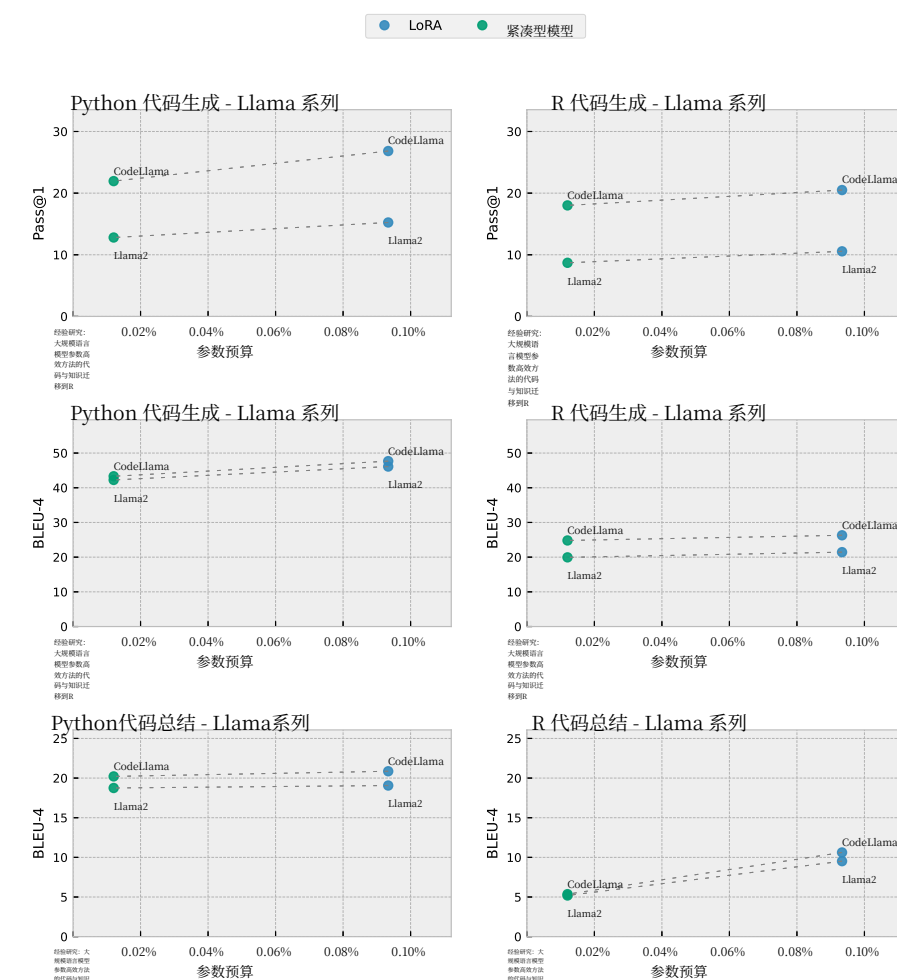


图 2 使用 Llama 系列模型进行 Python 和 R 代码生成与摘要任务中，Compacter 和 LoRA 的性能与可训练参数预算关系。BLEU-4 表示性能。参数预算为已训练参数占总参数数量的百分比。

在可用训练资源成为限制时，基础模型和目标任务。

5.2 可训练参数预算的影响

如前所述，LoRA在Python和R中的表现明显优于Compacter。然而，采用其作者提出的超参数时，LoRA的性能提升伴随着更大的可训练参数规模和更多的计算资源。为了公平比较它们的性能，我们对其进行了修改。

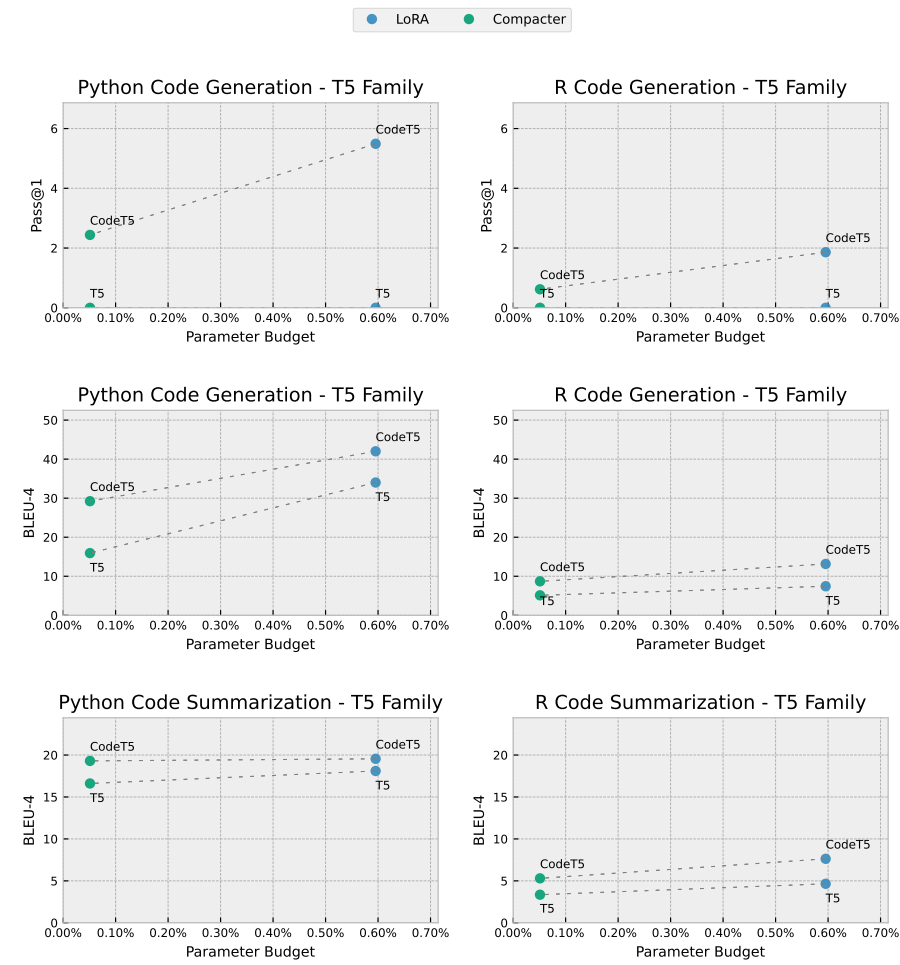


Fig. 3 Performance versus parameter budget of Compacter and LoRA for Python and R code generation and summarization tasks using the T5 family of models. BLEU-4 represents performance. The parameter budget is the percentage of trained parameters compared to the number of total parameters.

hyperparameters of both PEFT methods to reach the parameter budget of the other one. More specifically, in one scenario, we increased the parameter budget of Compacter to match the number of parameters used by LoRA, and in another, we decreased LoRA's parameters to become closer to the number of parameters of Compacter. In each case, we considered one of them as the baseline and the changed one as the modified one.

To achieve this, we used a higher *phm_rank* of 16 on Compacter (referred to as Compacter-large) to increase its trainable parameter budget to 1,254,592 and 8,921,152 on CodeT5 and CodeLlama, respectively, and a lower *r* of 1 on LoRA

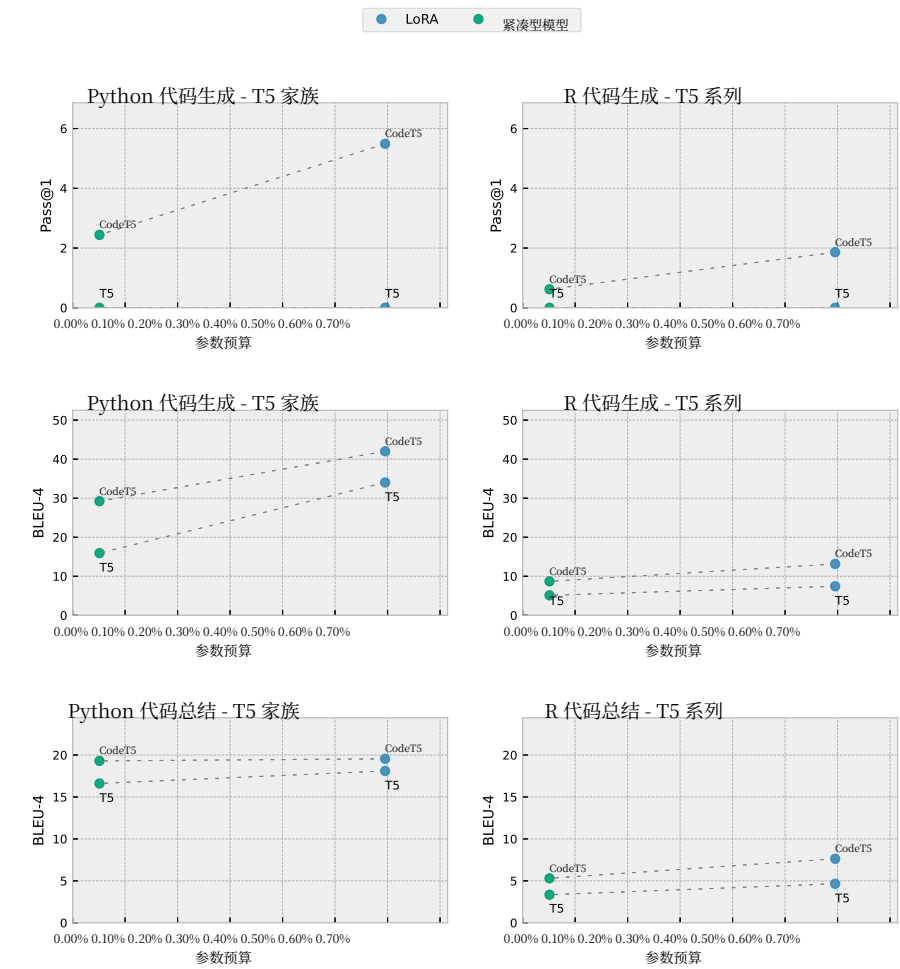


图 3 使用 T5 系列模型进行 Python 和 R 代码生成及摘要任务中，Compacter 和 LoRA 的性能与可训练参数预算关系。BLEU-4 表示性能。参数预算是指训练参数占总参数数量的百分比。

两种PEFT方法的超参数以达到另一种方法的参数预算。更具体地，在一种情境中，我们增加了Compacter的参数预算，以匹配LoRA所使用的参数数量；在另一种情境中，我们减少了LoRA的参数，使其更接近Compacter的参数数量。在每种情况下，我们都将其中一种作为基线，另一种作为修改对象。

为此，我们在Compacter（简称Compacter-large）上使用了更高的 phm_rank 16，以将其可训练参数预算分别提高到1,254,592和8,921,152，在CodeT5和CodeLlama上，而在LoRA上则采用较低的 r 值1。

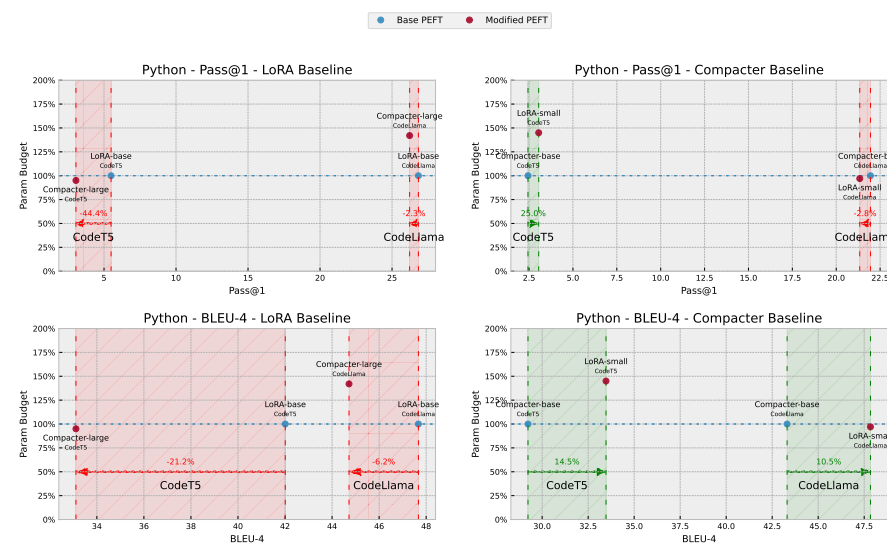


Fig. 4 Performance difference of LoRA and Compacter on CodeT5 and CodeLlama for Python code generation, with various parameter budgets. Performance is represented by Pass@1 (top) and BLEU-4 (bottom), with a left-pointing arrow and a red area indicating a decrease and a right-pointing arrow and a green area indicating an increase in performance compared to the baseline PEFT. The parameter budget on the y-axis is the ratio of trainable parameters compared to the number of the baseline's trainable parameters.

(referred to as LoRA-small) to decrease its trainable parameter budget to 165,888 and 786,432 on CodeT5 and CodeLlama, respectively.

We note that even though these parameter budget sizes are not exactly the same as the baseline parameter budget sizes (i.e., LoRA and Compacter default configurations in previous studies), given that we control discrete hyperparameters, they are within an acceptable range of the baselines relative to the much larger parameter budget of their base models. Due to computational restrictions, we only conduct this experiment for the code generation task. However, as we use the BLEU metric in addition to Pass@1, the textual similarity comparisons are considered here.

The general pattern we observe is that PEFT methods with similar parameter budgets obtain comparable performance in terms of functional accuracy (i.e., Pass@1) for both Python (top plots on Fig. 4) and R (top plots on Fig. 5). Compacter-large on CodeT5 receives the highest performance drop compared to the baseline LoRA for Python and R, followed by LoRA-small on CodeLlama for R. The Pass@1 score changes are minimal in other settings.

In terms of textual similarity (i.e., BLEU-4), however, we observe that LoRA is more robust in different parameter budgets (bottom plots on Fig. 4 for Python and Fig. 5 for R). When scaled down, LoRA-small consistently outperforms the baseline Compacter on both models and both languages, while the baseline LoRA outperforms the up-scaled Compacter-large in all settings.

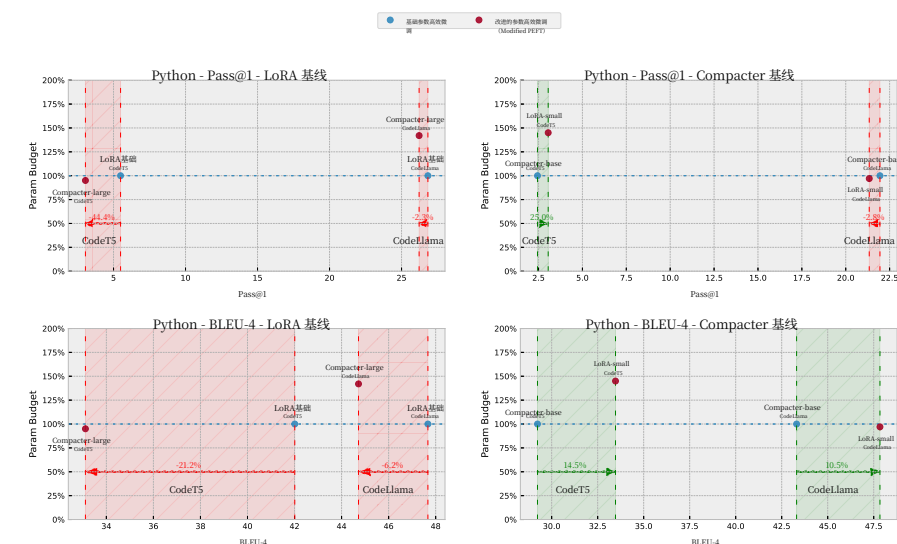


图4 LoRA与Compacter在CodeT5和CodeLlama上针对Python代码生成的性能差异，涵盖不同的参数预算。性能以Pass@1（上）和BLEU-4（下）表示，左箭头和红色区域表示性能相较于基线PEFT的下降，右箭头和绿色区域表示性能提升。y轴上的参数预算为可训练参数比例，相对于基线模型的可训练参数数量。

（以下简称LoRA-small）将其可训练参数预算分别降低至CodeT5的165,888和CodeLlama的786,432。

我们注意到，尽管这些参数预算规模与基线参数预算规模（即前述研究中的LoRA和Compacter默认配置）不完全相同，但由于我们控制了离散超参数，它们相对于基础模型更大参数预算的范围内，仍在可接受范围内。由于计算限制，我们仅对代码生成任务进行了此项实验。然而，鉴于我们除了Pass@1之外还使用了BLEU指标，因此这里考虑了文本相似性比较。

我们观察到的总体模式是，具有相似参数预算的PEFT方法在功能准确性（即Pass@1）方面表现出相当的性能，无论是Python（图4顶部图）还是R（图5顶部图）。在CodeT5上，Compacter-large相较于基线LoRA在Python和R中的性能下降最大，其次是CodeLlama上的LoRA-small在R中的表现。在其他设置中，Pass@1分数的变化较小。

在文本相似性（即BLEU-4）方面，我们观察到LoRA在不同参数预算下表现更为稳健（图4的底部图为Python，图5为R）。当缩小规模时，LoRA-small在两种模型和两种语言中始终优于基线的Compacter，而基线的LoRA在所有设置中均优于放大版的Compacter-large。

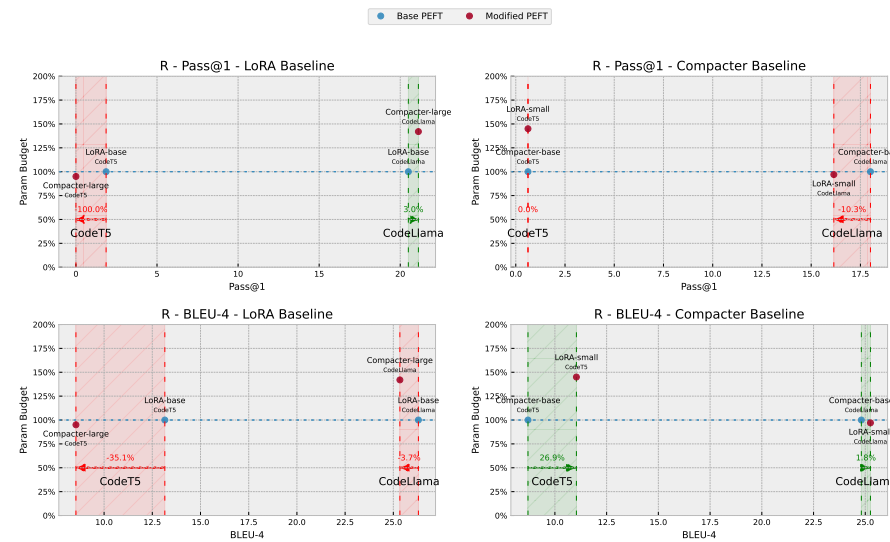


Fig. 5 Performance difference of LoRA and Compacter on CodeT5 and CodeLlama for R code generation, with various parameter budgets. Performance is represented by Pass@1 (top) and BLEU-4 (bottom), with a left-pointing arrow and a red area indicating a decrease and a right-pointing arrow and a green area indicating an increase in performance compared to the baseline PEFT. The parameter budget on the y-axis is the ratio of trainable parameters compared to the number of the baseline’s trainable parameters.

Overall, we notice that for functional accuracy, i.e., Pass@1, the number of trainable parameters outweighs the effect of the PEFT’s architecture. The architecture becomes more important for textual similarity, i.e., BLEU-4, where LoRA outperforms Compacter in similar parameter budgets.

5.3 Robustness

For code generation, we also assess the robustness of PEFT methods in code generation tasks. Recent work by Zhu et al. [77] highlights that LLMs are prone to errors when prompted with inputs containing plausible user errors such as typos or synonyms. This emphasizes the need to understand how PEFT strategies can affect such challenges in code generation tasks. To achieve this, we randomly select 100 samples from Python and R, representing both seen and unseen languages in our experiments. The description docstrings within these samples are manipulated to mimic textual errors, i.e., user typos and improper synonyms, using GPT-4o⁵. The prompt used to generate the modified samples and example original and modified prompts and their generations by each PEFT are present in Appendix A and B, respectively. We repeated the experiment three times and report the average scores in Table 5. By comparing the results obtained from these manipulated samples with their original format, we evaluate the robustness of Compacter and LoRA

⁵ <https://platform.openai.com/docs/models#gpt-4o>

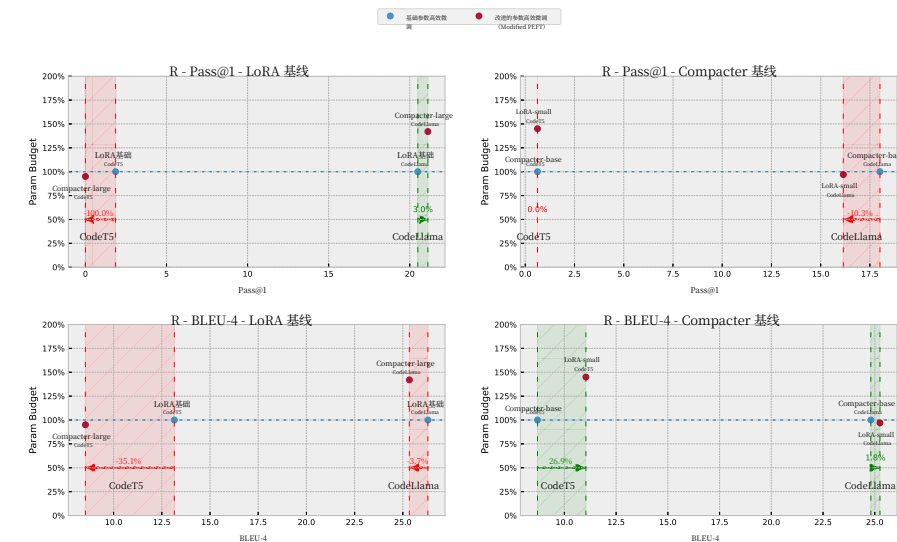


图5 LoRA与Compacter在CodeT5和CodeLlama上进行R代码生成的性能差异，涵盖不同参数预算。性能指标包括Pass@1（上）和BLEU-4（下），左箭头和红色区域表示性能相较于基线PEFT的下降，右箭头和绿色区域表示性能提升。y轴上的参数预算为可训练参数的比例，相对于基线模型的可训练参数数量。

总体而言，wenote到对于功能准确性，即Pass@1，可训练参数的数量超过了PEFT架构的影响。在文本相似性方面，即BLEU-4，架构变得更加重要，其中LoRA在相似的参数预算下优于Compacter。

5.3 鲁棒性

对于代码生成，我们还评估了PEFT方法在代码生成任务中的鲁棒性。Zhu等人[77]的最新研究指出，LLMs在面对包含合理用户错误（如拼写错误或同义词）的输入时容易出现错误。这强调整理解PEFT策略如何影响代码生成任务中此类挑战的必要性。为此，我们从Python和R中随机抽取了100个样本，涵盖实验中已见和未见的语言。这些样本中的描述性文档字符串被操控以模拟文本错误，即用户拼写错误和不当同义词，使用GPT-4o⁵实现。用于生成修改样本的提示以及每个PEFT的原始提示和修改后提示及其生成结果，分别列于附录A和B。我们重复了三次实验，并在表5中报告了平均得分。通过比较这些操控样本与其原始格式的结果，我们评估了Compacter和LoRA的鲁棒性。

<https://platform.openai.com/docs/models#gpt-4o>

PEFT	Python			R		
	Original	Modified	Change	Original	Modified	Change
<i>CodeT5</i>						
Compacter	11.99	11.21	-6.51% ↓	5.32	5.78	8.67% ↑
LoRA	41.36	37.84	-8.52% ↓	13.13	11.95	-8.94% ↓
<i>T5</i>						
Compacter	3.02	2.71	-10.47% ↓	4.22	4.08	-3.30% ↓
LoRA	33.45	31.74	-5.11% ↓	8.36	7.52	-9.97% ↓
<i>CodeLlama</i>						
Compacter	41.05	41.54	1.21% ↑	20.62	20.37	-1.19% ↓
LoRA	48.68	47.48	-2.47% ↓	24.79	22.65	-8.62% ↓
<i>Llama2</i>						
Compacter	40.13	37.98	-5.36% ↓	17.21	15.55	-9.67% ↓
LoRA	46.00	45.30	-1.53% ↓	20.93	20.01	-4.41% ↓

Table 5 The BLEU-4 score of all models fine-tuned using Compacter and LoRA for Python and R code generation tasks. The original column reports the BLEU-4 score across the unmodified 100 samples. The modified column reports the BLEU-4 score across the 100 samples modified to represent user typos and errors. The change column reports the change in the BLEU-4 score after modification.

when applied to the T5 and Llama family of models. In this experiment, we compare performance using the BLEU-4 score.

As illustrated in Table 5, LoRA suffers from textual errors on all models for both languages. The performance loss for R is larger compared to Python, suggesting that the unseenness of R can impact LoRA’s robustness. Notably, with LoRA, T5 and CodeT5 have the most performance decrease resulting from textual errors in R samples and Python samples, respectively. The Llama models have less performance decrease compared to the T5 models when using LoRA.

Compared to LoRA, Compacter tolerates textual errors better for both R and Python, in some cases even slightly surpassing the BLEU-4 score obtained by the original samples. In Python, textual errors do not impact the performance of any model fine-tuned with Compacter significantly. However, introducing textual errors to R samples results in a pattern where Compacter on code-LLMs, i.e., CodeT5 and CodeLlama, demonstrate strong robustness against these errors. On general-LLMs however, these textual errors can result in a notable performance drop. A similar pattern is observed for Compacter for Python, resulting in -1.4% performance drop or $+3.3\%$ performance increase when applied on CodeT5 and CodeLlama, respectively.

In summary, Compacter shows robustness in code generation when facing textual errors for all models, compared to LoRA. This observation might be due to having fewer parameters in Compacter, thus it can be less overfit to the data compared to LoRA. However, finding the exact reason requires a separate experiment, which is out of the scope of this study.

5.4 Qualitative Study on Code Summarization Quality

To further analyze the results obtained from the code summarization tasks by LoRA and Compacter, we conducted a qualitative analysis of the quality of the generated

PEFT	Python			R		
	经验研究：大规模数据 占模型参数比例的方法	参数高效方法在大模型 占模型参数的比例研究	变更	请提供需要 修改的地方	修改	变更
<i>CodeT5</i>						
紧凑型模型	11.99	11.21	-6.51% ↓	5.32	5.78	8.67% ↑
LoRA	41.36	37.84	-8.52% ↓	13.13	11.95	-8.94% ↓
<i>T5</i>						
紧凑型模型	3.02	2.71	-10.47% ↓	4.22	4.08	-3.30% ↓
LoRA	33.45	31.74	-5.11% ↓	8.36	7.52	-9.97% ↓
<i>CodeLlama</i>						
紧凑型模型	41.05	41.54	1.21% ↑	20.62	20.37	-1.19% ↓
LoRA	48.68	47.48	-2.47% ↓	24.79	22.65	-8.62% ↓
<i>Llama2</i>						
紧凑型模型	40.13	37.98	-5.36% ↓	17.21	15.55	-9.67% ↓
LoRA	46.00	45.30	-1.53% ↓	20.93	20.01	-4.41% ↓

表 5 使用 Compacter 和 LoRA 微调的所有模型在 Python 和 R 代码生成任务中的 BLEU-4 分数。原始列报告未修改的 100 个样本的 BLEU-4 分数。修改列报告经过修改以反映用户拼写错误和错误的 100 个样本的 BLEU-4 分数。变化列报告修改后 BLEU-4 分数的变化。

当应用于T5和Llama系列模型时。在本次实验中，我们使用BLEU-4分数进行性能比较。

如表5所示，LoRA在所有模型中都存在文本错误，适用于两种语言。对于R语言，性能损失比Python更大，表明R语言的非一致性可能影响LoRA的鲁棒性。值得注意的是，采用LoRA时，T5和CodeT5在R样本和Python样本中因文本错误导致的性能下降最大。与T5模型相比，Llama模型在使用LoRA时的性能下降较少。

与LoRA相比，Compacter在R和Python中对文本错误的容忍度更高，在某些情况下甚至略微超过了原始样本所获得的BLEU-4分数。在Python中，文本错误对任何使用Compacter微调的模型性能影响不大。然而，向R样本引入文本错误会导致一种模式，即在代码-LLMs（如CodeT5和CodeLlama）上，Compacter表现出对这些错误的强鲁棒性。而在通用LLMs中，这些文本错误可能导致性能显著下降。在Python中，Compacter也表现出类似的模式，应用于CodeT5和CodeLlama时，分别导致 -1 的 4% 性能下降或 $+3$ 的 3% 性能提升。

总之，与LoRA相比，Compacter在面对所有模型的文本错误时，在代码生成方面表现出更强的鲁棒性。这一观察可能是由于Compacter参数较少，因此相较于LoRA，它对数据的过拟合程度较低。然而，确切原因的探究需要进行单独的实验，这超出了本研究的范围。

5.4 代码总结质量的定性研究

为了进一步分析通过LoRA和Compacter在代码总结任务中获得的结果，我们进行了对生成内容质量的定性分析

PEFT	Python				R			
	I	R	F	Average	I	R	F	Average
<i>CodeT5</i>								
LoRA	3.89	4.17	4.65	4.24	2.89	3.2	4.15	3.41
Compacter	3.93	4.19	4.62	4.25	3.12	3.3	4.1	3.51
<i>CodeLlama</i>								
LoRA	4.18	4.36	4.63	4.39	3.55	3.54	4.06	3.72
Compacter	4.16	4.43	4.69	4.43	3.39	3.44	4.04	3.62
Ground Truth	4.68	4.43	4.35	4.49	4.14	3.69	4.01	3.95

Table 6 The average scores given by the evaluators for Informativeness (I), Relevance (R) and Fluency (F) for LoRA and Compacter on CodeT5 and CodeLlama for Python and R code summarization. The last row represents the same scores for the ground truth samples of each language. All scores are out of five, with **bold** indicating the highest value in each row per language.

summaries. For this task, we evaluated samples following the approach of previous studies [55, 57], where we randomly selected 100 samples from the test dataset for Python and R (other programming languages are eliminated due to resource restrictions), which contain the code, the ground truth and the generated summaries by the code-LLMs. Each sample is then ranked by two software engineers who are proficient in both programming languages according to three criteria based on a Likert scale of one to five for each criterion, where five is the highest. The criteria are as follows:

- Informativeness (I), scores the comments according to covering the key aspects of code.
- Relevance (R), evaluates the consistency of the details in the comments compared to code.
- Fluency (F), scores the fluency of the comments in English, considering if the sentences are well written and are grammatically correct.

Table 6 represents the obtained scores across the three metrics for LoRA and Compacter on CodeLlama and CodeT5. The average score of both PEFT methods and both models are close to the ground truth, being slightly higher for CodeLlama. Among the three metrics, Informativeness is lower than other scores (R and F) for both Compacter and LoRA compared to the ground truth. Among the two models, CodeLlama has higher scores for all three metrics. Comparing Compacter and LoRA, we observe mixed results. On CodeT5, Compacter has higher scores in I and R, for both Python and R. However, on CodeLlama, LoRA has better scores in all three aspects for R but slightly lags behind Compacter in R and F for Python. Overall, both PEFT approaches have scores close to each other, especially for CodeLlama.

For both PEFT methods on both base models, we observe a higher Fluency score compared to their respective ground truths, which may be due to the fact that these models have been pre-trained on large corpora of highly fluent data (in terms of grammar, punctuation and structure). Beyond the small differences in scores, both PEFT methods are capable of reaching the performance of the relatively large CodeLlama on the much smaller CodeT5 model, suggesting adequate knowledge transfer on small code-LLMs for both Python and R. For the R dataset,

PEFT	Python				R			
	I	R	F	平均	I	R	F	平均
<i>CodeT5</i>								
LoRA	3.89	4.17	4.65	4.24	2.89	3.2	4.15	3.41
Compacter	3.93	4.19	4.62	4.25	3.12	3.3	4.1	3.51
<i>CodeLlama</i>								
LoRA	4.18	4.36	4.63	4.39	3.55	3.54	4.06	3.72
Compacter	4.16	4.43	4.69	4.43	3.39	3.44	4.04	3.62
真实情况	4.68	4.43	4.35	4.49	4.14	3.69	4.01	3.95

表6评估者对LoRA和Compacter在CodeT5和CodeLlama上针对Python和R代码总结的信息量（I）、相关性（R）和流畅性（F）所给出的平均评分。最后一行表示每种语言的真实样本的相应评分。所有评分满分为五分，**加粗**表示每个语言行中的最高值。

总结。对于本任务，我们采用了前人研究的方法 [55, 57], , 随机从测试数据集中抽取了100个样本（Python和R, 其他编程语言由于资源限制被排除），这些样本包含代码、真实标签和由代码-LLMs生成的摘要。每个样本由两位精通这两种编程语言的软件工程师根据三个标准进行排名，评分采用一到五的Likert量表，其中五为最高。标准如下：

信息量（I），根据覆盖代码关键方面对评论进行评分。

相关性（R），评估评论中的细节与代码的一致性。

流畅性（F），评估英文评论的流畅程度，考虑句子是否表达流畅且语法正确。

表6展示了在CodeLlama和CodeT5上，LoRA与Compacter在三项指标上的得分情况。两种PEFT方法及两种模型的平均得分均接近真实值，CodeLlama的得分略高于其他。三项指标中，信息性（Informativeness）在Compacter和LoRA中均低于真实值（R和F）。在两种模型中，CodeLlama在所有三项指标上的得分均较高。比较Compacter与LoRA，结果呈现混合。在CodeT5上，Compacter在I和R指标上得分更高，适用于Python和R两种语言。然而，在CodeLlama上，LoRA在R的三项指标中表现更佳，但在Python的R和F指标上略逊于Compacter。总体而言，两种PEFT方法的得分彼此接近，尤其是在CodeLlama上。

lama。

对于两种基础模型上的PEFT方法，我们观察到其流畅性得分均高于各自的真实值，这可能是由于这些模型在大量高流畅性数据（在语法、标点和结构方面）的大型语料库上进行了预训练。除了得分的微小差异外，两个PEFT方法都能够在远小于CodeLlama的CodeT5模型上达到相对较大的CodeLlama的性能，表明在Python和R的小型代码语言模型中实现了充分的知识迁移。对于R数据集，

Category	LoRA				Compacter			
	Structure	Naming	Readability	Total	Structure	Naming	Readability	Total
Python								
Passed	4.98	4.95	4.68	14.61	4.93	4.98	4.83	14.74
Failed	5.00	4.95	4.35	14.30	4.68	4.73	4.85	14.26
Average	4.99	4.95	4.51	14.46	4.80	4.86	4.84	14.50
R								
Passed	4.95	4.95	4.50	14.40	4.98	4.93	4.83	14.74
Failed	4.93	4.98	4.60	14.51	4.68	4.98	4.68	14.34
Average	4.94	4.96	4.55	14.45	4.83	4.96	4.75	14.54
Average								
Passed & Failed	4.96	4.96	4.53	14.45	4.82	4.91	4.80	14.52

Table 7 The average structure, naming and readability (out of five) and the total scores (out of 15) of LoRA and Compacter on CodeLlama for *Passed* (i.e., samples that pass unit tests) and *Failed* (i.e., samples that fail unit tests but are syntax error-free) for Python and R styling quality survey, reported by the participants. **Bold** in the total column indicates the highest score per row. The last row represents the average scores across *Passed* and *Failed* Python and R samples (i.e., average scores across both languages and categories).

it is notable that the ground truth evaluations are around 4, out of 5, for all three aspects of informativeness, relevance, and fluency. This indicates that the quality of the datasets needs to be improved for R.

5.5 Qualitative Study on Styling Rules

While the HumanEval benchmark has unit tests assessing the correctness of the generated code, it is incapable of assessing the styling rules of generated code. Following previous work [9], we will create one styling rubric for R and one for Python (other programming languages are omitted due to resource limitations) to access the capabilities of LoRA and Compacter in adhering to the best practices established by the software engineering communities. These rubrics contain three categories of rules based on Google’s styling guidelines for R, and PEP 8 styling guidelines for Python.

- code structure and cleanness pertaining to the proper use of indentation, new lines, commenting and function definition,
- naming convention pertaining to appropriate naming of code constructs such as variables, functions, and
- code readability and best practices pertaining to the adoption of defined best practices such as usage of white spaces around operators.

Every category of rules will have an impact score on a scale of one to five according to the aforementioned guidelines. During the evaluation process, each code sample will receive a negative penalty for violating any of the rules in its corresponding rubric, based on the rule’s impact score. Finally, the styling rating of each sample will be computed based on the accumulated penalties assigned to it. Rubrics we employed in this study are available in Appendix C.

类别	LoRA				Compacter			
	结构	命名	可读性		结构	命名	可读性	参数高效方法在大型语言模型中的实证研究：代码与知识迁移到R
Python								
已通过	4.98	4.95	4.68	14.61	4.93	4.98	4.83	14.74
未能完成请求。	5.00	4.95	4.35	14.30	4.68	4.73	4.85	14.26
平均	4.99	4.95	4.51	14.46	4.80	4.86	4.84	14.50
R								
通过	4.95	4.95	4.50	14.40	4.98	4.93	4.83	14.74
未能完成请求。	4.93	4.98	4.60	14.51	4.68	4.98	4.68	14.34
平均	4.94	4.96	4.55	14.45	4.83	4.96	4.75	14.54
平均值								
通过 & 未能完成	4.96	4.96	4.53	14.45	4.82	4.91	4.80	14.52

表7 LoRA 和 Compacter 在 CodeLlama 上关于通过（即通过单元测试的样本）和未通过（即未通过单元测试但无语法错误的样本）在 Python 和 R 样式质量调查中的平均结构、命名和可读性（满分五分）以及总分（满分十五分），由参与者报告。**加粗**的总分列表示每行的最高分。最后一行代表通过和未通过 Python 和 R 样本的平均分（即两个语言和类别的平均分）。

值得注意的是，所有三个方面——信息量、相关性和流畅性——的真实评估分数都在4分左右（满分5分）。这表明用于 R 的数据集质量有待提升。

5.5 样式规则的定性研究

虽然 HumanEval 基准测试具有评估生成代码正确性的单元测试，但它无法评估生成代码的风格规则。沿袭先前的研究 [9], , 我们将为 R 和 Python 创建各自的风格评分标准（由于资源限制，其他编程语言被省略），以评估 LoRA 和 Compacter 在遵循软件工程社区制定的最佳实践方面的能力。这些评分标准包含基于 Google 对 R 的风格指南和 PEP 8 对 Python 的风格指南的三类规则。

- 代码结构与整洁性，涉及缩进、换行、注释和函数定义的规范使用
- 与变量、函数等代码结构的恰当命名相关的命名规范
- 代码可读性及采用已定义最佳实践的最佳实践，例如在运算符周围使用空格。

每个规则类别将根据上述指南在一到五的范围内赋予一个影响分数。在评估过程中，每个代码示例在违反其对应评分标准中的任何规则时，将根据该规则的影响分数受到负面惩罚。最终，每个示例的样式评分将基于累积的惩罚进行计算。我们在本研究中采用的评分标准详见附录C。

This evaluation is conducted manually by two software engineers proficient in R and Python on 160 randomly selected samples. The samples are chosen from HumanEval problems (i.e., unseen samples) to eliminate models' styling bias towards their training dataset. Upon initial processing, we observed that only CodeLlama is capable of generating code samples adequate for styling analysis, among which many do not pass as valid Python or R code snippets. To this end, we only evaluate samples generated by CodeLlama that have either passed their tests successfully or have failed them only due to logical errors (i.e., samples are syntactically correct). We separately evaluate the passed samples, named “*Passed*”, and the failed samples, named “*Failed*”, to maintain fairness in evaluation, as the performance of the model in terms of functional accuracy may affect the styling quality of the generated code snippets. The inter-rater agreement among the two evaluators using Cohen-Kappa score [41] is 0.71, which shows a high agreement among the two raters.

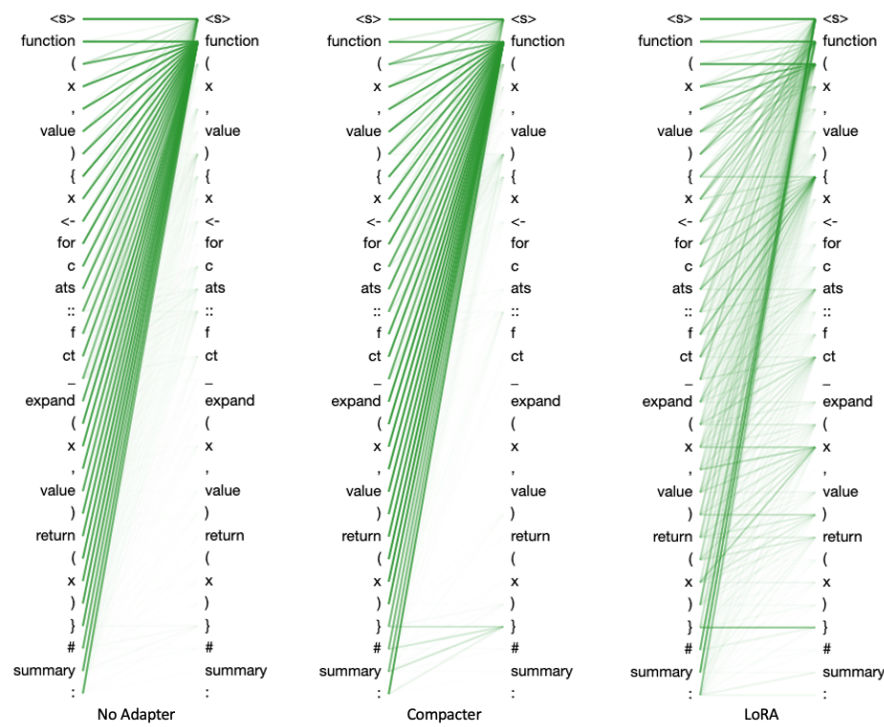


Fig. 6 Attention map of CodeLlama to the input sequence on an example R function, with zero-shot setting (“No Adapter” in the plot), compared to the fine-tuned version of CodeLlama with Compacter and LoRA.

As shown in Table 7, while both PEFT methods follow the targeted styling rules very closely, overall Compacter tends to produce higher quality code in terms of styling rules for both the seen language (i.e., Python) and the unseen language (i.e.,

本次评估由两名精通R和Python的软件工程师对160个随机抽取的样本进行手动评估。样本选自HumanEval问题（即未见样本），以消除模型对训练数据集的风格偏差。在初步处理过程中，我们观察到，只有CodeLlama能够生成适合风格分析的代码样本，其中许多不符合有效的Python或R代码片段。为此，我们仅评估由CodeLlama生成的样本，这些样本要么成功通过测试，要么仅因逻辑错误而未通过（即样本在语法上是正确的）。我们将成功通过测试的样本单独评估，命名为“*Passed*”，将未通过测试的样本命名为“*Failed*”，以保持评估的公平性，因为模型在功能准确性方面的表现可能影响生成代码片段的风格质量。两名评估者之间的Cohen-Kappa系数 [41] 为0.71，显示出较高的一致性。

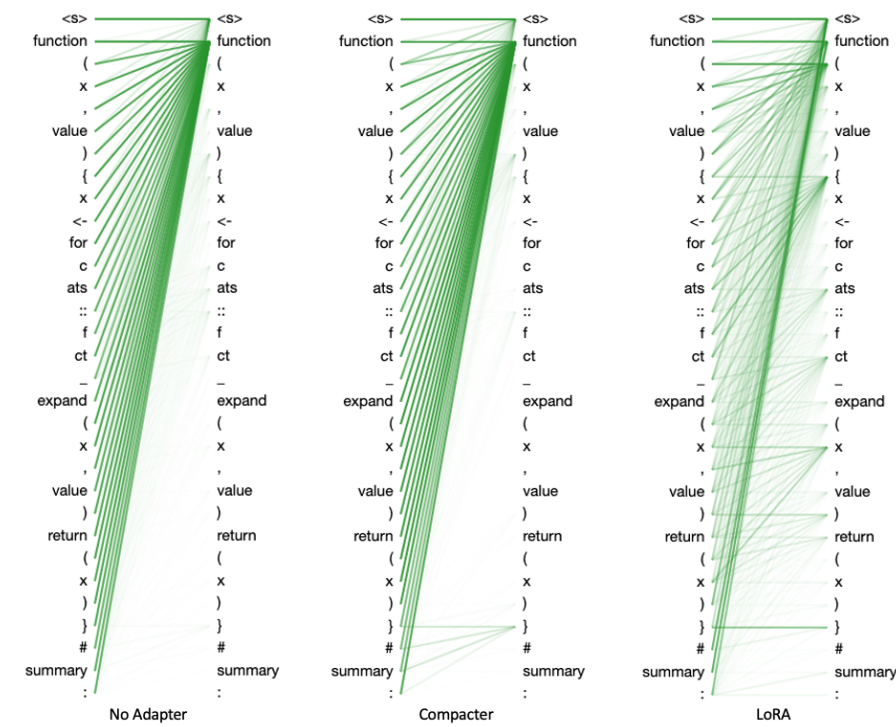


图6 CodeLlama 对示例 R 函数输入序列的注意力图，在零样本设置（图中为“*No Adapter*”），与使用 Compacter 和 LoRA 微调版本的 CodeLlama 进行对比。

如表7所示，尽管两种PEFT方法都非常贴合目标风格规则，但总体而言，Compacter在代码风格规则方面倾向于生成更高质量的代码，无论是针对已见语言（即Python）还是未见语言（即

R)–see last row of Table 7. Notably, however, LoRA exhibits better adherence to the styling rules when producing code that fails their unit tests (i.e., *Failed* samples). The overall high styling quality of generations produced by both PEFT methods may be due to the fact that the code samples in their training data already follow these rules, enabling both PEFT methods to generate code with the learned style.

5.6 Attention Changes

Following previous works [54], we highlight the attention change on an R code generation sample in the last transformer block of CodeLlama in the zero-shot setting and after fine-tuning with LoRA and Compacter in Fig. 6. We utilize Bertviz⁶ to visualize the attention changes.

The attention map of the zero-shot CodeLlama (left plot of Fig. 6) suggests that the model mainly attends to the `<S>` and *function* tokens, without any significant attention to other tokens that construct the R code snippet. In contrast, both Compacter (middle plot of Fig. 6) and LoRA (right plot of Fig. 6) better attend to other tokens. Furthermore, we observe a large difference between the attention map of LoRA and Compacter, with the former attending to a more diverse set of important code tokens.

This difference in attention is aligned with the observed performance of these PEFT methods. However, the influence of attention map diversity and the performance exhibit a weak association; even though Compacter’s attention map is closer to the zero-shot model, its performance is more comparable with LoRA’s.

6 Related Work

In this section, we review the works that study PEFT methods in Natural Language Processing and Software Engineering research.

6.1 PEFT Studies in Natural Language Processing

He et al. [22] conducted a comparative analysis of adapter-based fine-tuning and full fine-tuning across a diverse range of NLP tasks and datasets. Their findings highlighted several key observations. They found out that adapter-based tuning showcased superior performance in zero-shot cross-lingual tasks, and it exhibited higher stability and better generalization capabilities. Additionally, in scenarios requiring monolingual adaptation, adapter-based tuning demonstrated better outcomes in low-resource settings, particularly in more domain-specific tasks. Ding et al. [17] carried out extensive experiments involving numerous language models and various PEFT methods, including Adapters [24], LoRA [25], and Prefix-Tuning [31], as well as their combination on a single model. Their study encompassed over 100 NLP tasks, examining these methods’ performance, knowledge

⁶ <https://github.com/jessevig/bertviz>

R)–见表7的最后一行。然而，值得注意的是，LoRA在生成未通过单元测试的代码（即*Failed* 样本）时，表现出更好的风格规则遵循性。两种PEFT方法生成的代码整体风格质量较高，可能是因为它们的训练数据中的代码样本已经遵循了这些规则，从而使两种PEFT方法都能生成具有学习风格的代码。

5.6 注意力变化

根据前述研究 [54]，我们在图6中突出显示了在零样本设置下以及经过LoRA和Compacter微调后，CodeLlama最后一个Transformer块中R代码生成样本的注意力变化。我们采用Bertviz⁶对注意力变化进行可视化。

零样本CodeLlama的注意力图（图6左图）表明，该模型主要关注 `<S>`和 *function* 标记，而对构建R代码片段的其他标记没有显著关注。相比之下，Compacter（图6中间图）和LoRA（图6右图）对其他标记的关注更为充分。此外，我们观察到LoRA的注意力图与Compacter存在较大差异，前者对更为多样化的重要代码标记表现出关注。

这种注意力差异与这些PEFT方法的观察性能一致。然而，注意力图多样性与性能之间的关联较弱；即使Compacter的注意力图更接近零样本模型，其性能仍与LoRA更为接近。

6 相关工作

在本节中，我们回顾了了在自然语言处理和软件工程研究中研究PEFT方法的相关工作。

6.1 大规模语言模型参数高效微调方法在自然语言处理中的研究 g

He 等人 [22] 对适配器微调与全量微调在多样化的自然语言处理任务和数据集上的表现进行了比较分析。研究结果突出了几个关键观察点。他们发现，基于适配器的微调在零样本跨语言任务中表现出更优的性能，并且具有更高的稳定性和更好的泛化能力。此外，在需要单语适应的场景中，基于适配器的微调在低资源环境下表现出更佳的效果，尤其是在更具领域特定性的任务中。Ding 等人 [17] 进行了大量实验，涉及多个语言模型和多种参数高效微调（PEFT）方法，包括 Adapters [24]、LoRA [25]，和 Prefix-Tuning [31]，及其在单一模型上的组合。这项研究涵盖了超过 100 个自然语言处理任务，评估了这些方法的性能和知识

<https://github.com/jessevig/bertviz>

transferability, and scalability. Lialin et al. [34] provided a detailed analysis of diverse PEFT methods introduced in recent years, categorizing them into three major classes: Addition-based, Selective-based, and Reparametrization-based. They conducted comparative assessments of these methods concerning storage efficiency, memory efficiency, computational efficiency, accuracy, and inference overhead.

6.2 PEFT Studies in Software Engineering

Goel et al. [20] was among the first to study adapters, a specific technique of PEFT, in software engineering. They used bottleneck adapters and assessed their capabilities for knowledge transfer from natural language models to code tasks. They extended the work in [54] to other software engineering tasks and code-LLM and showed that by using adapters, the model performance increases or is on par with fully fine-tuning results for the studied languages and tasks. Liu et al. [36] conducted an empirical study of PEFT methods for code summarization among other tasks using several encoder-only and encoder-decoder code-LLMs, including CodeBERT [18] and CodeT5 [68]. They investigated the effect of the number of trainable parameters on LoRA, adapter and two other PEFT methods, and their performance on low-resource, cross-lingual, and cross-projects.

Weyssow et al. [71] investigated LoRA, IA³ and Prompt Tuning [28] when applied to code-LLMs for code generation. Their primary emphasis lies in evaluating the benefits of PEFT methods when employed in large language models versus small language models. Additionally, they conducted a comparison between the performance of LLMs when combined with PEFT and In-Context Learning and provided more insights into the results. Wang et al. [66] empirically evaluated various prompt tuning techniques for code intelligent tasks. They analyzed CodeBERT [18] and CodeT5 [68], fully fine-tuned and with prompt tuning, for defect prediction, code summarization, and code translation in terms of accuracy and data efficiency.

Li et al. [29] evaluated fine-tuning of general-LLMs and code-LLMs including T5 and CodeT5 fine-tuned with several PEFT techniques such as LoRA, Houlsby [24] and Pfeiffer [45] adapters for clone and defect detection and code search and translation. They additionally analyzed the GPU usage and training time of these PEFT methods on the evaluated tasks. In their empirical study, Storhaug and Li [61] focused on unit test generation using CodeGen [42], StarCoder [30] and CodeLlama [52] family of models fine-tuned with LoRA, IA³ and prompt tuning, and report that LoRA outperforms other studied PEFT methods, even surpassing full fine-tuning in some cases.

Though there are many studies using PEFT methods for code-related tasks, PEFT methods are under-researched in the low-resource domain and particularity for R. Nevertheless, Cassano et al. [9] proposed a novel technique to fine-tune LLMs on low-resource languages by generating semi-synthetic data from high-resource languages and leveraging LLM-generated unit tests. Using their approach, they evaluated the StarCoder and CodeLlama models on several low-resource lan-

迁移能力和可扩展性。Lialin 等人 [34] 对近年来引入的各种参数高效微调（PEFT）方法进行了详细分析，将其归类为三大类：基于添加的方法、基于选择的方法和基于重参数化的方法。他们对这些方法在存储效率、内存效率、计算效率、准确性和推理开销方面进行了比较评估。

6.2 软件工程中的参数高效微调研究

Goel 等人 [20] 是最早在软件工程中研究适配器（adapters），一种特定的 PEFT 技术的学者之一。他们采用瓶颈适配器，并评估其在自然语言模型向代码任务知识迁移中的能力。他们在 [54] 的基础上，将该方法扩展到其他软件工程任务和代码-大模型（code-LLMs），并显示通过使用适配器，模型性能得到了提升或与完全微调的结果相当，适用于所研究的语言和任务。Liu 等人 [36] 对包括 CodeBERT [18] 和 CodeT5 [68] 在内的多种编码器单一和编码器 - 解码器代码大模型，进行了关于代码总结等任务的 PEFT 方法的实证研究。他们探讨了可训练参数数量对 LoRA、适配器及其他两种 PEFT 方法的影响，以及这些方法在低资源、跨语言和跨项目环境中的性能表现。

Weyssow 等人 [71] 研究了 LoRA、IA³ 和 Prompt Tuning [28] 在应用于代码生成的代码-LLMs 时的表现。他们的主要重点在于评估 PEFT 方法在大型语言模型与小型语言模型中的优势。此外，他们还比较了结合 PEFT 和 In-Context Learning 的大模型性能，并提供了更多关于结果的见解。Wang 等人 [66] 实证评估了多种用于代码智能任务的提示调优技术。他们分析了在缺陷预测、代码总结和代码翻译任务中，CodeBERT[18] 和 CodeT5 [68], 经过完全微调和提示调优后的准确性和数据效率。

李等人 [29] 评估了通用大规模语言模型（LLMs）和代码大规模语言模型（Code-LLMs）的微调效果，包括使用多种PEFT技术（如LoRA、Houlsby[24] 和 Pfeiffer [45]）对T5和CodeT5进行微调，应用于克隆和缺陷检测以及代码搜索与翻译任务。他们还分析了这些PEFT方法在所评估任务中的GPU使用情况和训练时间。在他们的实证研究中，Storhaug和李 [61] 重点关注使用CodeGen [42]、StarCoder [30] 和CodeLlama [52] 家族模型，通过LoRA、IA³ 和提示调优进行微调的单元测试生成，并报告LoRA在性能上优于其他研究的PEFT方法，甚至在某些情况下超越了全量微调。

尽管已有许多研究采用PEFT方法进行与代码相关的任务，但在低资源领域，尤其是R中的PEFT方法仍然研究不足。然而，Cassano等人 [9] 提出了一种新颖的技术，通过从高资源语言生成半合成数据并利用LLM生成的单元测试，对大规模语言模型进行微调，以适应低资源语言。利用他们的方法，他们在多个低资源语言上对StarCoder和CodeLlama模型进行了评估。

guages including R before and after fine-tuning them on their datasets. Chen et al. [10] explored the transferability of several high-resource and low-resource languages for clone detection, error detection, solution domain classification and code repair using CodeT5. Building on their findings, they proposed a method to detect the best combination of source and target programming languages for knowledge transfer. Though these works are related to low-resource languages, they do not study PEFT approaches.

The styling quality of LLM-generated code is relatively under-explored, particularly for PEFT methods and low-resource languages such as R. Shivashankar and Martini [59] conduct an in-depth study on Python code quality generated by various language models, including language models fine-tuned with QLoRA [15].

Differences between our work and the current literature.

The number of studies on PEFT approaches in software engineering is limited. With the advent of newer PEFT approaches, and large language models with billions of parameters, there is still a gap in understanding which of the NLP techniques can be applied to code, and to what extent. Though some of our proposed PEFT methods, target tasks, and LLMs have been used separately in previous works, our proposed study is novel and different in the following aspects.

First, to the best of our knowledge, no work delves into a direct comparison of the applicability of PEFT methods on code summarization and generation when applied to a diverse range of open-source code-LLMs and general-LLMs, including large-scale decoder-only models and relatively smaller encoder-decoder models. Open-source LLMs gained increased attention from the research community as they are accessible and provide opportunities for various research such as training the models, rather than just using the models. Therefore, we consider investigating open-source models an important aspect of our study.

Second, while Compacter is investigated by other works [26, 72] in natural language, its application in the software engineering domain remains unknown. As far as we are aware, we are the first to employ and examine Compacter in software engineering. Finally, employing PEFT for knowledge transfer to low-resource programming languages in software engineering is under-explored. R in particular, is a low-resource language that has not been studied in this domain before.

7 Threats to Validity

Internal Threats. An internal threat can be associated with our chosen data types and quantization configuration for training and testing models. Since we conduct a comparative analysis and use identical settings for LoRA and Compacter, we avoid this threat in their case. For IA³, we minimize the effect by only reporting its performance metrics and omitting it from further experiments in section 5.

Hyperparameters and implementation could be the cause of another internal threat. We resort to the default hyperparameters of libraries and those employed by the authors of each PEFT method as the same best practice hyperparameters, in addition to exploring LoRA and Compacter's effectiveness with different hyperparameters in section 5, and utilize greedy decoding to report deterministic and

包括 R 在内的多种语言，在对其数据集进行微调之前和之后。Chen 等人[10] 利用 CodeT5 探讨了多种高资源和低资源语言在克隆检测、错误检测、解题领域分类和代码修复方面的迁移能力。在此基础上，他们提出了一种方法，用于检测最优的源语言和目标语言组合以实现知识迁移。尽管这些工作与低资源语言相关，但并未研究参数高效微调（PEFT）方法。

LLM生成代码的样式质量尚未得到充分研究，特别是对于PEFT方法和低资源语言如R。Shivashankar和Martini [59] 对由各种语言模型生成的Python代码质量进行了深入研究，包括使用QLoRA [15]进行微调的语言模型。

我们工作与现有文献的差异。

在软件工程中，关于参数高效微调（PEFT）方法的研究数量有限。随着更新的PEFT方法的出现以及具有数十亿参数的大型语言模型（LLMs）的发展，仍存在对哪些自然语言处理（NLP）技术可以应用于代码以及应用程度的理解差距。尽管我们提出的部分PEFT方法、目标任务和LLMs在以往的研究中已被单独使用，但我们提出的研究在以下几个方面具有创新性和差异性。

据我们所知，目前尚无研究对PEFT方法在代码总结与生成任务中在不同类型的开源代码大型语言模型（Code-LLMs）和通用大型语言模型（General-LLMs）上的适用性进行直接比较，包括大规模解码器模型和相对较小的编码器-解码器模型。开源大型语言模型因其可获取性而受到研究界的广泛关注，为模型训练等多种研究提供了机会，而不仅仅是模型的使用。因此，我们认为研究开源模型是本研究的重要方面。

其次，虽然Compacter在自然语言领域由其他研究 [26, 72] 进行了研究，但其在软件工程领域的应用仍然未知。据我们所知，我们是首个在软件工程中采用并检验Compacter的研究。最后，利用PEFT进行软件工程中低资源编程语言的知识迁移尚未得到充分探索。特别是R语言，作为一种低资源语言，在该领域尚未被研究过。

7 影响效度的威胁

内部威胁。内部威胁可能与我们选择的数据类型和量化配置用于训练和测试模型相关。由于我们进行了比较分析并对LoRA和Compacter采用了相同的设置，因此在它们的情况下避免了此威胁。对于IA³，我们通过仅报告其性能指标并在第5节中省略其后续实验，最大程度地减少了其影响。

超参数和实现方式可能是另一个内部威胁的原因。我们采用库的默认超参数以及各个PEFT方法作者所使用的超参数，作为相同的最佳实践超参数，此外在第5节中探索了LoRA和Compacter在不同超参数下的效果，并采用贪婪解码以报告确定性结果。

reproducible results. Given that implementation is out of this paper's scope, we resort to the implementations provided by well-established HuggingFace's peft [40] and AdapterHub [47] libraries.

Another potential threat could be related to the evaluation of RQ3, where we assess the models' performance on the *unseen* programming language, R. We have investigated the *unseenness* by referring to the original papers of the selected models. CodeT5 has released its training data and R is not included in that dataset. Though the training dataset of CodeLlama is not public, the list of programming languages used in its training is available and R is not among them. So the threat related to R being seen during the pre-training of the models is alleviated.

The final internal threat revolves around analyzing the GPU memory utilization of the PEFT methods by measuring their peak memory usage, as libraries' internal implementations and functionalities such as caching mechanisms and periodical evaluation phases can introduce misleading momentary memory usage peaks. To alleviate this threat, we have taken measures to disable such functionalities. Additionally, for all models, we have confirmed the correctness of peak memory usage by comparing them against the average of periodical memory usage measurements during their training.

External Threats. This threat is related to the generalization of the results. We have conducted experiments for code summarization and code generation, two families of models, T5 and Llama, using IA³, Compacter, and LoRA and unseen language R. Though we conducted several experiments, the results might not be applicable to other programming languages and PEFT methods, or other code-related tasks.

Construct Threats. One thread can be our choice of models. We mitigate this threat by selecting models from a wide range of sizes, architecture and modalities (i.e., code and natural language LLMs) that are commonly employed in similar studies of language models in software engineering. Moreover, we omit specialized models (e.g., instruction-tuned models) and avoid prompt engineering practices to mitigate the confounding effects of prompting format on the effectiveness of PEFT methods.

Not opting to fully fine-tune the Llama family of models as baselines can be another threat. Given the substantial resources required, fully fine-tuning these models was not within our capabilities. We opt to use these models in zero-shot settings to minimize this threat, which is a common practice of leveraging large pre-trained LLMs in the literature.

A common construct threat in this domain is misrepresenting the overall performance of LLMs or PEFT methods by purely comparing metrics such as BLEU-4, CodeBLEU and Pass@k. We avoid this threat by clarifying how we applied each method and separately reporting these methods when available. Additionally, we perform a set of qualitative studies to shed light on other capability aspects of PEFT methods.

Conclusion Threats. Finally, the lack of sufficient experiment repetition can be considered a threat. Due to the significantly high number of experiments in this study, their resource cost and negative environmental side-effects, we avoided performing multiple runs of each fine-tuning. To mitigate this effect, however, we

可复现的结果。鉴于实现超出了本文的范围，我们采用了由成熟的 HuggingFace 的 peft [40] 和 AdapterHub [47] 库提供的实现。

另一个潜在的威胁可能与对 RQ3 的评估有关，即我们评估模型在未见过编程语言 R 上的性能。我们通过参考所选模型的原始论文，调查了未见性。CodeT5 已公布其训练数据，但该数据集中不包括 R。虽然 CodeLlama 的训练数据集未公开，但其使用的编程语言列表是可获得的，R 也不在其中。因此，与模型预训练期间 R 被见到相关的威胁得到了缓解。

最终的内部威胁涉及通过测量 PEFT 方法的峰值内存使用情况，分析其 GPU 内存利用率，因为库的内部实现和功能（如缓存机制和周期性评估阶段）可能引入误导性的瞬时内存峰值。为减轻这一威胁，我们已采取措施禁用此类功能。此外，对于所有模型，我们通过将峰值内存使用与其训练期间周期性内存测量的平均值进行比较，确认了峰值内存使用的正确性。

External Threats. 此威胁与结果的泛化性相关。我们针对代码总结和代码生成这两类模型，使用 IA³, Compacter、LoRA 以及未见过的语言 R，进行了实验。尽管我们进行了多次实验，但结果可能不适用于其他编程语言、PEFT 方法或其他与代码相关的任务。

构建威胁。 一个潜在威胁可能来自我们选择的模型。我们通过选择不同规模、架构和模态（即代码和自然语言的大型语言模型）范围内的模型来减轻这一威胁，这些模型在软件工程中的语言模型相关研究中被广泛采用。此外，我们省略了专用模型（例如，指令调优模型）并避免使用提示工程技术，以减轻提示格式对 PEFT 方法效果的混淆影响。

不选择对 Llama 系列模型进行完全微调作为基线，可能是另一种威胁。鉴于所需资源巨大，完全微调这些模型超出了我们的能力范围。我们选择在零样本设置中使用这些模型，以最大程度地减少这一威胁，这也是文献中常用的利用大型预训练大模型的做法。

在该领域中，一个常见的构造性威胁是通过纯粹比较 BLEU-4、CodeBLEU 和 Pass@k 等指标，误导对 LLMs 或 PEFT 方法整体性能的评估。我们通过澄清每种方法的应用方式，并在可用时单独报告这些方法，以避免此类威胁。此外，我们还进行了一系列定性研究，以揭示 PEFT 方法的其他能力方面。

结论威胁。最后，缺乏充分的实验重复可以被视为一种威胁。鉴于本研究中实验数量显著较多，资源成本高且对环境产生负面影响，我们避免对每次微调进行多次运行。然而，为了减轻这一影响，我们

establish the significance of the results through inferential statistical tests, detailed in section 3. We also conducted several other experiments including manual evaluation and robustness to ensure the reliability of the results.

8 Conclusion and Future Works

In this study, we presented an empirical comparison of PEFT methods within the domain of software engineering, employing a variety of foundational models, including code-specific and natural language LLMs. Our analysis encompasses the performance evaluation of each model augmented with distinct PEFT methods across six prevalent programming languages, accompanied by a comprehensive discussion of their effectiveness with different parameter budgets and their generation's characteristics. Furthermore, we explore the PEFT methods' efficacy when applied to a programming language that was not included during the training phase, R in our case, to evaluate their performance in transferring knowledge to low-resource domains. This multifaceted analysis provides valuable insights into the adaptability and performance of PEFT methods in scenarios involving previously unseen programming languages, contributing to a deeper understanding of their practical applicability in software engineering contexts. Future lines of research include an in-depth analysis of the characteristics of PEFT methods' architectures in software engineering to introduce more effective methods for code-related tasks as well as exploiting current techniques for automatic tools to support code generation and other code-related tasks in R.

Data Availability Statement

We include all scripts and tooling used to obtain the results in our GitHub repository.⁷

Conflict of Interest

The authors declare that they have no conflict of interest.

References

1. Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1:*

⁷ https://github.com/Amirresm/empirical_peft_suite

通过推理统计检验（详见第3节）确立了结果的显著性。我们还进行了包括人工评估和鲁棒性测试在内的多项其他实验，以确保结果的可靠性。

8 结论与未来工作

在本研究中，我们在软件工程领域内对PEFT方法进行了实证比较，采用了包括代码特定模型和自然语言大模型（LLMs）在内的多种基础模型。我们的分析涵盖了在六种常用编程语言中，结合不同PEFT方法对每个模型的性能评估，并对其在不同参数预算下的有效性以及生成特征进行了全面讨论。此外，我们还探讨了在训练阶段未包含的编程语言（以R为例）中应用PEFT方法的效果，以评估其在知识迁移到低资源领域中的表现。这一多方面的分析为PEFT方法在面对未曾见过的编程语言场景中的适应性和性能提供了宝贵的见解，有助于深入理解其在软件工程中的实际应用潜力。未来的研究方向包括对软件工程中PEFT方法架构特征的深入分析，以引入更有效的代码相关任务方法，以及利用现有技术开发自动化工具，支持R中的代码生成及其他代码相关任务。

数据可用性声明

我们在我们的 GitHub 仓库中包含了用于获得结果的所有脚本和工具。⁷

利益冲突

作者声明不存在利益冲突。

参考文献

1. Armen Aghajanyan, Sonal Gupta, 和 Luke Zettlemoyer. 内在维度解释了语言模型微调的有效性。在宗庆清、夏飞、李文杰和Roberto Navigli编辑，第59届计算语言学协会年会暨第11届自然语言处理国际联合会议论文集（第1卷：

<https://github.com/Amirresm/empiricalpeftsuite>

- Long Papers*), pages 7319–7328, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.568. URL <https://aclanthology.org/2021.acl-long.568>.
2. Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Unified pre-training for program understanding and generation. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2655–2668, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.211. URL <https://aclanthology.org/2021.naacl-main.211>.
 3. Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. A transformer-based approach for source code summarization. *ArXiv*, abs/2005.00653, 2020. URL <https://api.semanticscholar.org/CorpusID:218486987>.
 4. Neel Alex, Eli Lifland, Lewis Tunstall, Abhishek Thakur, Pegah Maham, C. Jess Riedel, Emmie Hine, Carolyn Ashurst, Paul Sedille, Alexis Carlier, Michael Noetel, and Andreas Stuhlmüller. Raft: A real-world few-shot text classification benchmark. *ArXiv*, abs/2109.14076, 2021. URL <https://api.semanticscholar.org/CorpusID:238215290>.
 5. Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *ArXiv*, abs/2108.07732, 2021. URL <https://api.semanticscholar.org/CorpusID:237142385>.
 6. Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1–9, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-short.1. URL <https://aclanthology.org/2022.acl-short.1>.
 7. Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.

- 《大型语言模型参数高效方法的实证研究：代码与知识迁移到R》 Long Papers), 第7319–7328页, 2021年8月在线发表。计算语言学协会。doi: 10.18653/v1/2021.acl-long.568。网址 <https://aclanthology.org/2021.acl-long.568>。
2. Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, 和 Kai-Wei Chang. 统一预训练用于程序理解与生成。在 Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, 和 Yichao Zhou 编者, 《2021年北美计算语言学协会人类语言技术会议论文集》, 第2655–2668页, 2021年6月在线发表。计算语言学协会。doi: 10.18653/v1/2021.naacl-main.211。网址 <https://aclanthology.org/2021.naacl-main.211>。
 3. Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, 和 Kai-Wei Chang. 基于Transformer的源代码摘要方法。 *ArXiv*, abs/2005.00653, 2020。网址 <https://api.semanticscholar.org/CorpusID:218486987>。
 4. Neel Alex, Eli Lifland, Lewis Tunstall, Abhishek Thakur, Pegah Maham, C. Jess Riedel, Emmie Hine, Carolyn Ashurst, Paul Sedille, Alexis Carlier, Michael Noetel, 和 Andreas Stuhlmüller. Raft: 一个真实世界的少样本文本分类基准。 *ArXiv*, abs/2109.14076, 2021。网址 <https://api.semanticscholar.org/CorpusID:238215290>。
 5. Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, 和 Charles Sutton. 使用大型语言模型进行程序合成。 *ArXiv*, abs/2108.07732, 2021。网址 <https://api.semanticscholar.org/CorpusID:237142385>。
 6. Elad Ben Zaken, Yoav Goldberg, 和 Shauli Ravfogel. BitFit: 一种简单的参数高效微调Transformer基础的掩码语言模型。在 Smaranda Muresan, Preslav Nakov, 和 Aline Villavicencio 编者, 《第 60 届计算语言学协会年会论文集 (第二卷: 短论文)》, 第1–9页, 爱尔兰都柏林, 2022年5月。计算语言学协会。doi: 10.18653/v1/2022.acl-short.1。网址 <https://aclanthology.org/2022.acl-short.1>。
 7. Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, 和 Dario Amodei. 语言模型是少样本学习者。在 H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, 和 H. Lin 编者, 《神经信息处理系统进展》, 第 33卷, 页1877–1901. Curran Associates, Inc., 2020。网址 https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf。

8. Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation . *IEEE Transactions on Software Engineering*, 49(07): 3675–3691, July 2023. ISSN 1939-3520. doi: 10.1109/TSE.2023.3267446. URL <https://doi.ieeecomputersociety.org/10.1109/TSE.2023.3267446>.

9. Federico Cassano, John Gouwar, Francesca Lucchetti, Claire Schlesinger, Carolyn Jane Anderson, Feldman, Molly Q, Michael Greenberg, Jangda, Abhinav, and Guha, Arjun. Knowledge Transfer from High-Resource to Low-Resource Programming Languages for Code LLMs. *Proceedings of the ACM on Programming Languages (PACMPL)*, 8(OOPSLA), 2024.

10. Fuxiang Chen, Fatemeh H. Fard, David Lo, and Timofey Bryksin. On the transferability of pre-trained language models for low-resource programming languages. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension, ICPC ’22*, page 401–412, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392983. doi: 10.1145/3524610.3527917. URL <https://doi.org/10.1145/3524610.3527917>.

11. Guanzheng Chen, Fangyu Liu, Zaiqiao Meng, and Shangsong Liang. Revisiting parameter-efficient tuning: Are we really there yet? In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2612–2626, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.168. URL <https://aclanthology.org/2022.emnlp-main.168>.

12. Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sas- try, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, S. Arun Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021. URL <https://api.semanticscholar.org/CorpusID:235755472>.

13. Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garca, Vedant Misra, Kevin Robinson, Liam Fedus,

8. Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, 和 Abhinav Jangda. MultiPL-E: 一种可扩展的多语言方法, 用于神经代码生成的基准测试。 *IEEETransactionson Software Engineering*, 第49卷第07期: 3675–3691页, 2023年7月。ISSN 1939-3520。doi: 10.1109/TSE.2023.3267446。网址 <https://doi.ieeecomputersociety.org/10.1109/TSE.2023.3267446>。9. Federico Cassano, John Gouwar, Francesca Lucchetti, Claire Schlesinger, Carolyn Jane Anderson, Feldman, Molly Q, Michael Greenberg, Jangda, Abhinav, 和 Guha, Arjun. 从高资源到低资源编程语言的知识迁移用于代码大模型。 *Proceedings of the ACM on Programming Languages (PACMPL)*, 第8卷 (OOPSLA) , 2024年。10. Fuxiang Chen, Fatemeh H. Fard, David Lo, 和 Timofey Bryksin. 预训练语言模型在低资源编程语言中的迁移能力研究。发表于 *Proceedings of the 30th IEEE/ACM International Conference onProgramComprehension*, ICPC ’ 22, 第401–412页, 纽约, 美国, 2022年。计算机协会。ISBN 9781450392983。doi: 10.1145/ 3524610.3527917。网址 <https://doi.org/10.1145/3524610.3527917>。11. Guanzheng Chen, Fangyu Liu, Zaiqiao Meng, 和 Shangsong Liang. 重访参数高效调优: 我们真的已经到达了吗? 由Yoav Goldberg、Zornitsa Kozareva 和 Yue Zhang编辑, 发表于2022年自然语言处理实证方法会议论文集, 第2612–2626页, 阿布扎比, 阿联酋, 2022年12月。计算语言学协会。doi: 10.18653/v1/2022.emnlp-main.168。网址 <https://aclanthology.org/2022.emnlp-main.168>。

12. Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sas- try, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, S. Arun Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, 和 Wojciech Zaremba. 评估在代码上训练的大型语言模型。 *ArXiv*, abs/2107.03374, 2021。URL<https://api.semanticscholar.org/CorpusID:235755472>。

13. Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garca, Vedant Misra, Kevin Robinson, Liam Fedus

- Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *J. Mach. Learn. Res.*, 24:240:1–240:113, 2022. URL <https://api.semanticscholar.org/CorpusID:247951931>.
14. Andrew M Dai and Quoc V Le. Semi-supervised sequence learning. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/7137debd45ae4d0ab9aa953017286b20-Paper.pdf.
 15. Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=OUIFPHEgJU>.
 16. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics*, 2019. URL <https://api.semanticscholar.org/CorpusID:52967399>.
 17. Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235, 2023.
 18. Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In Trevor Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.139. URL <https://aclanthology.org/2020.findings-emnlp.139>.
 19. Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. Vulrepair: A t5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, page 935–947, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394130. doi: 10.1145/3540250.3549098. URL <https://doi.org/10.1145/3540250.3549098>.
 20. Divyam Goel, Ramansh Grover, and Fatemeh H Fard. On the cross-modal transfer from natural language to code through adapter modules. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, pages 71–81, 2022.

- Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, Noah Fiedel. Palm: 利用路径扩展的语言模型规模化。 *J. Mach. Learn. Res.*, 2022, 24: 240:1–240:113. URL <https://api.semanticscholar.org/CorpusID:247951931>.
14. Andrew M Dai 和 Quoc V Le. 半监督序列学习。收录于 C. Cortes、N. Lawrence、D. Lee、M. Sugiyama 和 R. Garnett 编著的《*Advances in Neural Information Processing Systems*》，第28卷。Curran Associates, Inc., 2015。URL https://proceedings.neurips.cc/paper_files/paper/2015/file/7137debd45ae4d0ab9aa953017286b20-Paper.pdf。
15. Tim Dettmers、Artidoro Pagnoni、Ari Holtzman 和 Luke Zettlemoyer。QLoRA: 量化大规模语言模型的高效微调。收录于第37届神经信息处理系统会议, 2023。URL <https://openreview.net/forum?id=OUIFPHEgJU>。
16. Jacob Devlin、Ming-Wei Chang、Kenton Lee 和 Kristina Toutanova。BERT: 用于语言理解的深度双向变换器预训练模型。收录于北美计算语言学协会, 2019。URL <https://api.semanticscholar.org/CorpusID:52967399>。
17. Ning Ding、Yujia Qin、Guang Yang、Fuchao Wei、Zonghan Yang、Yusheng Su、Shengding Hu、Yulin Chen、Chi-Min Chan、Weize Chen 等。大规模预训练语言模型的参数高效微调。 *Nature Machine Intelligence*, 2023年第5卷第3期: 220–235。
18. Zhangyin Feng、Daya Guo、Duyu Tang、Nan Duan、Xiaocheng Feng、Ming Gong、Linjun Shou、Bing Qin、Ting Liu、Daxin Jiang 和 Ming Zhou。CodeBERT: 一种用于编程和自然语言的预训练模型。收录于 Trevor Cohn、Yulan He 和 Yang Liu 编著的《自然语言处理协会会议论文集: *EMNLP 2020*》，第1536–1547页, 线上, 2020年11月。自然语言处理协会。doi: 10.18653/v1/2020.findings-emnlp.139。URL <https://aclanthology.org/2020.findings-emnlp.139>。
19. Michael Fu、Chakkrit Tantithamthavorn、Trung Le、Van Nguyen 和 Dinh Phung。Vulrepair: 基于T5的自动软件漏洞修复。收录于第30届ACM欧洲软件工程联合会议暨软件工程基础研讨会, ESEC/FSE 2022, 第935–947页, 纽约, 美国, 2022。计算机协会。ISBN 9781450394130。doi: 10.1145/3540250.3549098。URL <https://doi.org/10.1145/3540250.3549098>。
20. Divyam Goel、Ramansh Grover 和 Fatemeh H Fard。通过适配器模块实现自然语言到代码的跨模态迁移。收录于第30届IEEE/ACM国际程序理解会议, 第71–81页, 2022。

21. Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Jian Yin, Daxin Jiang, and M. Zhou. Graphcodebert: Pre-training code representations with data flow. *ArXiv*, abs/2009.08366, 2020. URL <https://api.semanticscholar.org/CorpusID:221761146>.

22. Ruidan He, Linlin Liu, Hai Ye, Qingyu Tan, Bosheng Ding, Liying Cheng, Jiawei Low, Lidong Bing, and Luo Si. On the effectiveness of adapter-based tuning for pretrained language model adaptation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2208–2222, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.172. URL <https://aclanthology.org/2021.acl-long.172>.

23. Lars St hle and Svante Wold. Analysis of variance (anova). *Chemometrics and Intelligent Laboratory Systems*, 6(4):259–272, 1989. ISSN 0169-7439. doi: [https://doi.org/10.1016/0169-7439\(89\)80095-4](https://doi.org/10.1016/0169-7439(89)80095-4). URL <https://www.sciencedirect.com/science/article/pii/0169743989800954>.

24. Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.

25. Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

26. Ting Hu, Christoph Meinel, and Haojin Yang. Scaled prompt-tuning for few-shot natural language generation. *ArXiv*, abs/2309.06759, 2023. URL <https://api.semanticscholar.org/CorpusID:261705827>.

27. Hamel Husain, Hongqiu Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. *ArXiv*, abs/1909.09436, 2019. URL <https://api.semanticscholar.org/CorpusID:202712680>.

28. Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.243. URL <https://aclanthology.org/2021.emnlp-main.243>.

29. Guochang Li, Chen Zhi, Jialiang Chen, Junxiao Han, and Shuiguang Deng. A comprehensive evaluation of parameter-efficient fine-tuning on automated program repair. *CoRR*, abs/2406.05639, 2024. URL <https://doi.org/10.48550/arXiv.2406.05639>.

30. Raymond Li, Loubna Ben allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier De-

21. 郭大雅、任硕、卢帅、冯张银、唐笃宇、刘书杰、周龙、段楠、尹剑、江大新、周迈。GraphCodeBERT：利用数据流进行代码表示的预训练。*ArXiv*, abs/2009.08366, 2020. 网址 <https://api.semanticscholar.org/CorpusID:221761146>。

22. 何睿丹、刘琳琳、叶海、谭青瑜、丁博胜、程丽颖、Low J-awei、冰丽东、司罗。基于适配器的预训练语言模型调优效果研究。收录于宗成庆、夏飞、李文杰、纳吉·罗伯托编辑的《第59届计算语言学协会年会论文集及第11届自然语言处理国际联合会议（第1卷：长论文）》中，页码2208–2222，线上，2021年8月。计算语言学协会。doi: 10.18653/v1/2021.acl-long.172. 网址 <https://aclanthology.org/2021.acl-long.172>。

23. Lars Stähle 和 Svante Wold. 方差分析 (ANOVA)。 *Chemometrics and Intelligent Laboratory Systems*, 6(4):259–272, 1989. ISSN 0169-7439. doi: [https://doi.org/10.1016/0169-7439\(89\)80095-4](https://doi.org/10.1016/0169-7439(89)80095-4)。 网址 <https://www.sciencedirect.com/science/article/pii/0169743989800954>。

24. Neil Houlsby、Andrei Giurgiu、Stanislaw Jastrzebski、Bruna Morrone、Quentin De Laroussilhe、Andrea Gesmundo、Mona Attariyan 和 Sylvain Gelly。参数高效的迁移学习在自然语言处理中的应用。收录于国际机器学习会议，页码2790–2799。PMLR，2019。

25. Edward J Hu, Yelong Shen、Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang 和 Weizhu Chen。LoRA：大规模语言模型的低秩适应。收录于国际学习表征会议，2022。网址 <https://openreview.net/forum?id=nZeVKeeFYf9>。

26. 胡婷、Christoph Meinel 和 杨浩金。用于少样本自然语言生成的缩放提示调优。*ArXiv*, abs/2309.06759，2023。网址 <https://api.semanticscholar.org/CorpusID:261705827>。

27. Hamel Husain、吴宏秋、Gazit Tiferet、Miltiadis Allamanis 和 Marc Brockschmidt。Codesearchnet挑战：评估语义代码搜索的现状。*ArXiv*, abs/1909.09436，2019。网址 <https://api.semanticscholar.org/CorpusID:202712680>。

28. Brian Lester、Rami Al-Rfou 和 Noah Constant。规模化在参数高效提示调优中的作用。收录于Marie-Francine Moens、黄轩晶、Lucia Specia 和 Scott Wen-tau Yih编辑的《2021年自然语言处理实证方法会议论文集》，页码3045–3059，线上及多明尼加共和国蓬塔卡纳，2021年11月。计算语言学协会。doi: 10.18653/v1/2021.emnlp-main.243。 网址 <https://aclanthology.org/2021.emnlp-main.243>。

29. 李国昌、陈臻、陈佳良、韩俊骁、邓水光。参数高效微调在自动程序修复中的全面评估。*CoRR*, abs/2406.05639，2024。网址 <https://doi.org/10.48550/arXiv.2406.05639>。

30. Raymond Li、Loubna Ben Allal、Zi Yangtian、Niklas Muennighoff、Denis Kocetkov、Mou Chenghao、Marc Marone、Christopher Akiki、Jia Li、Jenny Chim、Qian Liu、Evgenii Zheltonozhskii、Terry Yue Zhuo、Thomas Wang、Olivier De

haene, Joel Lamy-Poirier, Joao Monteiro, Nicolas Gontier, Ming-Ho Yee, Lo- gesh Kumar Umapathi, Jian Zhu, Ben Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason T Stillerman, Siva Sankalp Patel, Dmitry Ab- ulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Urvashi Bhattacharyya, Wenhao Yu, Sasha Luccioni, Paulo Villegas, Fedor Zhdanov, Tony Lee, Nadav Timor, Jennifer Ding, Claire S Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jer- nite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Lean- dro Von Werra, and Harm de Vries. Starcoder: may the source be with you! *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=KoFOg41haE>. Reproducibility Certi- fication.

31. Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Com- putational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online, Au- gust 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021. acl-long.353. URL <https://aclanthology.org/2021.acl-long.353>.

32. Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. Coderretrieve: Uni- modal and bimodal contrastive learning. *ArXiv*, abs/2201.10866, 2022. URL <https://api.semanticscholar.org/CorpusID:246285444>.

33. Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158. URL <https://www.science.org/doi/abs/10.1126/science.abq1158>.

34. Vladislav Lialin, Vijeta Deshpande, Xiaowei Yao, and Anna Rumshisky. Scal- ing down to scale up: A guide to parameter-efficient fine-tuning, 2024. URL <https://arxiv.org/abs/2303.15647>.

35. Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *Advances in Neural Information Processing Systems*, 35:1950–1965, 2022.

36. J. Liu, C. Sha, and X. Peng. An empirical study of parameter-efficient fine- tuning methods for pre-trained code models. In *2023 38th IEEE/ACM Inter- national Conference on Automated Software Engineering (ASE)*, pages 397–408, Los Alamitos, CA, USA, sep 2023. IEEE Computer Society. doi: 10.1109/ ASE56229.2023.00125. URL <https://doi.ieeecomputersociety.org/10.1109/ASE56229.2023.00125>.

haene, Joel Lamy-Poirier, Joao Monteiro, Nicolas Gontier, Ming-Ho Yee, Lo- gesh Kumar Umapathi, Jian Zhu, Ben Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason T Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Urvashi Bhattacharyya, Wenhao Yu, Sasha Luccioni, Paulo Villegas, Fedor Zhdanov, Tony Lee, Nadav Timor, Jennifer Ding, Claire S Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro Von Werra, and Harm de Vries. Starcoder: 愿源代码与你同在! *Transactions on Machine Learning Research*, 2023年. ISSN 2835-8856. URL <https://openreview.net/forum?id=KoFOg41haE>. 可复现性认 证。

31. Xiang Lisa Li 和 Percy Liang. 前缀调优：优化连续提示以进行生成。在 Chengqing Zong、Fei Xia、Wenjie Li 和 Roberto Navigli 编者，第59届计算语 言学协会年会论文集及第11届自然语言处理国际联合会议（第1卷：长论文），第 4582–4597页，线上，2021年8月。计算语言学协会。doi: 10.18653/v1/2021.acl-long.353。网址 <https://aclanthology.org/2021.acl-long.353>。

32. 李晓楠，龚叶云，沈野龙，邱锡鹏，张航，姚博伦，齐维震，江大鑫，陈维竹， 段楠。Coderretrieve：单模态与双模态对比学习。 *ArXiv*, abs/2201.10866, 2022。链接 <https://api.semanticscholar.org/CorpusID:246285444>。

33. Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’ Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, 和 Oriol Vinyals. 使用AlphaCode实现竞赛级代码生成。 *Science*, 378(6624):1092–1097, 2022年。doi: 10.1126/science.abq1158. URL <https://www.science.org/doi/abs/10.1126/science.abq1158>。

34. Vladislav Lialin, Vijeta Deshpande, Xiaowei Yao, 和 Anna Rumshisky. 从缩小到 扩大：参数高效微调指南，2024。URL <https://arxiv.org/abs/2303.15647>。

35. Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, 和 Colin A. Raffel. 少样本参数高效微调优于上下文学习，神经信息处 理系统进展，35:1950–1965，2022。

36. J. Liu, C. Sha, 和 X. Peng. 预训练代码模型 参数高效微调方法的实证研究。发表于2023年第38届IEEE/ACM自动化软件工程国际会 议（ASE），第397–408页，洛斯阿拉米托斯，加利福尼亚州，美国，2023年9月。 IEEE计算机学会。doi: 10.1109/ASE56229.2023.00125。 U <https://doi.ieeecomputersociety.org/10.1109/ASE56229.2023.00125>。

37. Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019. URL <http://arxiv.org/abs/1907.11692>.
38. Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, MING GONG, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie LIU. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021. URL <https://openreview.net/forum?id=6lE4dQXaUcb>.
39. Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, 2021. URL <https://openreview.net/forum?id=bqGK5PyI6-N>.
40. Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
41. Mary McHugh. Interrater reliability: The kappa statistic. *Biochemia medica : časopis Hrvatskoga društva medicinskih biokemičara / HDMB*, 22:276–82, 10 2012. doi: 10.11613/BM.2012.031.
42. Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=iaYcJKpY2B_.
43. Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ACL '02, page 311–318, USA, 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL <https://doi.org/10.3115/1073083.1073135>.
44. Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In Marilyn Walker, Heng Ji, and Amanda Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1202. URL <https://aclanthology.org/N18-1202>.
45. Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, and Sebastian Ruder. MAD-X: An Adapter-Based Framework for Multi-Task Cross-Lingual Transfer. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*

37. Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, 和 Veselin Stoyanov. Roberta: 一种稳健优化的BERT预训练方法. *CoRR*, abs/1907.11692, 2019. 网址 <http://arxiv.org/abs/1907.11692>.
38. Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, MING GONG, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, 和 Shu-jie LIU. CodeXGLUE: 用于代码理解与生成的机器学习基准数据集. 在第35届神经信息处理系统会议数据集与基准赛道（第一轮），2021. 网址 <https://openreview.net/forum?id=6lE4dQXaUcb>.
39. Rabeeh Karimi Mahabadi, James Henderson, 和 Sebastian Ruder. Compacter: 高效低秩超复杂适配器层. 在A. Beygelzimer, Y. Dauphin, P. Liang, 和 J. Wortman Vaughan编辑的神经信息处理系统进展，2021. 网址 <https://openreview.net/forum?id=bqGK5PyI6-N>.
40. Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, 和 Benjamin Bossan. Peft: 最先进的参数高效微调方法. <https://github.com/huggingface/peft>, 2022.
41. Mary McHugh. 评估者一致性: κ 系数. *Biochemia medica*: 克罗地亚医学生物化学学会杂志, 22: 276–82, 2012年10月. doi: 10.11613/BM.2012.031.
42. Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, 和 Caiming Xiong. Codegen: 一种用于多轮程序合成的开源大型语言模型. 在第十一届表示学习国际会议，2023. 网址 https://openreview.net/forum?id=iaYcJKpY2B_.
43. Kishore Papineni, Salim Roukos, Todd Ward, 和 Wei-Jing Zhu. Bleu: 一种机器翻译自动评估方法. 在第40届计算语言学协会年会议论文集, ACL '02, 第311–318页, 美国, 2002. 计算语言学协会. doi: 10.3115/1073083.1073135. 网址 <https://doi.org/10.3115/1073083.1073135>.
44. Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, 和 Luke Zettlemoyer. 深度上下文化词表示. 在Marilyn Walker, Heng Ji, 和 Amanda Stent编辑的2018年北美计算语言学会会议: 人类语言技术论文集（长文），第2227–2237页, 新奥尔良, 路易斯安那州, 2018年6月. 计算语言学会. doi: 10.18653/v1/N18-1202. 网址 <https://aclanthology.org/N18-1202>.
45. Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, 和 Sebastian Ruder. MAD-X: 一种基于适配器的多任务跨语言迁移框架. 在Bonnie Webber, Trevor Cohn, Yulan He, 和 Yang Liu编辑的2020年自然语言处理实证方法会议论文集。

- (EMNLP), pages 7654–7673, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.617. URL <https://aclanthology.org/2020.emnlp-main.617>.
46. Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Annibal, Alec Peltekian, and Yanfang Ye. CoTexT: Multi-task learning with code-text transformer. In Royi Lachmy, Ziyu Yao, Greg Durrett, Milos Gligoric, Junyi Jessy Li, Ray Mooney, Graham Neubig, Yu Su, Huan Sun, and Reut Tsarfaty, editors, *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog 2021)*, pages 40–47, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.nlp4prog-1.5. URL <https://aclanthology.org/2021.nlp4prog-1.5>.
 47. Clifton Poth, Hannah Sterz, Indraneil Paul, Sukannya Purkayastha, Leon Engländer, Timo Imhof, Ivan Vulić, Sebastian Ruder, Iryna Gurevych, and Jonas Pfeiffer. Adapters: A unified library for parameter-efficient and modular transfer learning, 2023.
 48. Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. *OpenAI blog*, 2018.
 49. Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, page 9, 2019.
 50. Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), jan 2020. ISSN 1532-4435.
 51. Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, M. Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis. *ArXiv*, abs/2009.10297, 2020. URL <https://api.semanticscholar.org/CorpusID:221836101>.
 52. Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, I. Evtimov, Joanna Bitton, Manish P Bhatt, Cristian Cantón Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre D’efossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code. *ArXiv*, abs/2308.12950, 2023. URL <https://api.semanticscholar.org/CorpusID:261100919>.
 53. Iman Saberi, Fatemeh Fard, and Fuxiang Chen. Advfusion: Multilingual adapter-based knowledge transfer for code summarization, 2024. URL <https://arxiv.org/abs/2307.07854>.
 54. Iman Saberi, Fatemeh Fard, and Fuxiang Chen. Utilization of pre-trained language models for adapter-based knowledge transfer in software engineering. *Empirical Softw. Engg.*, 29(4), June 2024. ISSN 1382-3256. doi: 10.1007/s10664-024-10457-5. URL <https://doi.org/10.1007/s10664-024-10457-5>.
 55. R. Shahbazi and F Fard. Apicontext2com: Code comment generation by incorporating pre-defined api documentation. In *2023 IEEE/ACM 31st In-*

- (EMNLP), 第7654-7673页, 线上, 2020年11月。计算语言学协会. doi: 10.18653/v1/2020.emnlp-main.617. 网址 <https://aclanthology.org/2020.emnlp-main.617>。
46. Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Annibal, Alec Peltekian, 和 Yanfang Ye. CoTexT: 基于代码-文本变换器的多任务学习。在 Royi Lachmy, Ziyu Yao, Greg Durrett, Milos Gligoric, Junyi Jessy Li, Ray Mooney, Graham Neubig, Yu Su, Huan Sun 和 Reut Tsarfaty 编者, 《第一届自然语言处理编程工作坊论文集 (*NLP4Prog 2021*) 》, 第40-47页, 线上, 2021年8月。计算语言学协会. doi: 10.18653/v1/2021.nlp4prog-1.5. 网址 <https://aclanthology.org/2021.nlp4prog-1.5>。
 47. Clifton Poth, Hannah Sterz, Indraneil Paul, Sukannya Purkayastha, Leon Engländer, Timo Imhof, Ivan Vulić, Sebastian Ruder, Iryna Gurevych 和 Jonas Pfeiffer. 适配器: 一种统一的参数高效和模块化迁移学习库, 2023年。
 48. Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever 等。通过生成式预训练提升语言理解能力。 *OpenAI* 博客, 2018年。
 49. Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever 等。语言模型是无监督多任务学习者。 *OpenAI* 博客, 第9页, 2019年。
 50. Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li 和 Peter J. Liu. 探索统一文本到文本变换器的迁移学习极限。 *J. Mach. Learn. Res.*, 第21卷第1期, 2020年1月。ISSN 1532-4435。
 51. Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, M. Zhou, Ambrosio Blanco 和 Shuai Ma. Codebleu: 一种自动评估代码合成的方法。 *ArXiv*, abs/2009.10297, 2020年。网址 <https://api.semanticscholar.org/CorpusID:221836101>。
 52. Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, I. Evtimov, Joanna Bitton, Manish P Bhatt, Cristian Cantón Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre D’efossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom 和 Gabriel Synnaeve. Code llama: 面向代码的开源基础模型。 *ArXiv*, abs/2308.12950, 2023年。网址 <https://api.semanticscholar.org/CorpusID:261100919>。
 53. Iman Saberi, Fatemeh Fard 和 Fuxiang Chen. Advfusion: 基于多语言适配器的代码摘要知识迁移, 2024年。网址 <https://arxiv.org/abs/2307.07854>。
 54. Iman Saberi, Fatemeh Fard 和 Fuxiang Chen. 利用预训练语言模型进行软件工程中的适配器知识迁移。 *Empirical Softw. Engg.*, 第29卷第4期, 2024年6月。ISSN 1382-3256. doi: 10.1007/s10664-024-10457-5. 网址 <https://doi.org/10.1007/s10664-024-10457-5>。
 55. R. Shahbazi 和 F. Fard. Apicontext2com: 通过整合预定义API文档进行代码注释生成。在 *2023 IEEE/ACM第31届国际会议*

ternational Conference on Program Comprehension (ICPC), pages 13–24, Los Alamitos, CA, USA, may 2023. IEEE Computer Society. doi: 10.1109/ICPC58990.2023.00012. URL <https://doi.ieeecomputersociety.org/10.1109/ICPC58990.2023.00012>.

56. S. S. SHAPIRO and M. B. WILK. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3-4):591–611, dec 1965. doi: 10.1093/biomet/52.3-4.591. URL <https://doi.org/10.1093/biomet/52.3-4.591>.

57. R. Sharma, F. Chen, and F. Fard. Lamner: Code comment generation using character language model and named entity recognition. In *2022 IEEE/ACM 30th International Conference on Program Comprehension (ICPC)*, pages 48–59, Los Alamitos, CA, USA, may 2022. IEEE Computer Society. doi: 10.1145/3524610.3527924. URL <https://doi.ieeecomputersociety.org/10.1145/3524610.3527924>.

58. Noam M. Shazeer. Glu variants improve transformer. *ArXiv*, abs/2002.05202, 2020. URL <https://api.semanticscholar.org/CorpusID:211096588>.

59. Karthik Shivashankar and Antonio Martini. Better python programming for all: With the focus on maintainability, 2024. URL <https://arxiv.org/abs/2408.09134>.

60. Ankita Nandkishor Sontakke, Manasi Patwardhan, Lovekesh Vig, Raveendra Kumar Medicherla, Ravindra Naik, and Gautam Shroff. Code summarization: Do transformers really understand code? In *Deep Learning for Code Workshop*, 2022. URL https://openreview.net/forum?id=rI5l12_-1Zc.

61. André Storhaug and Jingyue Li. Parameter-efficient fine-tuning of large language models for unit test generation: An empirical study, 2024. URL <https://arxiv.org/abs/2411.02462>.

62. Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2023.127063>. URL <https://www.sciencedirect.com/science/article/pii/S0925231223011864>.

63. Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *ArXiv*, abs/2302.13971, 2023. URL <https://api.semanticscholar.org/CorpusID:257219404>.

64. Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Cantón Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schel-

国际程序理解会议 (ICPC), 第13–24页, 2023年5月, 洛杉矶阿拉米托斯, 加利福尼亚, 美国. IEEE计算机学会. doi: 10.1109/ICPC58990.2023.00012. 网址 <https://doi.ieeecomputersociety.org/10.1109/ICPC58990.2023.00012>.

56. S. S. SHAPIRO 和 M. B. WILK. 正态性方差分析检验 (完整样本)。 *Biometrika*, 52(3-4):591–611, 1965年12月. doi: 10.1093/biomet/52.3-4.591. 网址 <https://doi.org/10.1093/biomet/52.3-4.591>.

57. R. Sharma、F. Chen 和 F. Fard. Lamner: 使用字符语言模型和命名实体识别进行代码注释生成。在 *2022 IEEE/ACM 第30届国际程序理解会议 (ICPC)* , 第48–59页, 洛杉矶阿拉米托斯, 加利福尼亚, 美国, 2022年5月. IEEE计算机学会. doi: 10.1145/3524610.3527924. 网址 <https://doi.ieeecomputersociety.org/10.1145/3524610.3527924>.

58. Noam M. Shazeer. Glu变体改善Transformer. *ArXiv*, abs/2002.05202, 2020年. 网址<https://api.semanticscholar.org/CorpusID:211096588>.

59. Karthik Shivashankar 和 Antonio Martini. 更好的Python编程: 以可维护性为重点, 2024年. 网址<https://arxiv.org/abs/2408.09134>.

60. Ankita Nandkishor Sontakke、Manasi Patwardhan、Lovekesh Vig、Ravendra Kumar Medicherla、Ravindra Naik 和 Gautam Shroff. 代码总结: Transformer真的理解代码吗? 在深度学习与代码研讨会, 2022年. 网址https://openreview.net/forum?id=rI5l12_-1Zc.

61. André Storhaug 和 Jingyue Li. 大规模语言模型的参数高效微调: 一项实证研究, 2024年. 网址<https://arxiv.org/abs/2411.02462>.

62. Jianlin Su、Murtadha Ahmed、Yu Lu、Shengfeng Pan、Wen Bo 和 Yunfeng Liu。Roformer: 增强型旋转位置嵌入的Transformer. *Neurocomputing*, 第568页, 127063, 2024年. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2023.127063>. 网址 <https://doi.org/10.1016/j.neucom.2023.127063>.

63. Hugo Touvron、Thibaut Lavril、Gautier Izacard、Xavier Martinet、Marie-Anne Lachaux、Timothée Lacroix、Baptiste Rozière、Naman Goyal、Eric Hambro、Faisal Azhar、Aurelien Rodriguez、Armand Joulin、Edouard Grave 和 Guillaume Lample. Llama: 开源高效的基础语言模型. *ArXiv*, abs/2302.13971, 2023年. 网址 <https://api.semanticscholar.org/CorpusID:257219404>.

64. Hugo Touvron、Louis Martin、Kevin R. Stone、Peter Albert、Amjad Almahairi、Yasmine Babaei、Nikolay Bashlykov、Soumya Batra、Prajjwal Bhargava、Shruti Bhosale、Daniel M. Bikel、Lukas Blecher、Cristian Cantón Ferrer、Moya Chen、Guillem Cucurull、David Esiobu、Jude Fernandes、Jeremy Fu、Wenyin Fu、Brian Fuller、Cynthia Gao、Vedanuj Goswami、Naman Goyal、Anthony S. Hartshorn、Saghar Hosseini、Rui Hou、Hakan Inan、Marcin Kardas、Viktor Kerkez、Madian Khabsa、Isabel M. Kloumann、A. V. Korenev、Punit Singh Koura、Marie-Anne Lachaux、Thibaut Lavril、Jenya Lee、Diana Liskovich、Yinghai Lu、Yuning Mao、Xavier Martinet、Todor Mihaylov、Pushkar Mishra、Igor Molybog、Yixin Nie、Andrew Poulton、Jeremy Reizenstein、Rashi Rungta、Kalyan Saladi、Alan Schel-

ten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *ArXiv*, abs/2307.09288, 2023. URL <https://api.semanticscholar.org/CorpusID:259950998>.

65. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

66. C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, and M. R. Lyu. Prompt tuning in code intelligence: An experimental evaluation. *IEEE Transactions on Software Engineering*, 49(11):4869–4885, nov 2023. ISSN 1939-3520. doi: 10.1109/TSE.2023.3313881.

67. Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. One adapter for all programming languages? adapter tuning for code search and summarization. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE ’23, page 5–16. IEEE Press, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00013. URL <https://doi.org/10.1109/ICSE48619.2023.00013>.

68. Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8696–8708, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.685. URL <https://aclanthology.org/2021.emnlp-main.685>.

69. Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. CodeT5+: Open code large language models for code understanding and generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1069–1088, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.68. URL <https://aclanthology.org/2023.emnlp-main.68>.

70. Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models. *arXiv e-prints*, art. arXiv:2308.10462, August 2023. doi: 10.48550/arXiv.2308.10462.

71. Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. Exploring parameter-efficient fine-tuning techniques for code generation with large language models, 2024. URL <https://arxiv.org/abs/2308.10462>.

韦索 (Ten Wei), 鲁安·席尔瓦 (Ruan Silva), 埃里克·迈克尔·史密斯 (Eric Michael Smith), R. 苏布拉马尼亚安 (R. Subramanian), 谭夏 (Xia Tan), 汤平 (Binh Tang), 泰勒·罗斯 (Ross Taylor), 阿迪娜·威廉姆斯 (Adina Williams), 关建 (Jian Xiang Kuan), 徐普新 (Puxin Xu), 严正旭 (Zhengxu Yan), 伊利安·扎罗夫 (Iliyan Zarov), 张宇辰 (Yuchen Zhang), 安吉拉·范 (Angela Fan), 梅拉妮·坎巴杜尔 (Melanie Kambadur), 沙兰·纳兰 (Sharan Narang), 奥雷利安·罗德里格斯 (Aurelien Rodriguez), 罗伯特·斯托伊尼奇 (Robert Stojnic), 谢尔盖·埃杜诺夫 (Sergey Edunov), 托马斯·西亚洛姆 (Thomas Scialom)。Llama 2: 开源基础模型与微调聊天模型。*ArXiv*, abs/2307.09288, 2023. 网址 <https://api.semanticscholar.org/CorpusID:259950998>.

65. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin. Attention is all you need. 收录于I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (编辑), *Advances in Neural Information Processing Systems*, 第30卷. Curran Associates, Inc., 2017. 网址https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

66. C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, M. R. Lyu. 代码智能中的提示调优: 一项实验评估. *IEEETransactionson SoftwareEngineering*, 49(11): 4869–4885, 2023年11月. ISSN 1939-3520. doi: 10.1109/TSE.2023.3313881.

67. Deze Wang, 陈博星 (Boxing Chen), 李珊珊 (Shanshan Li), 罗伟 (Wei Luo), 彭少良 (Shaoliang Peng), 董伟 (Wei Dong), 廖翔科 (Xiangke Liao)。所有编程语言的一个适配器? 代码搜索与摘要的适配器调优。在第45届国际软件工程会议论文集, ICSE’ 23, 第5–16页。IEEE出版社, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00013. 网址 <https://doi.org/10.1109/ICSE48619.2023.00013>.

68. 王岳, 王伟石, Shafiq Joty, Steven C.H. Hoi. CodeT5: 面向标识符的统一预训练编码器-解码器模型, 用于代码理解与生成。在Marie-Francine Moens, 黄轩晶, Lucia Specia, Scott Wen-tau Yih (编辑), *2021年自然语言处理中的经验方法会议论文集*, 第8696–8708页, 线上与多明各共和国蓬塔卡纳, 2021年11月。计算语言学协会. doi: 10.18653/v1/2021.emnlp-main.685. 网址 <https://aclanthology.org/2021.emnlp-main.685>.

69. 王岳, 洪乐, Akhilesh Gotmare, Nghi Bui, Junnan Li, Steven Hoi. CodeT5+: 面向代码理解与生成的开源大型语言模型。在Houda Bouamor, Juan Pino, Kalika Bali (编辑), *2023年自然语言处理中的经验方法会议论文集*, 第1069–1088页, 新加坡, 2023年12月。计算语言学协会. doi: 10.18653/v1/2023.emnlp-main.68. 网址 <https://aclanthology.org/2023.emnlp-main.68>.

70. Martin Weyssow, 周鑫 (Xin Zhou), 金秀 (Kisub Kim), Lo大卫 (David Lo), Sahraoui Houari. 探索大规模语言模型在代码生成中的参数高效微调技术. *arXiv* 电子预印本, 编号arXiv:2308.10462, 2023年8月. doi: 10.48550/arXiv.2308.10462.

71. Martin Weyssow, 周鑫 (Xin Zhou), 金秀 (Kisub Kim), Lo大卫 (David Lo), Sahraoui Houari. 探索大规模语言模型在代码生成中的参数高效微调技术, 2024年. 网址 <https://arxiv.org/abs/2308.10462>.

72. Jie Xu, Karthikeyan Saravanan, Rogier van Dalen, Haaris Mehmood, David Tuckey, and Mete Ozay. Dp-dylora: Fine-tuning transformer-based models on-device under differentially private federated learning using dynamic low-rank adaptation, 2024. URL <https://arxiv.org/abs/2405.06368>.

73. Guang Yang, Yanlin Zhou, Chi Yu, and Xiang Chen. Deepsc: Source code classification based on fine-tuned roberta. In *International Conference on Software Engineering and Knowledge Engineering*, 2021. URL <https://api.semanticscholar.org/CorpusID:238259172>.

74. Wu Yetao. Synthetic python problems. <https://huggingface.co/datasets/wuyetao/spp>, 2023. Accessed: 2024-10-02.

75. Aston Zhang, Yi Tay, Shuai Zhang, Alvin Chan, Anh Tuan Luu, Siu Cheung Hui, and Jie Fu. Beyond fully-connected layers with quaternions: Parameterization of hypercomplex multiplications with 1/n parameters. *ArXiv*, abs/2102.08597, 2021. URL <https://api.semanticscholar.org/CorpusID:231942691>.

76. Zixiao Zhao and Fatemeh Fard. Do current language models support code intelligence for r programming language? *ArXiv*, abs/2410.07793, 2024. URL <https://api.semanticscholar.org/CorpusID:273233233>.

77. Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Yue Zhang, Neil Gong, and Xing Xie. Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts. In *Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis*, LAMPS ’24, page 57–68, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400712098. doi: 10.1145/3689217.3690621. URL <https://doi.org/10.1145/3689217.3690621>.

78. Terry Yue Zhuo and Armel Zebaze. Astraios: Parameter-efficient instruction tuning code large language models, 0 2024.

A Synthetic Dataset Prompt For Robustness Experiment

Listing 1 Synthetic Dataset generation prompt

```
Your job is to rewrite doc string to mimic a real-world user who is not proficient in english. Please
rewrite the sentence to include misspellings, typos, and grammar mistakes. Then while keeping the
errors, rephrase it so that it conveys the same idea but sounds different from the original version
.

Only change the description of the function in the doc string. Do not change the example usecases in the
doc string. Do not change the function signature, body or the name.
Only generate the modified doc string and function header. Do not add extra text. The output should have
the same format as the input.

Example input for Python:
def is_palindrome_string(test_str):
    """ Check if the given string is a palindrome or not.
    >>> is_palindrome_string("radar")
    True
    >>> is_palindrome_string("racecar")
    True
    >>> is_palindrome_string("raceca")
    False
    """

Example output for Python:
def is_palindrome_string(test_str):
    """ Is gave text palindorm?
    >>> is_palindrome_string("radar")
    True
    >>> is_palindrome_string("racecar")
```

72. Jie Xu, Karthikeyan Saravanan, Rogier van Dalen, Haaris Mehmood, David Tuckey, 和 Mete Ozay. Dp-dylora: 在差分隐私联邦学习下，基于动态低秩适应的端设备微调Transformer模型，2024年。网址 <https://arxiv.org/abs/2405.06368>。

73. Guang Yang, Yanlin Zhou, Chi Yu, 和 Xiang Chen. Deepsc: 基于微调Roberta的源代码分类。在软件工程与知识工程国际会议，2021年。网址 <https://api.semanticscholar.org/CorpusID:238259172>。

74. Wu Yetao. 合成Python问题。 <https://huggingface.co/datasets/wuyetao/spp>, 2023年。访问时间: 2024-10-02。

75. Aston Zhang, Yi Tay, Shuai Zhang, Alvin Chan, Anh Tuan Luu, Siu Cheung Hui, 和 Jie Fu. 超越全连接层的四元数：具有1/n参数的超复数乘法参数化。 *ArXiv*, abs/2102.08597, 2021年。网址 <https://api.semanticscholar.org/CorpusID:231942691>。

76. Zixiao Zhao 和 Fatemeh Fard. 目前的语言模型是否支持R编程语言的代码智能? *ArXiv*, abs/2410.07793, 2024年。网址 <https://api.semanticscholar.org/CorpusID:273233233>。

77. Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Yue Zhang, Neil Gong, 和 Xing Xie. Promptrobust: 评估大型语言模型在对抗性提示下的鲁棒性。收录于第一届大规模AI系统与模型的隐私与安全分析研讨会论文集, LAMPS ’ 24, 第57-68页, 纽约, 美国, 2024年。计算机协会。ISBN 9798400712098. doi: 10.1145/3689217.3690621。网址 <https://doi.org/10.1145/3689217.3690621>。

78. Terry Yue Zhuo 和 Armel Zebaze. Astraios: 参数高效的指令调优代码大规模语言模型，2024年。

用于鲁棒性实验的合成数据集提示

Listing1 合成数据集生成提示

```
def is_palindrome_string(test_str): """Check if the string given is a palindrom or not.>>> is_palindrome_string("radar") True>>>
is_palindrome_string("racecar") True>>> is_palindrome_string("raceca") False"""

Large Language Model参数高效方法的实证研究：代码与知识迁移到R
```

```
True
>>> is_palindrome_string("raceca")
False
"""

Example input for R:
# A quadratic cutoff that goes to zero smoothly at the cutoff boundary.
# Args:
#   r_cut (float): Cutoff value (in angstrom).
#   ri (float): Interatomic distance.
#   ci (float): Cartesian coordinate divided by the distance.
# Returns:
#   (float, float): Cutoff value and its derivative.
quadratic_cutoff <- function(r_cut, ri, ci) {

Example input for R:
# A curve drops down gradually to nothing right at the edge.
# Args:
#   r_cut (float): Cutoff value (in angstrom).
#   ri (float): Interatomic distance.
#   ci (float): Cartesian coordinate divided by the distance.
# Returns:
#   (float, float): Cutoff value and its derivative.
quadratic_cutoff <- function(r_cut, ri, ci) {
```

Input

B Sample Generations of Robustness Experiment

Listing 2 Python - LoRA - Original Sample

```
1 def count_Dict(input_dict,key):
2     """ Count the number of occurrences of the
3     given key in a dictionary.
4     1
5     >>> count_Dict({'a':1,'b':2,'c':3}, 'a')
6     1
7     >>> count_Dict({'a':1,'b':2,'c':3}, 'd')
8     0
9     >>> count_Dict({'a':1,'b':2,'c':3}, 'b')
10    2
11    """
12    return (input_dict.get(key,0))
```

Listing 3 Python - LoRA - Modified Sample

```
49 def count_Dict(input_dict,key):
50     """ How many time teh key apears in th dict
51     ?
52     1
53     >>> count_Dict({'a':1,'b':2,'c':3}, 'a')
54     1
55     >>> count_Dict({'a':1,'b':2,'c':3}, 'd')
56     0
57     >>> count_Dict({'a':1,'b':2,'c':3}, 'b')
58     2
59     """
60    count = 0
61    for i in input_dict:
62        if i == key:
63            count += 1
64    return count
```

Listing 4 Python - Compacter - Original Sample

```
1 def find_first_missing_positive(arr):
2     """ Find the smallest positive number
3     missing from the given array.
4     >>> find_first_missing_positive([2, 1, 3])
5     4
6     >>> find_first_missing_positive([1, 2, 0])
7     3
8     >>> find_first_missing_positive([1, 2, 3,
9     4])
10    5
11    """
12    arr.sort()
13    for i in range(len(arr)):
14        if arr[i] != i + 1:
15            return i + 1
16    return len(arr) + 1
```

Listing 5 Python - Compacter - Modified Sample

```
1 def find_first_missing_positive(arr):
2     """ Finde smalest missin pozitive numbr in
3     array give.
4     >>> find_first_missing_positive([2, 1, 3])
5     4
6     >>> find_first_missing_positive([1, 2, 0])
7     3
8     >>> find_first_missing_positive([1, 2, 3,
9     4])
10    5
11    """
12    for i in range(1, len(arr)+1):
13        if i not in arr:
14            return i
15    return len(arr)+1
```

True>>> 是回文字符串("raceca") False

R的示例输入：#在截止边界平滑地趋于零的二次截止

quadratic_cutoff <- 函数(r_cut, ri, ci) {

R的示例输入
一条曲线在边缘处逐渐下降至无

r_cut (float)：截断值（单位：埃） # ri (float)：原子间距离 # ci (float)：笛卡尔坐标除以距离 # 返回值： # (float, float)：截断值及其导数 quadratic_cutoff <- function(r_cut, ri, ci) {

大规模语言模型参数的高效验证研究：代码与知识迁移到R

B 鲁棒性实验的样本生成

Listing2 Python - LoRA - 原始示例

```
1 def count_Dict(input_dict,key):""" 统计字典中给定键的出现次数。"""count_Dict({' a' :1,' b' :2,' c' :3}, ' a' )
2 count_Dict({' a' :1,' b' :2,' c' :3}, ' d' )count_Dict({' a' :1,' b' :2,' c' :3}, ' b' )return (input_dict.get(key,0))
3
4
5
6 0
7
8 2
9
10
```

Listing3 Python - LoRA - 修改示例

```
49 def count_Dict(input_dict, key): """ 统计键在字典中出现的次数 """ count = 0 for i in input_dict: if i == key:
50 count += 1 return count
51
52 1
53
54 0
55
56 2
57
58
59
60
61
62
```

Listing4 Python - Compacter - 原始样本

```
1 def find_first_missing_positive(arr): """ 查找给定数组中缺失的最小正整数。 """ find_first_missing_positive( )
2 find_first_missing_positive( )
3 find_first_missing_positive( 4 ) arr.sort() for i in
4 range(len(arr)): if arr[i] != i: return i return len(arr)
5
6 3
7
8 5
9
10
11
12
13
14
```

Listing 5Python - Compacter - 修改样本

```
1 def find_first_missing_positive(arr): """在数组中找到最小的缺失正整数。 """ find_first_missing_positive( )
2 find_first_missing_positive( ) find_first_missing_positive( )
3 for i in range(1, len(arr)): if i not in arr: return i return
4 len(arr)
5 3
6
7 5
8
9
10
11
12
13
```

Listing 6 R - LoRA - Original Sample

```
1 #Assert that actual_value is within range (
  inclusive).
2 range <- function(actual_value, lower_limit,
  higher_limit) {
3   if (actual_value < lower_limit || actual_
    value > higher_limit) {
4     stop(
5       sprintf(
6         "Expected value to be between %s and %s
          , but was %s",
7         lower_limit, higher_limit, actual_value
8       )
9     )
10  }
11  return(actual_value)
12 }
```

Listing 7 R - LoRA - Modified Sample

```
1 # Check actual_vale inside the l o h limits.
2 range <- function(actual_value, lower_limit,
  higher_limit) {
3   if (actual_value >= lower_limit && actual_
    value <= higher_limit) {
4     return(TRUE)
5   } else {
6     return(FALSE)
7   }
8 }
9 range(10, 5, 15)
10 range(10, 15, 5)
11 range(10, 15, 15)
12 range(10, 5, 10)
13 range(10, 10, 5)
14 range(10, 5, 10)
15 range(10, 10, 10)
16 range(10, 10, 15)
17 range(10, 15, 10)
18 range(10, 15, 15)
19 range(10, 15, 10)
20 range(10, 15, 15)
21 range(10, 15, 15)
22 range(10, 15, 15)
23 range(10, 15, 15)
24 range(10, 15
```

Listing 8 R - Compacter - Original Sample

```
1 # Extracts all digits from incomig string
2 get_num <- function(x) {
3   x <- gsub("[^0-9]", "", x)
4   return(as.numeric(x))
5 }
```

Listing 9 R - Compacter - Modified Sample

```
1 # Pulls out all nums from givven text
2 get_num <- function(x) {
3   nums <- c()
4   for (i in 1:length(x)) {
5     if (is.numeric(x[i])) {
6       nums <- c(nums, x[i])
7     }
8   }
9   return(nums)
10 }
```

C Styling Rubrics

C.1 Python Styling Rubric

1. Code Structure and Cleanness (5 points)

- **Unused comments or Code:**
 - Comments must provide information or elaborate. Generation must be clear of comments that provide no additional information.
 - Generated code must be limited to the target function. Generation must be clear of code snippets written outside of the target function.
 - Deduct points for unnecessary comments and code snippets separately.
- **Indentation and Line Length:**
 - Ensure code uses **4 spaces** per indentation level (not tabs).
 - Nested code blocks (e.g., within loops or if statements) must be properly aligned with consistent indentation.
 - Deduct points for inconsistent indentation within the same function, such as mixing tabs and spaces or misaligned code blocks.
 - Each line should be limited to a maximum of **79 characters**.
 - Ensure that long expressions are broken into multiple lines, with proper continuation indentation (use 8 spaces for continuation or align with the opening delimiter of parentheses).
 - Deduct points for unnecessarily long lines or improper line breaks.
- **Blank lines and Bracket placement:**
 - Ensure logical sections within functions are separated by blank lines (e.g., separating the setup of variables from logic processing).

Listing6 R - LoRA - 原始样本

```
1 #断言 actual_value 在范围（包括边界）内。范围 <-
  function(actual_value, lower_limit, higher_limit) { if
2 (actual_value <lower_limit || actual_value >
  higher_limit) { stop( sprintf("预期值应在 %s 和 %s 之间， 但实
3 际值为 %s", lower_limit, higher_limit, actual_value) ) }
4 return(actual_value) }
5
6
7
8
9
10
11
12
```

Listing7 R - LoRA - 修改样本

```
1 # 检查 actual_value 是否在 loh 限制范围内。2 range <-
  function(actual_value, lower_limit, higher_limit) { if
2 (actual_value >= lower_limit && actual_value <=
  higher_limit) { 4 return(TRUE) 5 } else { 6 return(FALSE) 7 } 8 }
3 range(10, 5, 15) 0 range(10, 15, 5) range(10, 15, 15) 2 range(10,
  5, 10) range(10, 10, 5) 4 range(10, 5, 10) 5 range(10, 10, 10) 6
4 range(10, 10, 15) 7 range(10, 15, 10) 8 range(10, 15, 15)
5 range(10, 15, 10) 0 range(10, 15, 15) range(10, 15, 15) 2
6 range(10, 15, 15) range(10, 15, 15) 4 range(10, 15)
7
8
9
10
11
12
```

Listing8 R - Compacter - 原始样本

```
1 # 提取输入字符串中的所有数字get_num <- 函数(x) { x
2 <v12>- gsub("[^0-9]", "",x) return((as.numeric)(x)) }
3
4
5
```

Listing9 R - Compacter - 修改样本

```
1 # 从给定文本中提取所有数字2 get_num <- 函数
2 (x) {3 nums <-c()4 for (i in 1:length(x)) {5 if
  (is.numeric(x[i])) {6 nums <-c(nums, x[i])7
3 }8 }9 return(nums)10 }
```

C 风格评分标准

C.1 Python 编程风格评分标准

1. 代码结构与整洁性（5分）

- **未使用的注释或代码**
 - `!-` ◦ `-->`评论必须提供信息或进行阐述。生成内容必须避免无附加信息的评论。
 - 生成的代码必须限制在目标函数内。生成内容必须避免包含目标函数外部编写的代码片段。
 - 对不必要的注释和代码片段分别扣分。
- **缩进与行长:**
 - 确保代码在缩进级别使用**4个空格**（而非制表符）。
 - 嵌套代码块（例如在循环或条件语句中）必须正确对齐，保持一致的缩进。
 - 在同一函数内因缩进不一致而扣分，例如混用制表符和空格或代码块对齐不正确。
 - 每行应限制在**79 个字符**以内。确保将长表达式拆分为多行，并使用适当的续行缩进（续行使用8个空格，或与括号的起始符对齐）。
 - 对不必要的过长行或不当换行扣分。
- **空行与括号位置:**
 - 确保函数内的逻辑部分之间用空行分隔（例如，将变量的设置与逻辑处理分开）。

- Blank lines should not be overused. One blank line suffices between separate sections in a function.
 - Ensure proper placement of parentheses and brackets for readability (e.g., open brackets should align with the first non-whitespace character of the line for line continuations).
2. Naming Conventions (5 points)
- **Variable Names:**
 - Use snake_case for variable names (e.g., `data_frame`, not `DataFrame` or `dataFrame`).
 - Variable names should be **descriptive and meaningful**, reflecting their purpose in the function. Avoid single-letter variables except in trivial cases (e.g., loop indices like `i`, `j`).
 - Deduct points for using non-descriptive or unclear names, or for inconsistent use of naming conventions.
 - **Function Names:**
 - Ensure function and argument names are in snake_case and clearly reflect what the function does.
 - For functions with side effects, names should indicate the action being performed (e.g., `calculate_sum()` or `print_data()`).
 - Deduct points for unclear or overly generic function names (e.g., `do_stuff()`), or inconsistent use of naming conventions within the same codebase.
3. Code Readability and Best Practices (5 points)
- **Use of spaces:**
 - Ensure there is a consistent and appropriate use of spaces around operators (e.g., `x + y`, not `x+y`).
 - Inside parentheses, brackets, or braces, avoid extraneous spaces (e.g., `foo(x[1], {bar: 2})` instead of `foo( x[ 1 ], { bar : 2 } )`).
 - Deduct points for inconsistent or improper use of spaces that reduce readability.
 - **Use of idiomatic Python:**
 - Prefer Pythonic constructs, such as list comprehensions, where appropriate.
 - Use built-in functions like `sum()`, `any()`, or `max()` instead of loops where applicable.
 - Ensure loops are used efficiently, avoiding unnecessary computations.
 - Deduct points for overly complex or non-idiomatic solutions that could be simplified with standard Python features.

C.2 R Styling Rubric

1. Code Structure and Cleaness (5 points)
- **Unused comments or Code:**
 - Comments must provide information or elaborate. Generation must be clear of comments that provide no additional information.
 - Generated code must be limited to the target function. Generation must be clear of code snippets written outside of the target function.
 - Deduct points for unnecessary comments and code snippets separately.
 - **Indentation and Line Length:**
 - Code must be consistently indented with **2 spaces per level** (not tabs).
 - Nested code blocks (e.g., inside if statements or loops) should maintain consistent indentation.
 - Deduct points for inconsistent indentation, such as mixing tabs and spaces or misaligned code blocks.
 - Limit each line to a maximum of **80 characters** to maintain readability.
 - For long expressions, break the code into multiple lines and align with proper indentation.
 - Ensure line breaks occur at logical points (e.g., after a comma, or before an operator).
 - Deduct points for overly long lines or poor line-breaking practices.
 - **Blank lines and Bracket placement:**
 - Use blank lines to separate logical sections within the function (e.g., separating setup, processing, and return).

- 空行不应过度使用。在函数的不同部分之间，使用一行空行即可。
 - 确保括号和方括号的正确放置以提高可读性（例如，换行时，左括号应与行首第一个非空白字符对齐）。
- 命名规范（5分）
- 变量名称:
- 使用蛇形命名法（例如，`data_frame`，而不是 `DataFrame` 或 `dataFrame`）。
 - 变量名应具有 **描述性和意义**，反映其在函数中的作用。避免使用单字母变量，除非在琐碎的情况下（例如循环索引如 `i`、`j`）。
 - 对于使用非描述性或不清晰的命名，或命名规范不一致的情况，应扣分。
- 函数名称:
- 确保函数和参数名称采用 snake_case，并清楚地反映函数的功能。
- 对于具有副作用的函数，名称应指示所执行的操作（例如，`calculate_sum()` 或 `print_data()`）。
 - 对于不清晰或过于笼统的函数名称（例如，`do_stuff()`），以及命名不一致的情况，扣分。
- g 在相同代码库中的约定。
3. 代码可读性与最佳实践（5分）
- 空格的使用:
- 确保在运算符两侧使用一致且恰当的空格（例如，`x +y`，而非 `x+y`）。
 - 在括号、方括号或大括号内，避免多余的空格（例如，`foo(x[1], {bar: 2})`，而非 `foo(x[ 1 ], {bar : 2})`）。
 - 因空格使用不一致或不当而影响可读性时扣分。
- 使用习惯用语的Python:
- 在适用的情况下，偏好使用 Pythonic 构造，例如列表推导式。
 - 使用内置函数如 `sum()`、`any()` 或 `max()`，而非循环。
 - 确保循环的高效使用，避免不必要的计算。
 - 对于过于复杂或不符合习惯用法的解决方案，应扣分，这些方案本可通过标准Python特性简化。

C.2 R 样式评分标准

1. 代码结构与整洁性（5分）
- 未使用的注释或代码
- 评论必须提供信息或进行阐述。生成内容必须避免无附加信息的评论。
- 生成的代码必须限制在目标函数内。生成内容必须避免包含目标函数外部编写的代码片段。
- 对不必要的注释和代码片段分别扣分。
- 缩进与行长:
- 代码必须使用 **每级2个空格** 进行一致的缩进（不使用制表符）。
 - 嵌套代码块（例如在 if 语句或循环内部）应保持一致的缩进。
 - 因缩进不一致（如混用制表符和空格或代码块对齐不正确）而扣分。
 - 限制每行最多不超过 **80 个字符** 以保持可读性。
 - 对于较长的表达式，将代码拆分为多行，并进行适当的缩进对齐。
 - 确保换行发生在逻辑点（例如逗号之后或运算符之前）。
 - 对于过长的行或不良的换行实践扣分。
- 空行与括号位置:
- 使用空行在函数内分隔逻辑部分（例如，分隔设置、处理和返回）。

- Avoid excessive or inconsistent use of blank lines (one blank line between distinct sections suffices).
 - Place the opening brace { on the same line as the control statement (e.g., `if, for`), with the closing brace aligned with the control statement.
 - Deduct points for inconsistent brace placement or misaligned closing braces.
2. Naming Conventions (5 points)
- **Variable Names:**
 - Variable names should follow the snake_case convention (e.g., `data_frame`, not `dataFrame` or `DataFrame`).
 - Names must be **descriptive and meaningful**, reflecting their role in the function. Avoid single-character names except for loop counters (e.g., `i`, `j`).
 - Deduct points for non-descriptive, vague, or inconsistent naming practices.
 - **Function Argument Names:**
 - Function names and arguments should use snake_case and be **meaningful**, clearly reflecting their role in the function (e.g., `user_data`, `file_path`).
 - Avoid abbreviations unless they are widely understood in context.
 - Deduct points for ambiguous or overly abbreviated argument names.
3. Code Readability and Best Practices (5 points)
- **Spacing:**
 - Use proper spacing around operators for readability (e.g., `x <- y + 2`, not `x<-y+2`).
 - Inside function calls and indexing, avoid extraneous spaces (e.g., `foo(x[1], y)` instead of `foo( x[ 1 ], y )`).
 - Deduct points for inconsistent or improper spacing that reduces readability.
 - **String Handling:**
 - **Single quotes (')** should be used for text where possible, unless there's a compelling reason to use double quotes (e.g., for embedding quotes inside a string).
 - Deduct points for improper or inconsistent use of quotes for strings.
 - **Explicit Return:**
 - Functions should explicitly return a value using the `return()` function, even if the result is the last evaluated expression.
 - Deduct points for relying on implicit returns, as it reduces clarity.

- 避免过度或不一致的空行使用（不同节之间只需一个空行）。
- 将左大括号 { 放在控制语句（例如 `if`、`for`）所在行的同一行，右大括号与控制语句对齐。
- 对于大括号位置不一致或闭合大括号未对齐的情况扣分。

- 命名规范（5 分）
- 变量名称:**
- 变量名应遵循蛇形命名（snake_case）规范（例如，`data_frame`，而非`dataFrame`或`DataFrame`）。
- 命名必须是**描述性且具有意义**的，反映其在函数中的作用。避免使用单字符命名，除非用于循环计数（例如，`i`, `j`）。
 - 对于不具描述性、模糊或不一致的命名习惯，将扣分。

- 函数参数名称:**
- 函数名称和参数应采用 snake_case，并具有**意义明确**，清楚反映其在函数中的作用（例如，`user_data`、`file_path`）。
 - 除非在上下文中广泛理解，否则避免使用缩写。
 - 对于模糊或过度缩写的参数名称，应扣分。

3. 代码可读性与最佳实践（5 分）
- 间距:**
- 使用适当的空格以提高可读性（例如，`x <- y + 2`，而非`x<-y+2`）。
 - 在函数调用和索引中，避免多余的空格（例如 `foo(x[1],y)`，而非 `foo(x[ 1 ],y)`）。
 - 对于不一致或不当的空格导致可读性降低的情况， 扣分。

- 字符串处理:**
- ') 应尽可能使用单引号（ ' ），除非有充分的理由使用双引号（例如，用于在字符串中嵌套引号）。
 - 对字符串引号使用不当或不一致者扣分。

- 明确返回:**
- 函数应明确使用 `return()` 函数返回值，即使结果是最后计算的表达式。
 - 由于依赖隐式返回会降低清晰度，因此扣分。